Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Ho Chi Minh, October 2026 –

Table of Contents

I. Record of Changes 3

II. Software Design Document 4

1. System Design 4

1.1 System Architecture 4

1.2 Package Diagram 8

1.3 Technology Declaration 13

2. Database Design 15

3. Detailed Design 30

3.1 Identity and Session Management 30

3.2 Organization Onboarding and Administration 38

3.3 Private Knowledge and Personal Workspace 51

3.4 Shared Draft Authoring and Reader Selection 56

3.5 Review Governance and Approval 59

3.6 Indexing and Atomic Publication 64

3.7 Published Knowledge Library 68

3.8 Ask Arden with Permission Aware Citations 69

3.9 Installation System Administration and Security Boundaries 73

4. Cross Cutting Constraints and Diagram Index 22

I. Record of Changes

Date	A* M, D	In charge	Change Description
31 Oct 2026	A	Arden team	Arden-specific software design; diagram insertion locations identified.
31 Oct 2026	A	Ngô Quang Khôi	Added state machine diagrams M-01 Organization Invitation and M-02 Organization Membership (3.2.5), M-03 Knowledge Submission (3.5.5) and M-04 Knowledge Index Generation (3.6.4), mapped to the staging schema and code.
31 Oct 2026	A	Ngô Quang Khôi	Added 1.3 Technology Declaration: third-party services, development technologies with versions, source code and DevOps, deployment environments and AI tools, taken from the staging branch.

*A - Added M - Modified D - Deleted

II. Software Design Document

Arden is a customer-controlled organizational knowledge system with web and Windows desktop interfaces. Members keep private notes, prepare separate shared drafts, select readers, request human review, and consult approved publications through a permission-aware AI assistant. This report describes the system as represented by the inspected source, its data relationships, and the principal interactions between UI, Core API, PostgreSQL, the indexing worker and Arden-AI.

Implementation baseline (31 October 2026): Arden staging at 953be5418f5489f14cb01c919cb649c207637ec9; Arden-AI main at cc0b4d2cf042be4d54c92abecd90f8ac7c1da3ec. Arden-Knowledge main at d4020b480a2aa096688aef383857678d22f4cdff provides non-runtime engineering context. The application default main branch is not identical to staging. Source presence, passing tests and confirmed deployment are distinct claims.

Scope boundary: This document describes implemented code-level paths and marks unresolved deployment/integration checks. The graph workspace, connectors, full offline synchronization, durable chat history and automated external actions remain outside the implemented design. The separately discussed feat/flow-4-workspace branch was not available for inspection; WF-04 below is grounded in staging.

1. System Design

1.1 System Architecture

The architecture separates user interaction, domain authorization, canonical storage, and derived AI retrieval. A user never talks directly to Chroma for knowledge access. Core determines the actor, organization, current audiences, review authority, active publication generations and which evidence may be returned.

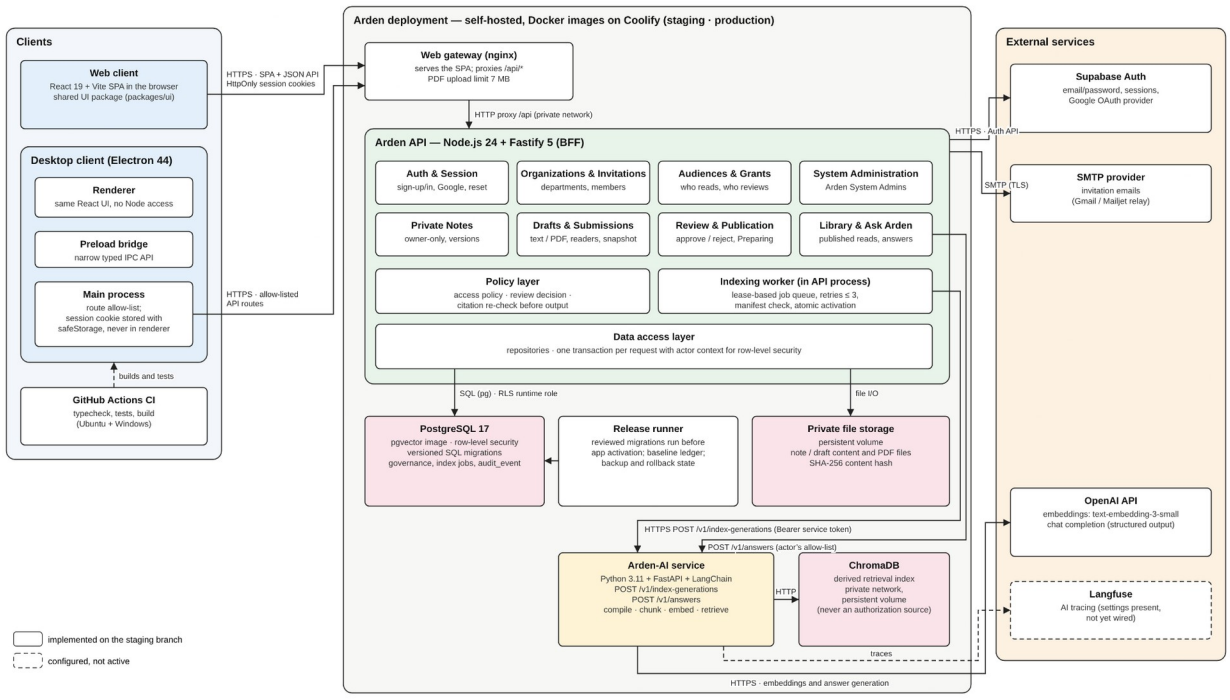


Figure A-01. *Arden system architecture*

Architecture notes: Both clients use the shared @arden/ui components and typed API client. The desktop renderer uses a restricted preload/main-process IPC bridge before reaching the same Core API. Fastify authorizes organization and knowledge operations; PostgreSQL with RLS stores canonical metadata, and the protected volume stores private content bytes. The separately executed worker leases indexing jobs from PostgreSQL, reads approved snapshots, and calls Arden-AI's indexing endpoint. Ask Arden sends a question and a Core-approved generation allow-list to Arden-AI; Core rechecks returned evidence before the response reaches the user. Arden-AI uses Chroma for derived vectors and OpenAI for embedding and answer generation. Supabase Auth and SMTP are Core integrations. This is an implemented source-code architecture, not confirmation that every service has passed live deployment testing.

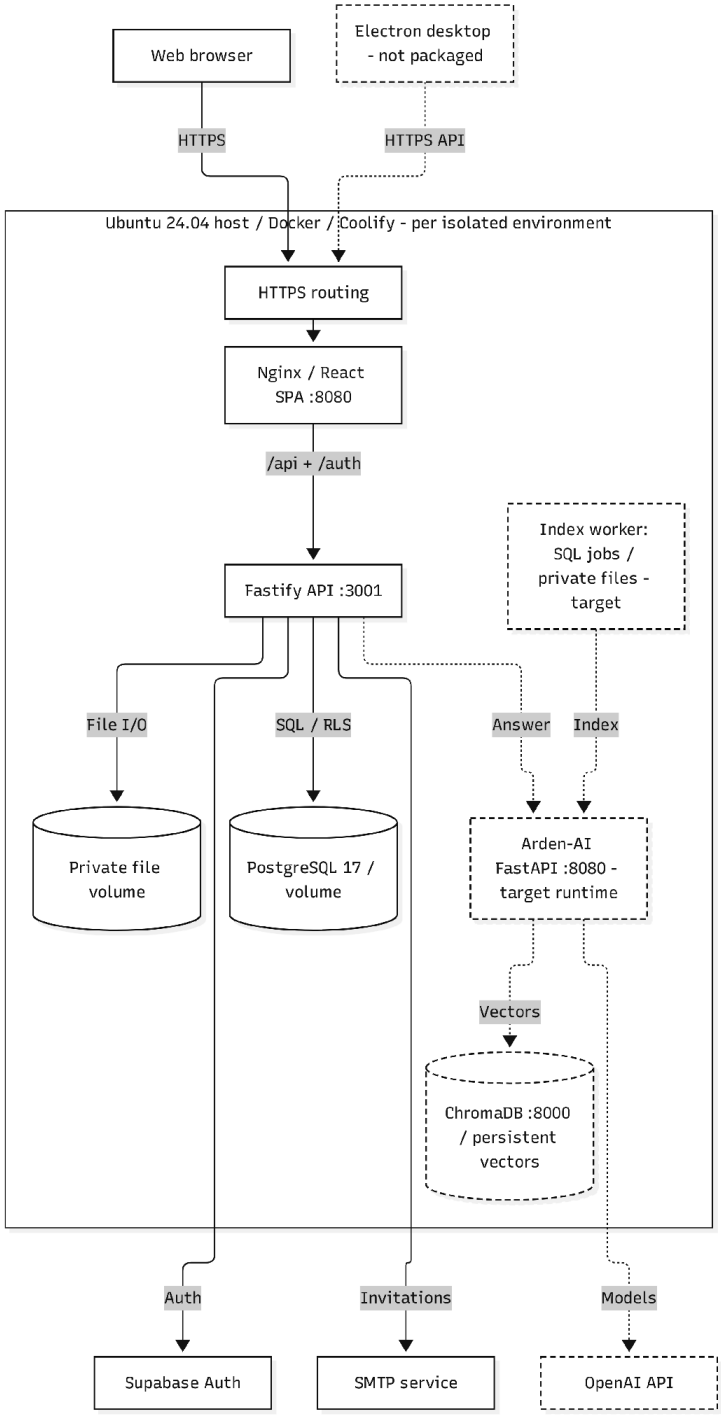


Figure A-03. Target Deployment Topology

Deployment overview: The figure shows one isolated staging or production environment on the Ubuntu/Coolify host. Web and API containers use dedicated PostgreSQL persistence and a protected note file mount. The separately runnable indexing worker, Arden-AI service, and ChromaDB index are shown as target private-network services, not as verified live deployments. Supabase Auth, email delivery and the model API remain separately managed integrations.

Architectural component descriptions

Component	Primary responsibility	Boundary / evidence
Web and Electron clients	Render Island workspace, authentication/onboarding, private notes, shared knowledge, reviews, published library, Ask Arden and administration.	apps/web; apps/desktop; packages/ui
Core API (Fastify)	Authenticate sessions, enforce organization and audience policy, handle draft/review transactions, resolve current published material and validate AI evidence.	apps/api/src/app.ts; policy and route modules
Supabase Auth	Email/password, Google PKCE, recovery and identity session backing; it does not decide Arden document access.	apps/api/src/auth; /api/v1/auth/*
PostgreSQL + RLS	Canonical accounts, membership, private/shared version records, review decisions, publications, indexing/outbox state and audit.	apps/api/migrations/0001–0023
Protected private volume	Store opaque content bytes referenced by UUID URIs and verified using hashes.	apps/api/src/private-notes/content-storage.ts
Index worker	Lease durable jobs, submit approved snapshots to AI, verify manifests and atomically activate eligible generations.	apps/api/src/indexing/worker.ts
Arden-AI (FastAPI)	Compile approved documents, chunk, embed and retrieve from Chroma; prepare grounded answers with evidence identities.	Arden-AI/src/arden_ai/*; /v1/*
Chroma + model provider	Derived vector index and text/embedding inference. Neither grants access to organization data.	Arden-AI infrastructure adapters
SMTP email service	Invitation email delivery through an external SMTP service; failures must not bypass authorization.	apps/api/src/email

Security and consistency principles: Core is the authoritative policy decision point; organization isolation is enforced on server operations and through PostgreSQL RLS; private note bytes are protected separately from shared publication; shared submissions are immutable review candidates; and only index-ready active versions may enter library and answer evidence.

Deployment qualification: The repository records controlled staging and production release automation, but its operational notes explicitly exclude broad claims of live AI, mail, sign-in or whole-product readiness. The topology diagram should therefore show intended components without marking all services as validated production dependencies.

1.2 Package Diagram

The TypeScript application is a monorepo with shared UI/domain packages, while Arden-AI is an independently versioned Python service. UML package diagrams should show source-level package dependencies, not deployment servers or invented object-oriented classes.

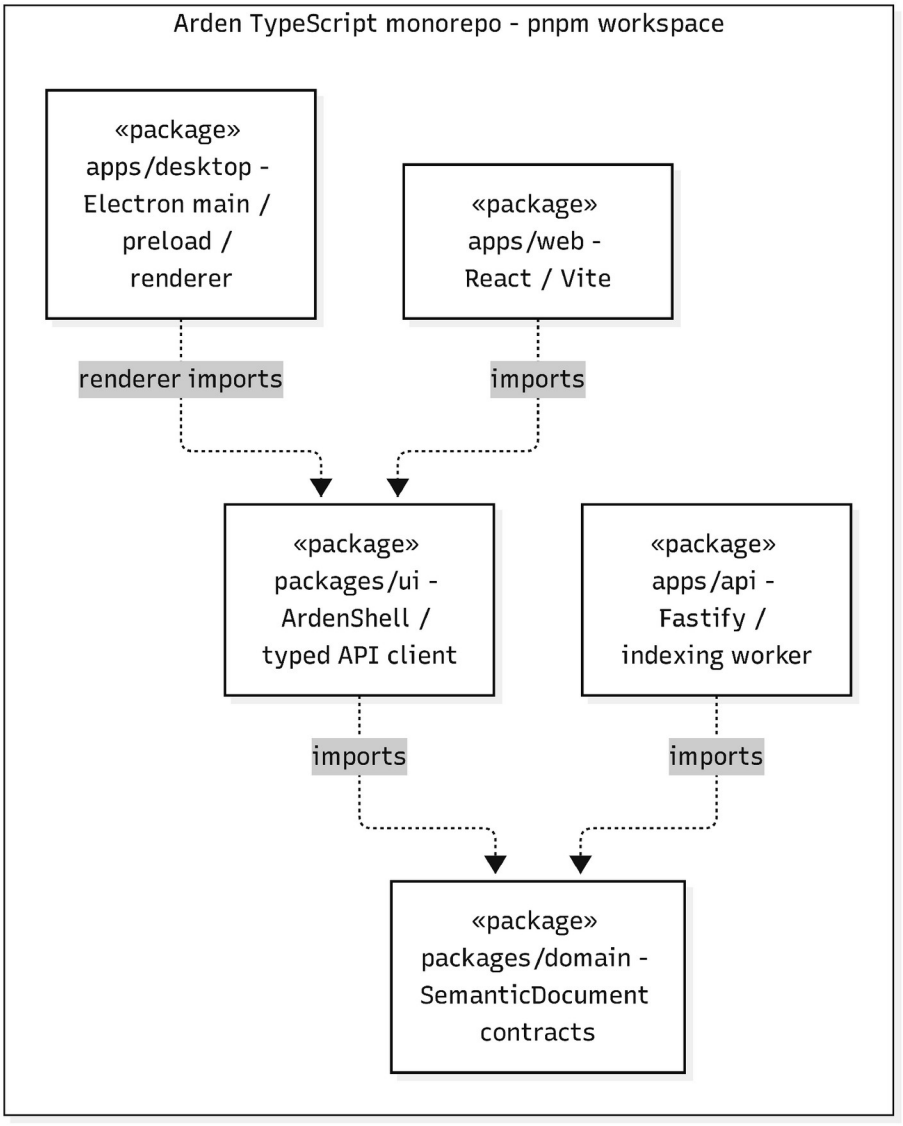


Figure P-01. TypeScript Monorepo Package Dependencies

Package dependency interpretation: The staging pnpm workspace contains three application packages and two shared packages. The web entry and Electron renderer import @arden/ui; Electron main and preload remain internal to apps/desktop. The ArdenShell and typed API client reside inside packages/ui, which imports @arden/domain. The Fastify API also imports @arden/domain for shared semantic-document contracts. Dashed dependency arrows point toward the imported package. Arden-AI is a separate repository, not a TypeScript workspace package.

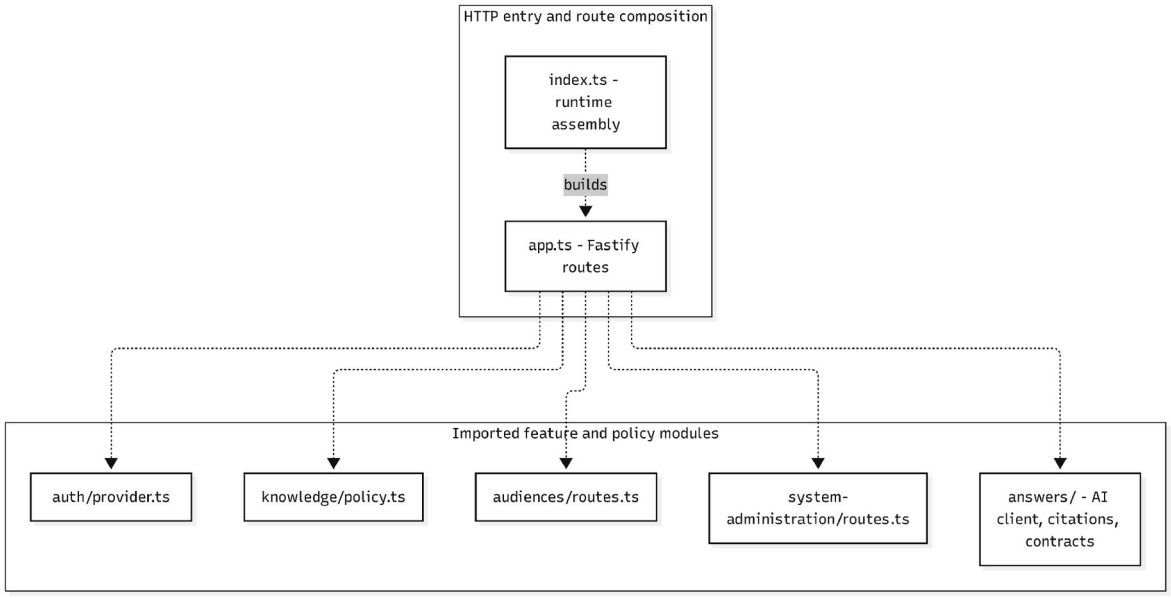


Figure P-02(a). Core API Package Dependencies — HTTP feature-module imports

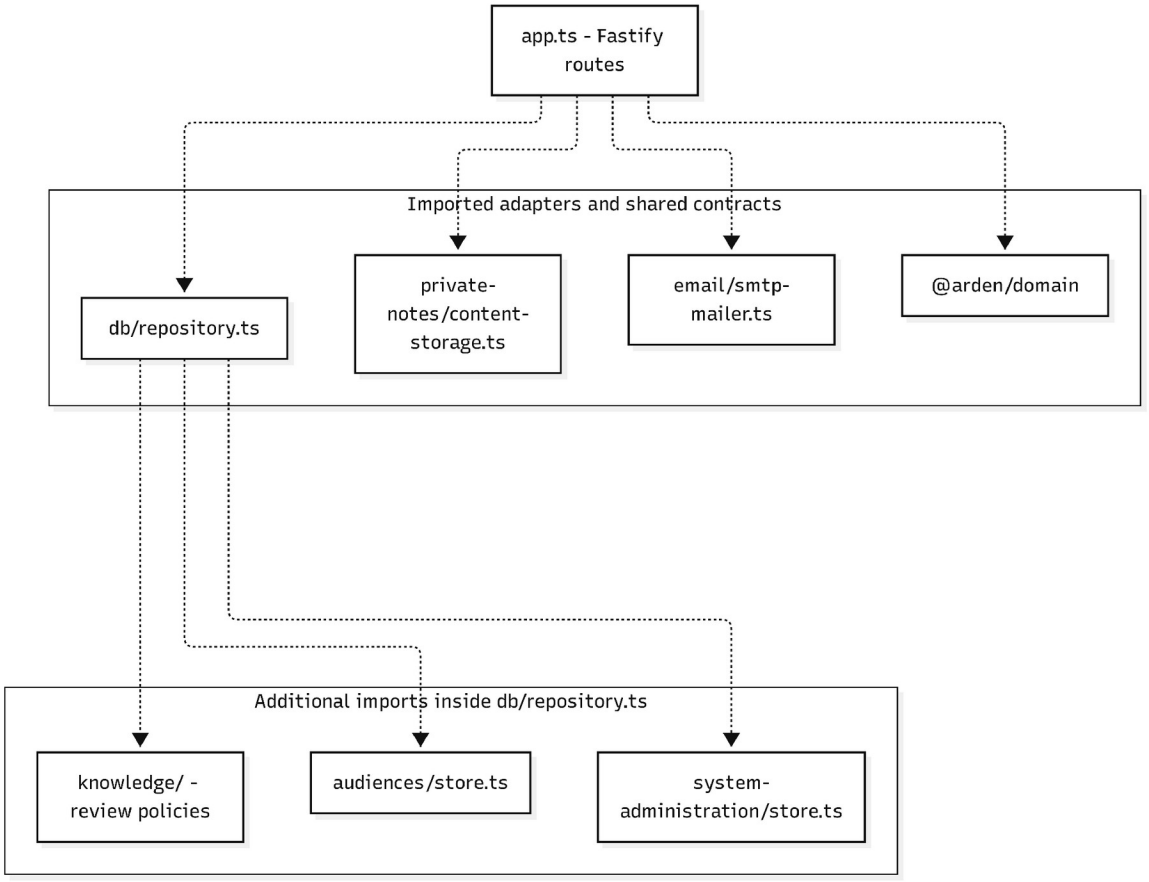


Figure P-02(b). Core API Package Dependencies — Data adapters, contracts and repository helpers

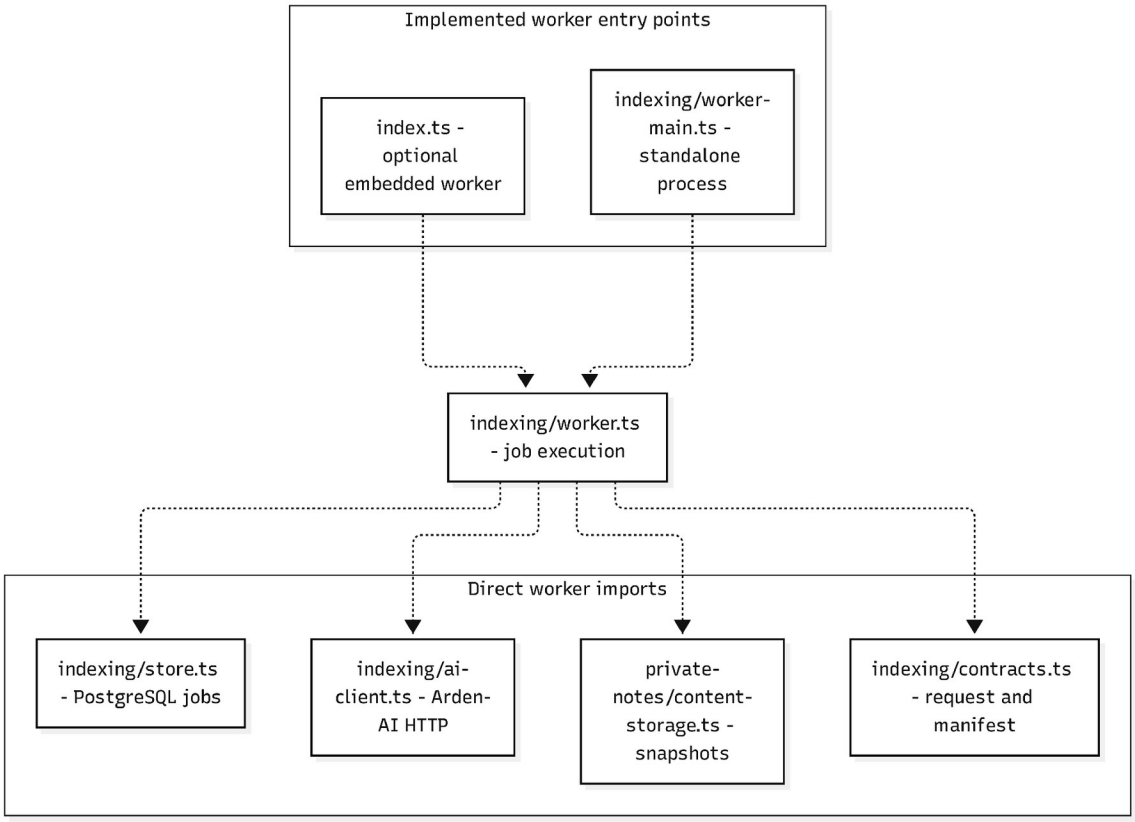


Figure P-02(c). Core API Package Dependencies — Indexing worker source dependencies

Source-level dependency interpretation: The three panels show verified relative imports in Arden staging's apps/api/src. Dashed arrows point from importing files to their dependencies, not between deployment nodes. The HTTP runtime is composed in index.ts, with routes built by app.ts; db/repository.ts reuses review policy and organization stores. IndexingWorker is implemented in indexing/worker.ts and may be started by index.ts when configured or by the independent indexing/worker-main.ts entry. The OpenAI-powered Arden-AI service is reached through existing HTTP adapter modules; it is not an imported TypeScript package.

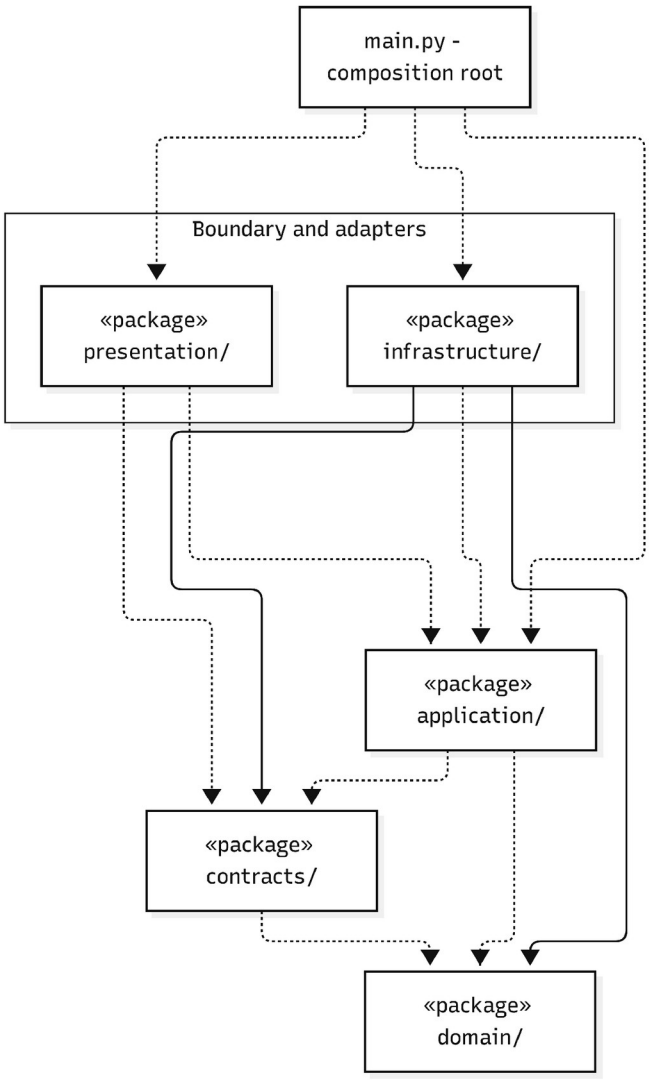


Figure P-03(a). Arden AI Python Package Dependencies — Python package import dependencies

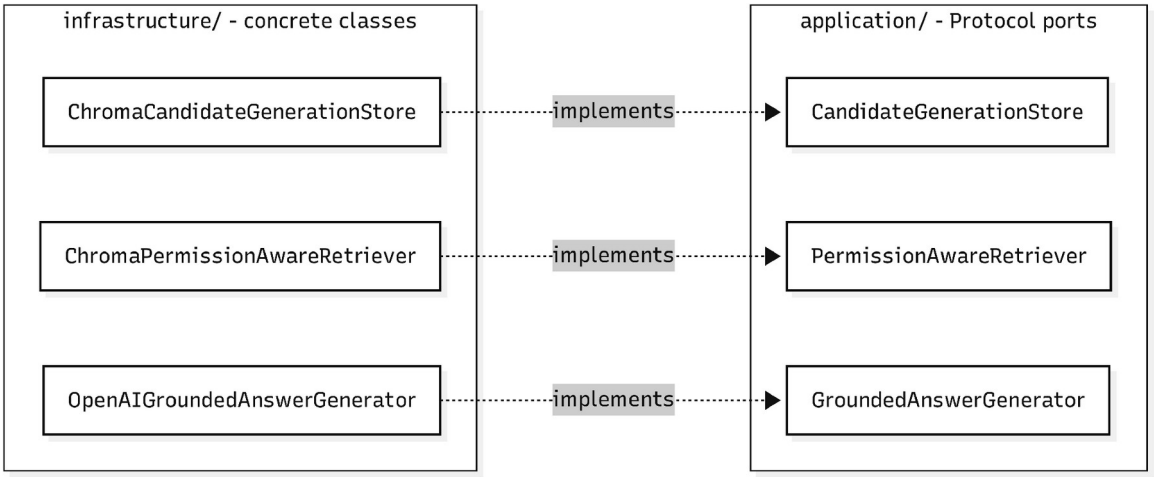


Figure P-03(b). Arden AI Python Package Dependencies — Application ports and concrete implementations

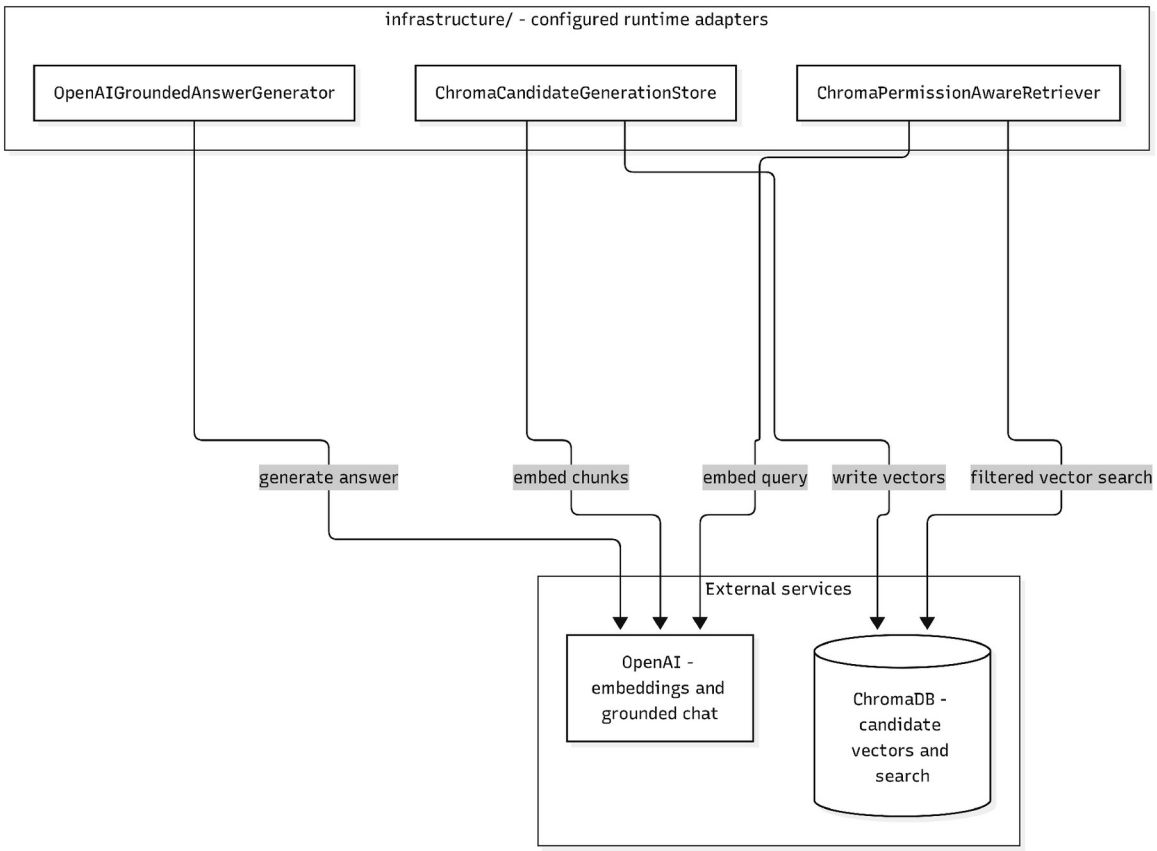


Figure P-03(c). Arden AI Python Package Dependencies — Infrastructure adapters and external services

Python package dependency interpretation: These views follow Arden-AI/main at cc0b4d2. main.py composes FastAPI presentation, application use cases and infrastructure adapters. Presentation handles bearer-token service authentication and versioned request/response contracts; indexing and answering use cases depend on domain compilation/chunking and application-declared provider Protocols. Infrastructure implementations supply Chroma candidate indexing, allow-list-filtered retrieval and OpenAI embeddings/chat. The domain layer remains independent of presentation and infrastructure, a rule checked by tests/test_architecture.py. Langfuse is declared as a package dependency but no Langfuse client or instrumentation is imported in src/arden_ai, so no active telemetry connection is shown.

Package descriptions

No	Package / namespace	Description and dependencies
01	apps/web	Vite browser entry point; mounts the shared interface and talks to Core over authenticated HTTP.
02	apps/desktop	Electron main, preload and renderer; narrow typed IPC and encrypted desktop session storage.
03	packages/ui	React screens: authentication, organization, personal workspace, knowledge, review and Ask Arden.
04	packages/domain	Versioned Semantic Document contract, validation and shared domain formats.
05	apps/api/src/auth	Supabase identity adapter, BFF session, Google PKCE and recovery endpoints.

No	Package / namespace	Description and dependencies
06	apps/api/src/knowledge	Shared draft validation, review eligibility, queue and decision policy.
07	apps/api/src/private-notes	Private-note content storage boundary and integrity checks.
08	apps/api/src/audiences	Published reader audiences and audience-group administration.
09	apps/api/src/indexing	Persistent generation/job store, external AI client and orchestration worker.
10	apps/api/src/answers	Answer contract, AI request/evidence mapping and citation-page policy.
11	apps/api/src/system-administration	Installation-level administrator lifecycle without organization document bypass.
12	apps/api/src/db and email	PostgreSQL transaction/policy context and invitation SMTP adapter.
13	Arden-AI/presentation	FastAPI endpoints, authentication and HTTP error/response mapping.
14	Arden-AI/application	Indexing and answering use cases, orchestration, external provider ports.
15	Arden-AI/domain	Semantic compilation and deterministic source/chunk mapping.
16	Arden-AI/infrastructure	OpenAI and Chroma adapters with runtime settings; implements the application indexing, retrieval and answer-generation ports.
17	Arden-AI/contracts	Cross-repository versioned /v1 indexing and answer request/response schemas.

1.3 Technology Declaration

This section declares the technologies, services, source-control rules, environments and AI tools used by Arden, as required for Review 2. Versions are the exact pins in package.json, pnpm-lock.yaml, the release Dockerfile and Arden-AI uv.lock on the staging branch (commit e9e4b3a) and Arden-AI main (commit cc0b4d2), checked on 31 October 2026.

1.3.1 Third-party services

Service	Purpose	Data sent to the service	Status / control
Supabase Auth	Authentication: email/password registration with email confirmation, sign-in, password reset code, sessions	Email address and password (only through the Core API BFF), Google PKCE code, verification and reset codes. Organization data and content are never sent.	Implemented. Keys stay server-side; Arden keeps its own organization and content authorization.
Google OAuth (via Supabase)	Authentication: Sign in with Google	OAuth redirect and authorization code; Google returns the verified account identity (email) to Supabase.	Implemented (PKCE, browser and Electron).
SMTP mail server (Nodemailer)	Email: organization invitations	Recipient email, organization name, inviter and the single-use invitation link.	Implemented. Delivery failure keeps the invitation Pending (INVITATION_DELIVERY_FAILED).
OpenAI API	AI APIs: embeddings (text-embedding-3-small) and answer generation (default chat model gpt-6-luna, configurable)	Indexing: text chunks of approved, published knowledge only. Ask Arden: the question plus the authorized retrieved chunks. Private notes, drafts, passwords and tokens are not sent.	Implemented, called only by Arden-AI; Core rechecks every citation.
Langfuse	AI tracing / evaluation	None at present.	Configured in Arden-AI settings only; not enabled.

Service	Purpose	Data sent to the service	Status / control
Storage, payment	Not used	No third-party storage or payment service. Files use a protected local volume.	Supabase Storage is planned, not integrated.

1.3.2 Development technologies

Layer	Technology and version	Use in Arden
Front-end (web)	React 19.3.0, Vite 7.3.6, TypeScript 5.9.3, BlockNote 0.55.0, Inter / Inter Tight 5.3.0, Flaticon Uicons 3.3.1	Shared @arden/ui package rendered by apps/web; rich-text shared knowledge editor; nginx stable-alpine serves the build and proxies /api.
Desktop	Electron 44.4.3, electron-vite 5.0.0	Windows desktop client reusing @arden/ui with safeStorage-backed session IPC (not a packaged installer).
Mobile	None	Out of scope.
Back-end (Core API)	Node.js 24.22.1, Fastify 5.12.5, pg 8.23.0, @supabase/supabase-js 2.117.2, Nodemailer 10.0.10, esbuild 0.28.2, pnpm 11.19.0	Domain API, authorization, governance, durable indexing worker and release migration runner.
AI service (Arden-AI)	Python 3.11, FastAPI 0.159.1, Uvicorn 0.54.0, Pydantic 2.14.1, LangChain (langchain-openai 1.7.2, langchain-chroma 1.1.0), pypdf 6.21.2, uv 0.9.27	POST /v1/index-generations and POST /v1/answers: compile, chunk, embed, retrieve and generate grounded answers.
Database	PostgreSQL 17 (pgvector/pgvector:pg17 image), SQL migrations 0001–0023 with row-level security; ChromaDB 1.5.9	PostgreSQL is the system of record; ChromaDB stores vectors per index generation.
Testing and quality	Vitest 5.0.1, pytest, ruff, mypy (strict)	Unit, request, real-PostgreSQL/RLS and Arden-AI contract tests.

1.3.3 Source code and DevOps

Item	Declaration
Platform	GitHub organization Arden-Enterprise: repositories Arden (web, desktop, Core API, deployment) and Arden-AI (Python AI service).
CI	GitHub Actions ci.yml on every push and pull request: frozen install, typecheck, tests and builds on Ubuntu and Windows; a disposable PostgreSQL job that applies all migrations twice and runs the RLS/authorization suite; image build with HIGH/CRITICAL vulnerability and secret scanning. pr-conventions.yml checks branch and PR names.
CD	Push or merged PR to staging builds the web and API images on GitHub, plans and applies eligible migrations, and activates them on staging. A merged promotion PR to main reuses the identical, already-scanned staging images for production.
Branch rules	Work branches are type/kebab-description (feat, fix, docs, refactor, perf, test, chore, build, ci, release, hotfix, security, revert). staging and main are reserved deployment branches: no direct work and no force-push. Feature PRs target staging; only staging is promoted to main. PR titles follow Conventional Commits.
Code review	Feature PRs into staging are reviewed before an authorized merge; configured AI reviewers are CodeRabbit and GitHub Copilot, and their coverage and findings are recorded in the PR template.

1.3.4 Deployment environments

Environment	Where	How it is deployed	Rollback
Development	Local Vite web / Electron client; four hosted API and database slots (dev-1 to dev-4) managed in Coolify; optional local PostgreSQL via Docker Compose	pnpm dev:remote syncs saved back-end files to the developer's own slot and reloads the API; synthetic data only.	Re-sync the slot from Git; slots are independent of staging and production.
Staging	https://staging.ardenai.tech on the team server (Coolify), branch staging	GitHub Actions builds and scans the image pair, uploads it to a restricted receiver that applies eligible expand migrations, then asks Coolify to recreate only the web and API containers.	A failed migration leaves the previous app running. The previous release is retained and an operator runs bootstrap_operator.py rollback staging. No automatic down-migration.
Production	https://ardenai.tech on the same server, branch main	After a staging → main promotion PR, the identical staging images (same Git tree, verified within 24 hours) are promoted.	Same operator rollback to the retained previous release (rollback production).
Arden-AI	Separate Python service with its own ChromaDB, reached by Core at ARDEN_AI_URL with a service token	Built from the Arden-AI repository Dockerfile (uv, Python 3.11); not part of the Arden release workflow.	Not automated: redeploy the previous Arden-AI commit. If indexing fails, Core keeps the current publication active.

1.3.5 AI tools

Tool	Model	Phases used	Evidence and rules
Claude (Claude app / Claude Code)	Claude Opus 5.5	Specification and design (SRS/SDD text, UML diagrams, Design Brief), code, review	Co-Authored-By trailers in Git history and docs/superpowers plans; outputs reviewed by the owning member.
OpenAI Codex	To be confirmed by the team	Code and tests	Commits authored by CodexSandboxOffline; AGENTS.md defines agent rules.
Cursor	To be confirmed by the team	Code	Co-authored-by: Cursor trailers.
CodeRabbit, GitHub Copilot review	Vendor-managed	Pull-request review	.coderabbit.yaml and .github/copilot-instructions.md.

AI usage rules: Every member is responsible for, and must be able to explain, all content in their owned area, including AI-generated content. Only synthetic data is given to AI tools; no real partner data, personal data, passwords, API keys or secrets.

2. Database Design

The canonical relational model is defined by the ordered SQL migrations apps/api/migrations/0001–0023. Several initial tables retain compatibility columns that newer migration policies supersede; the final physical schema must be interpreted after all ordered migrations. Chroma is an external derived index and is deliberately not drawn as a relational authoritative table.

2.1 Entity Relationships

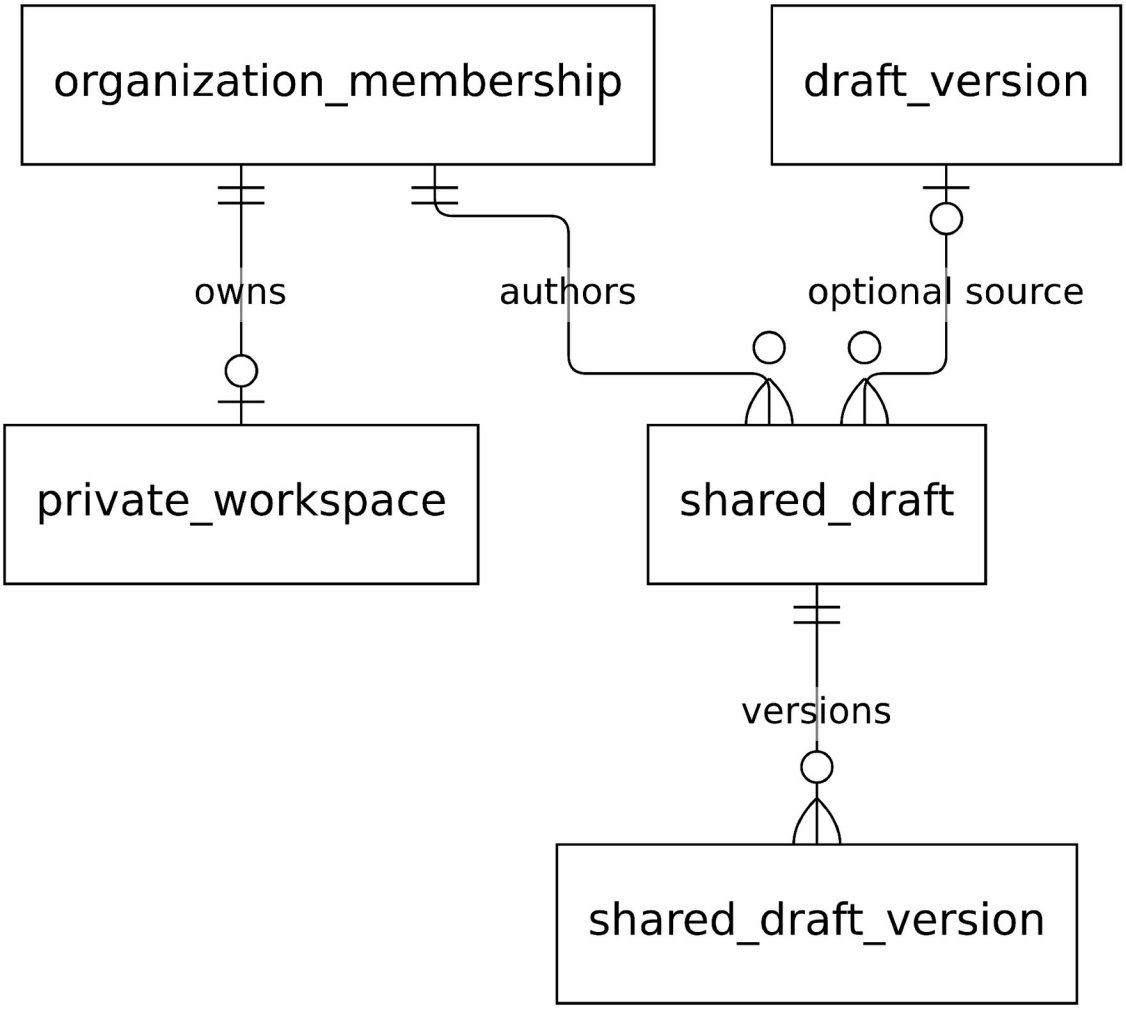


Figure D-01(a). Database Domain Overview ERD — Member-private origin and shared authoring

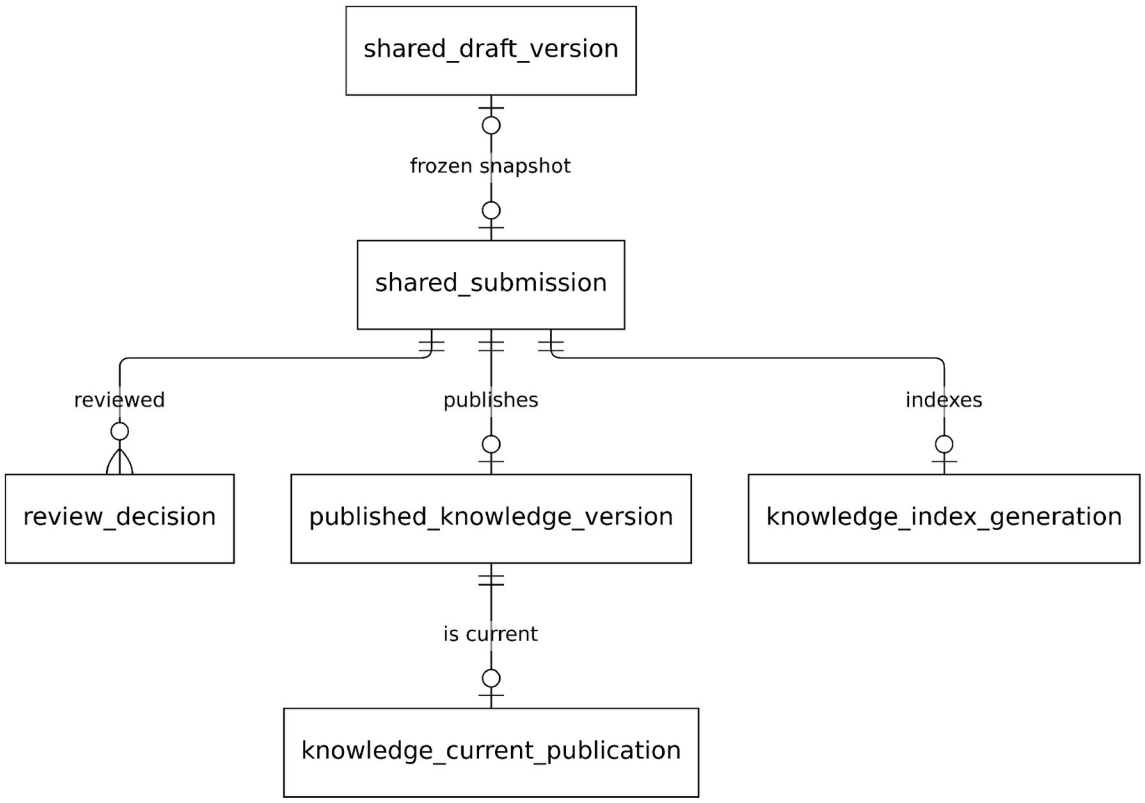


Figure D-01(b). Database Domain Overview ERD — Submission, review, publication, and index generation

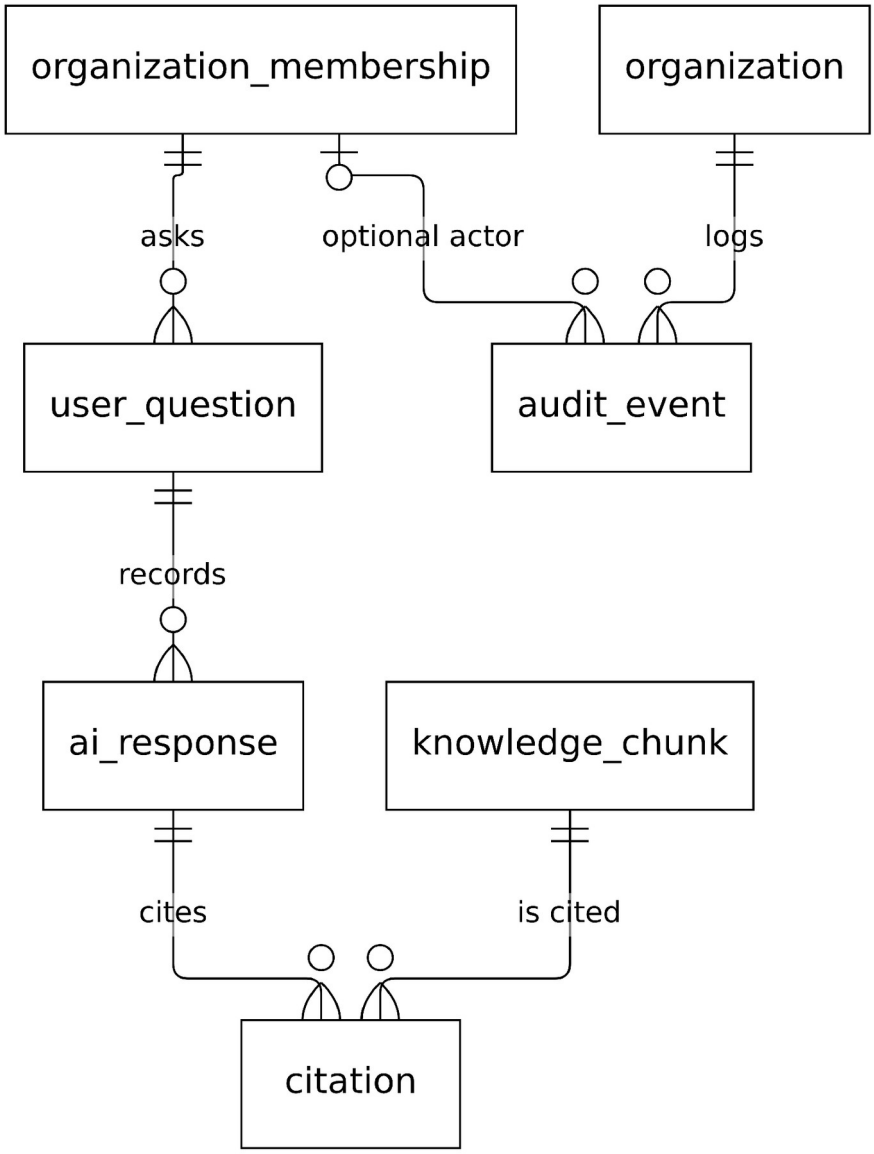


Figure D-01(c). Database Domain Overview ERD — Historical answer evidence and organization audit

Database overview and source: Read these three panels as a database-domain overview rather than a single exhaustive physical schema. The entities and crow's-foot relationships come from Arden/staging migrations 0001–0023. Private originals and shared copies are distinct. The current search/indexing path is represented by knowledge_index_generation, while user_question, ai_response, citation and knowledge_chunk are retained historical relational tables. The specialized D-02 to D-05 figures expose their primary keys and foreign keys in detail.

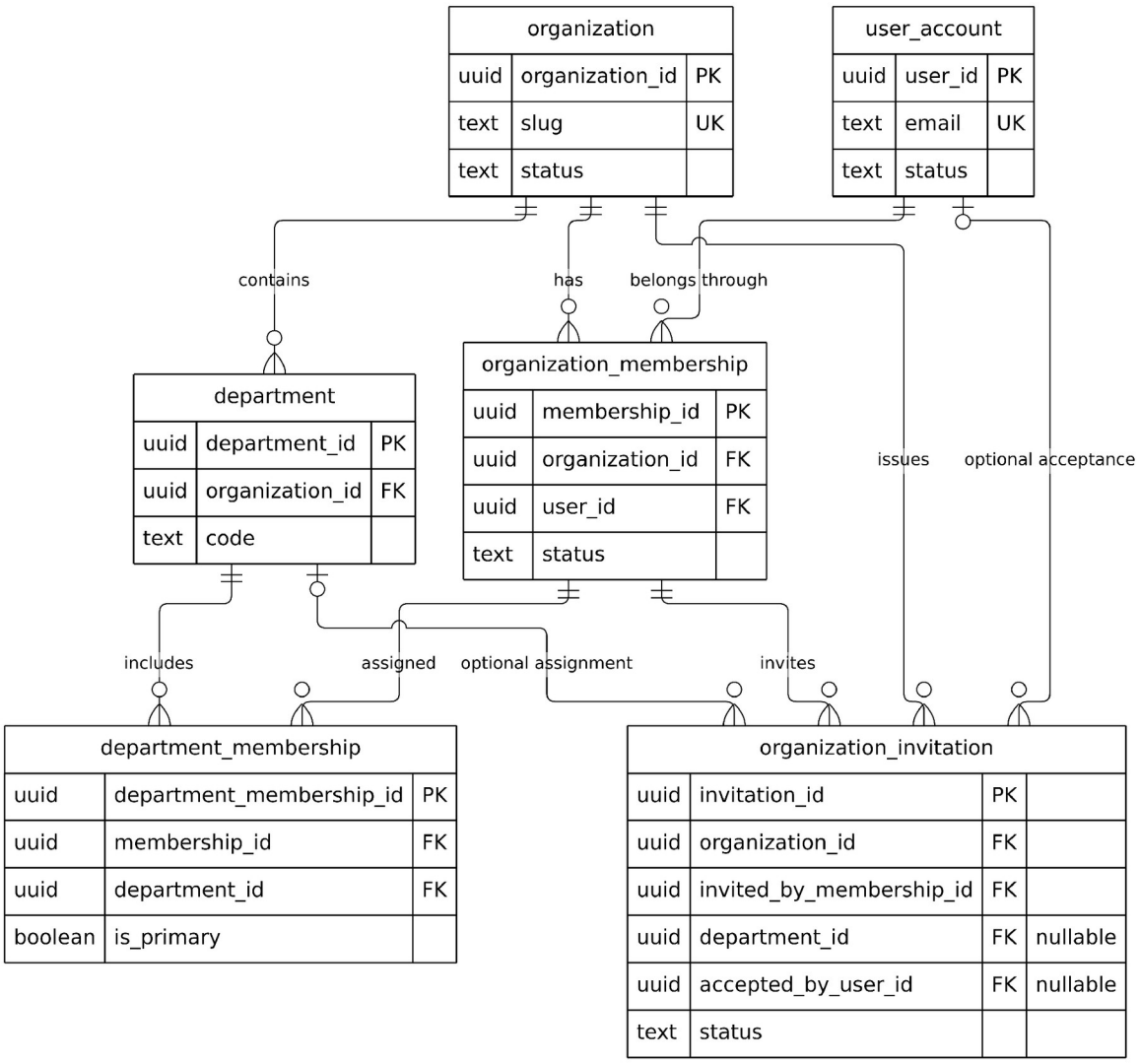


Figure D-02(a). Identity Organization and Membership ERD — Users, organizations, departments, memberships, and invitations

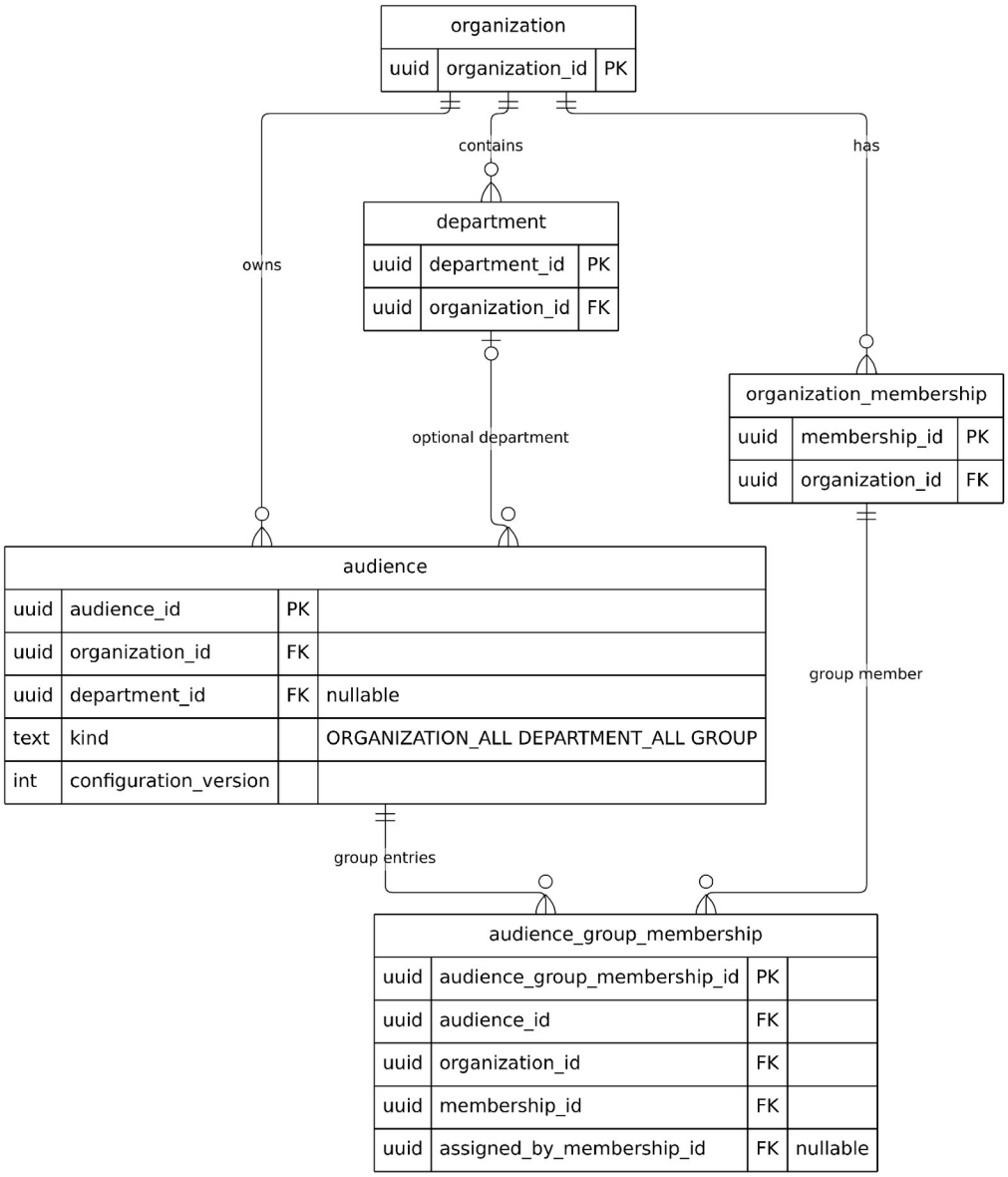


Figure D-02(b). Identity Organization and Membership ERD — Managed audiences, custom groups, and their members

Identity and audience constraints: A user may belong to multiple organizations through organization_membership; department membership is a separate association. Organization invitations reference an inviter with (invited_by_membership_id, organization_id) and optionally a department with (department_id, organization_id). Migration 0023 explicitly permits a NULL invitation department. Department-scoped audiences likewise use (department_id, organization_id); group entries use composite (audience_id, organization_id) and (membership_id, organization_id) FKs. These ERDs show the main owner/member link once even where the schema has additional inviter/assigner-role FKs.

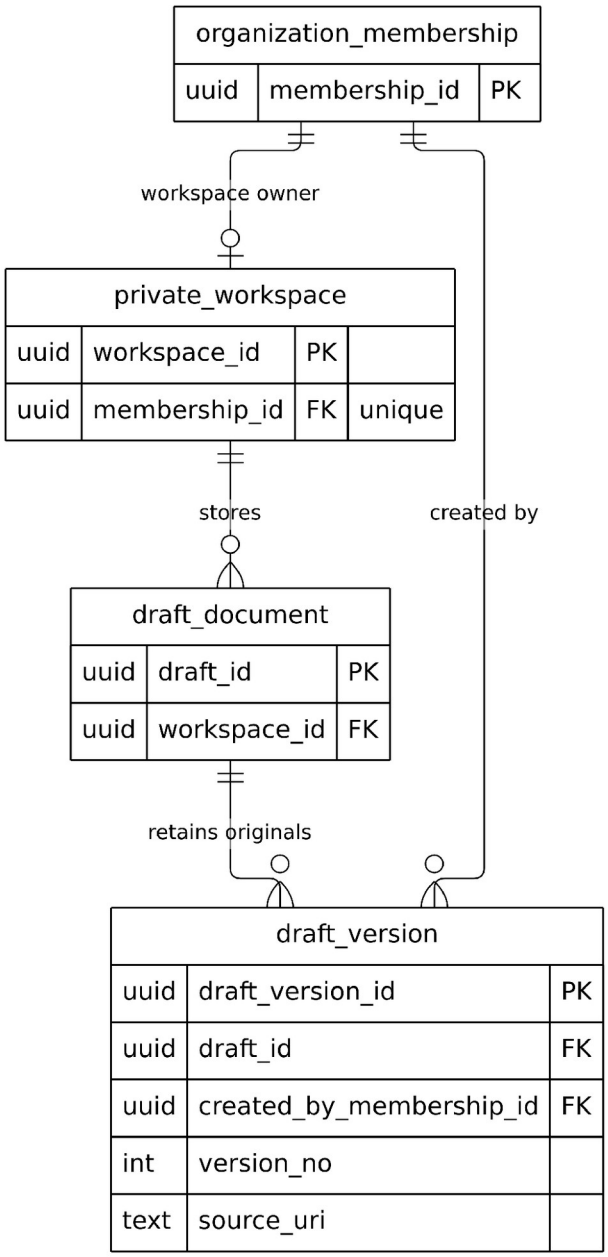


Figure D-03(a). Private Notes and Shared Draft Provenance ERD — Original private workspace, documents, and versions

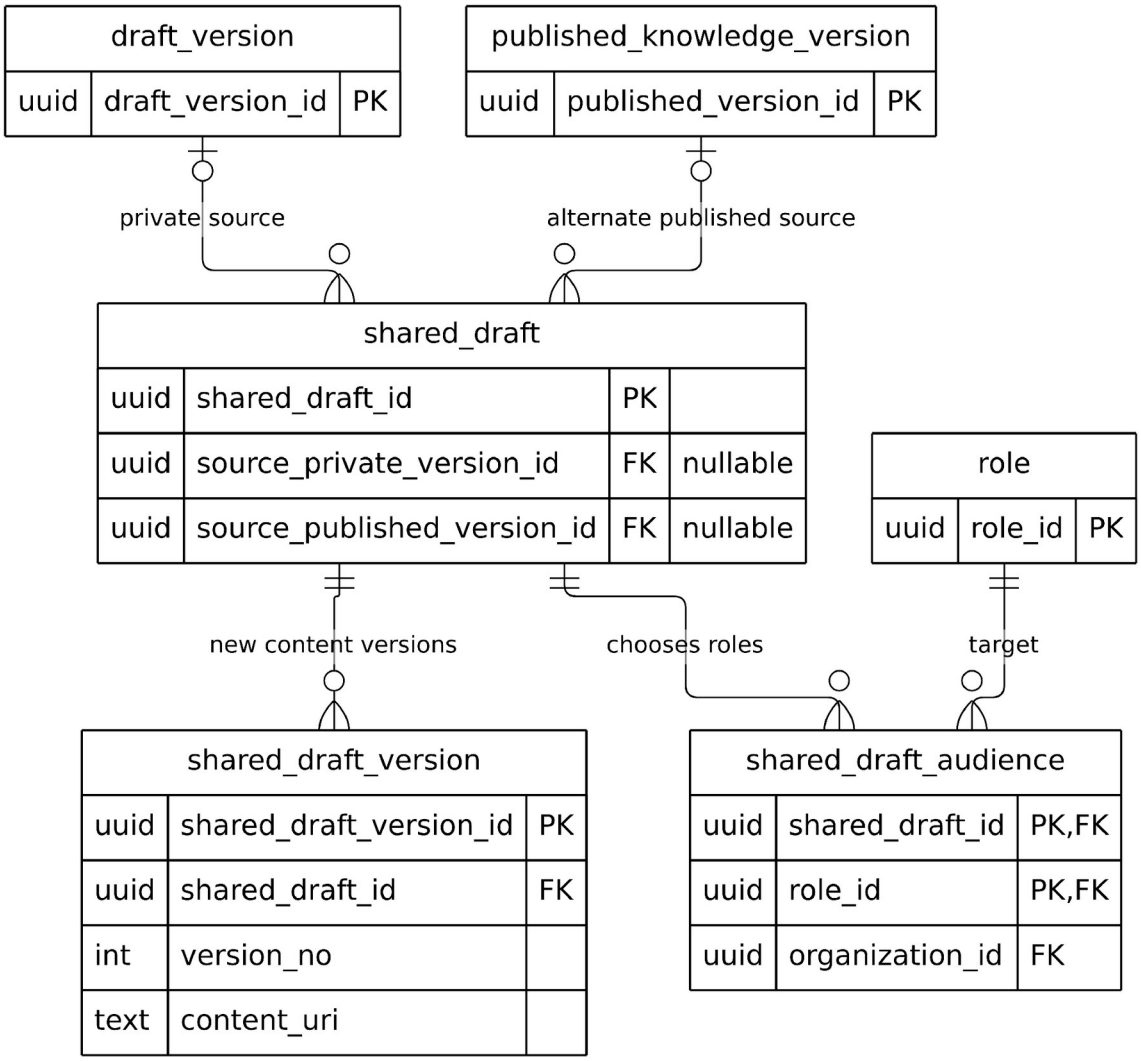


Figure D-03(b). Private Notes and Shared Draft Provenance ERD — Copy provenance, independent shared versions, and target roles

Private versus contributed content: The owner-private chain is private_workspace → draft_document → draft_version. A contributed shared_draft can reference a source_private_version_id, but neither that FK nor the copy operation converts or deletes the private original. A shared_draft may instead reference an earlier published knowledge version, or be authored without either source; a check permits no more than one source. New shared content goes into shared_draft_version. The shared_draft_audience join uses role_id rather than a direct FK to the newer audience table.

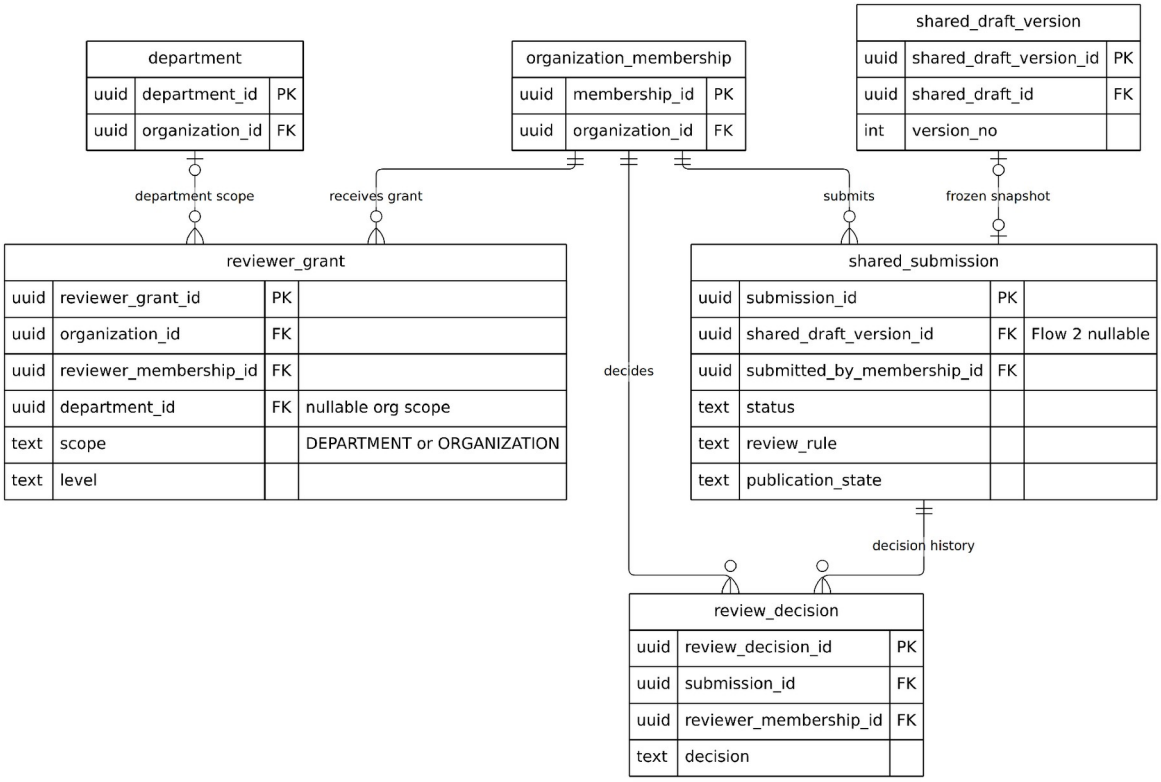


Figure D-04(a). Submission Review and Publication ERD — Immutable submission snapshot, reviewer decisions, and grants

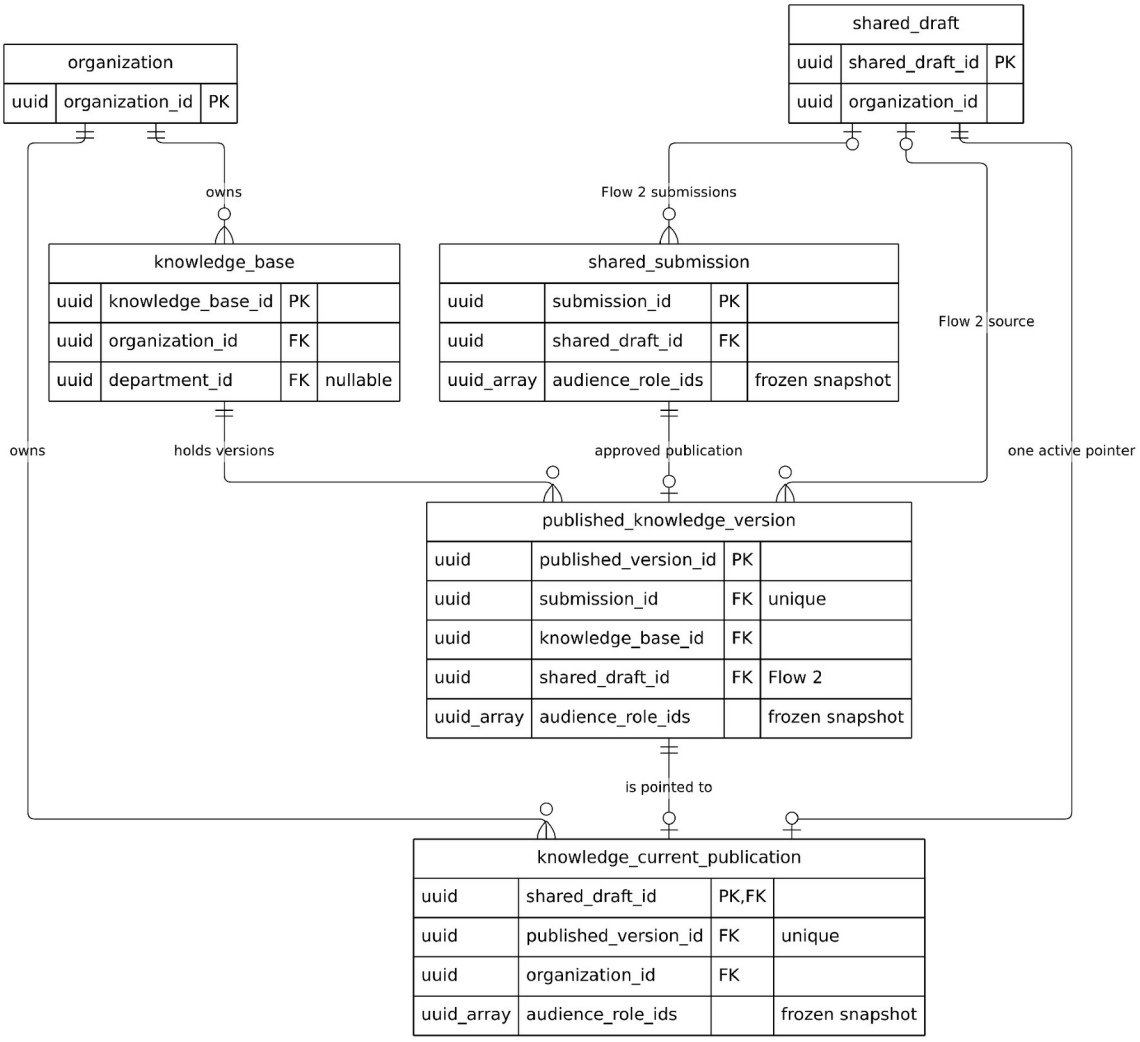


Figure D-04(b). Submission Review and Publication ERD — Approved publication versions and the unique active pointer

Submission, review and publication constraints: Flow 2 shared_submission freezes a referenced shared_draft_version plus policy and audience_role_ids; its original draft remains editable. review_decision references the submission and reviewer membership; the Flow 2 partial unique index permits one Flow 2 decision per submission. reviewer_grant references a reviewer membership and optionally a department (migration 0022 also supports ORGANIZATION scope); it is evaluated by policy and has no direct FK to review_decision or shared_submission. Published versions reference a submission and knowledge_base; knowledge_current_publication holds at most one active published-version pointer per shared_draft via its PK. The frozen audience_role_ids arrays are snapshots, not FKs to audience.

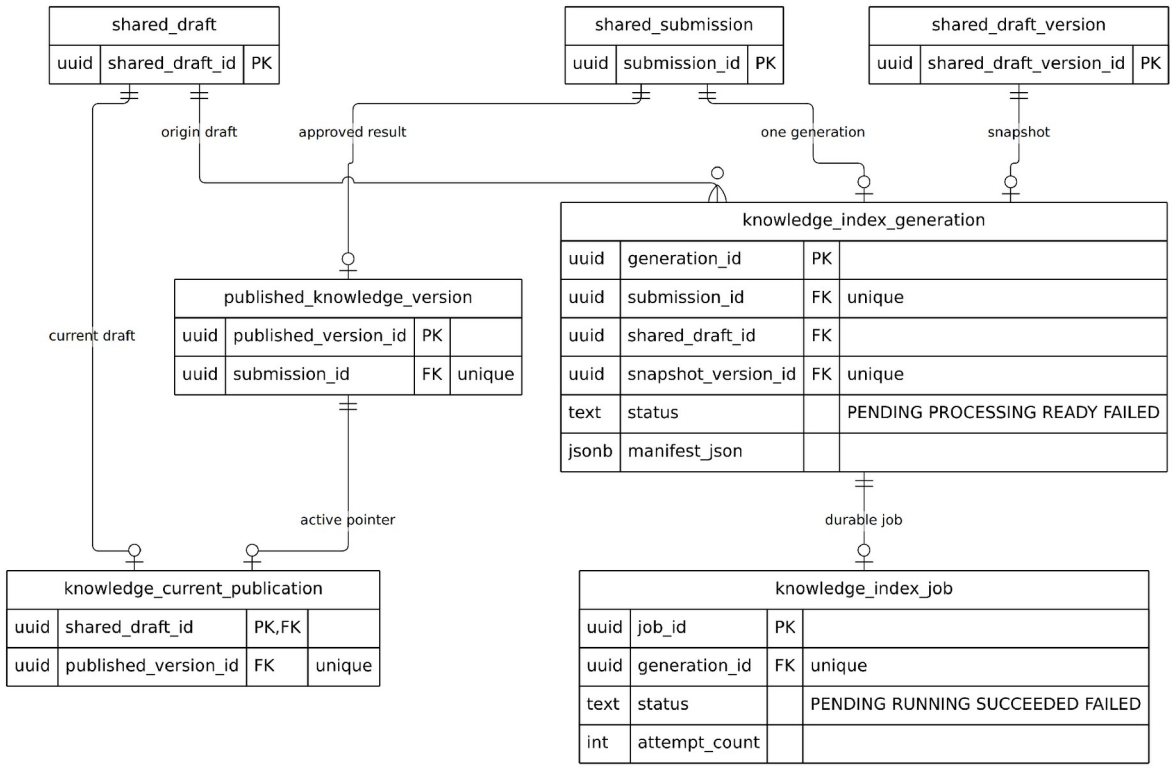


Figure D-05(a). Indexing Answer Evidence and Audit ERD — Durable index generation, job, and publication FKs

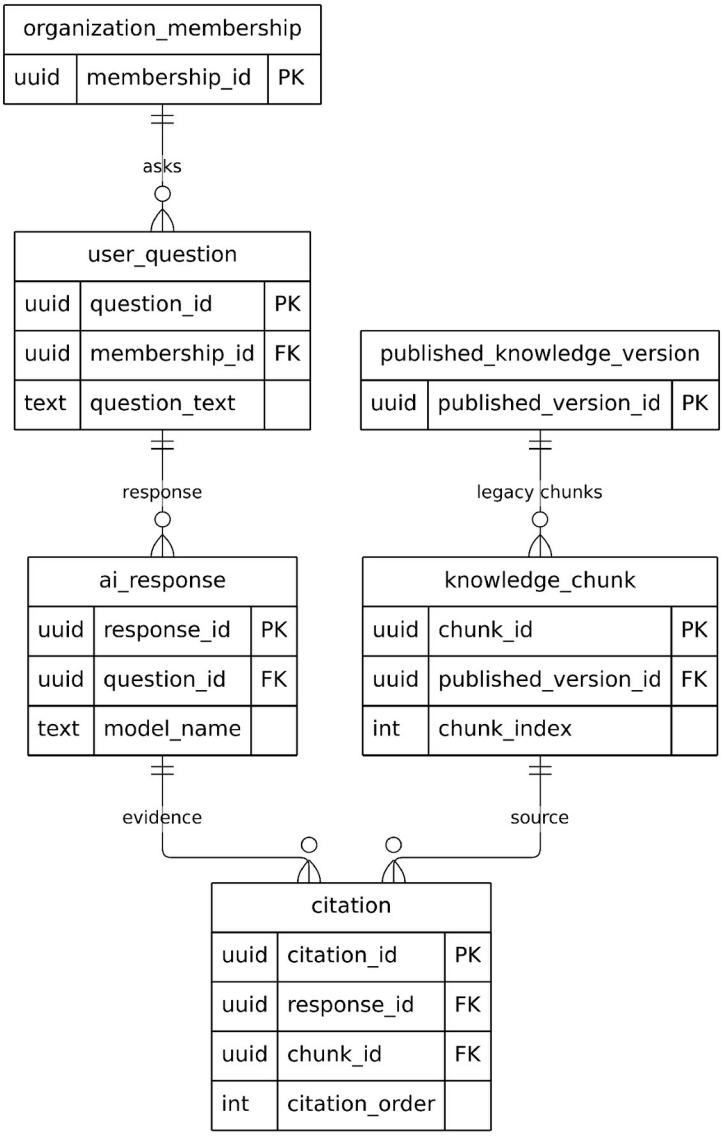


Figure D-05(b). Indexing Answer Evidence and Audit ERD — Historical relational Q&A and citation evidence

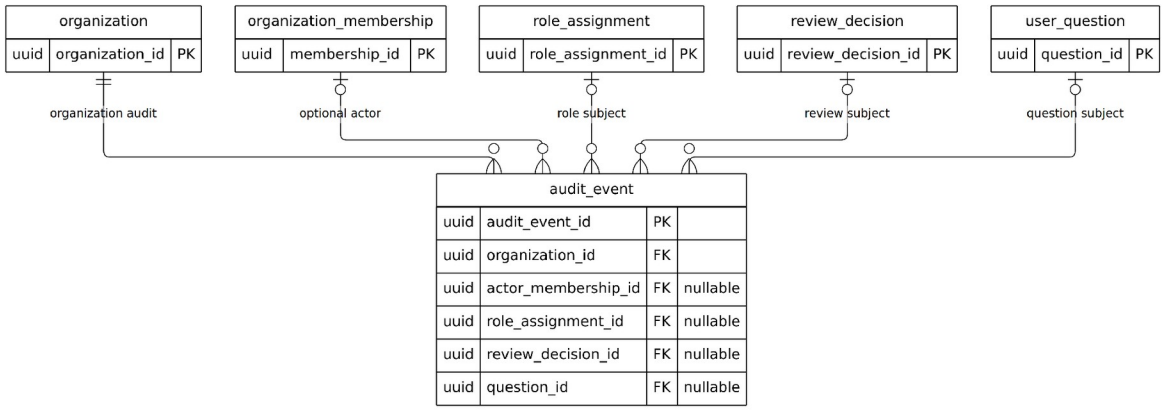


Figure D-05(c). Indexing Answer Evidence and Audit ERD — Organization audit and nullable audit subjects

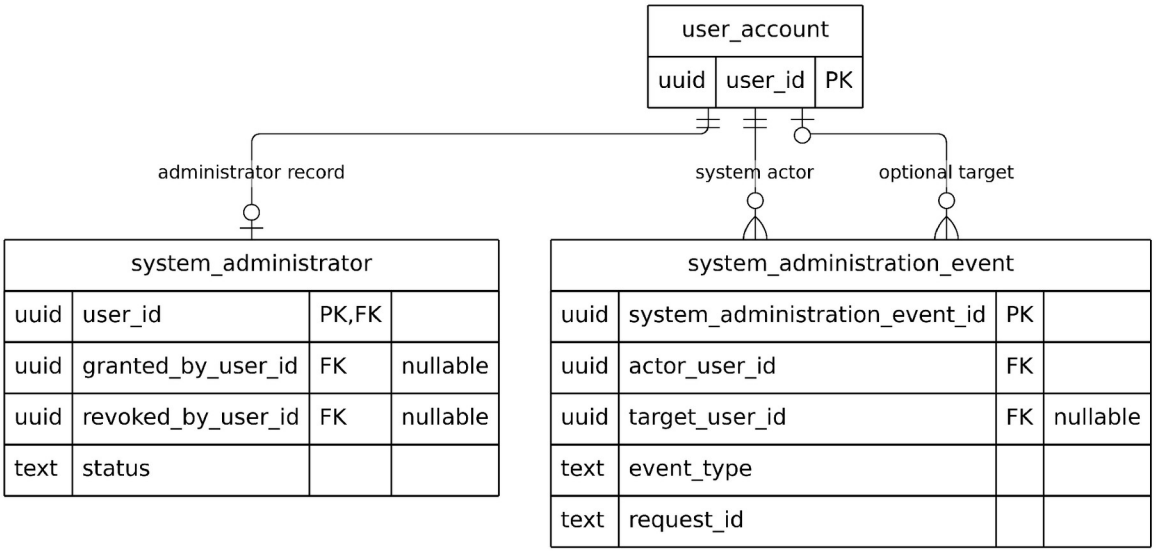


Figure D-05(d). Indexing Answer Evidence and Audit ERD — System-administrator records and independent system audit

Index, evidence and audit boundaries: knowledge_index_generation is unique by submission and snapshot; its knowledge_index_job has a unique generation FK. Runtime authorization is performed by Arden Core before a READY generation identifier can be used for retrieval; ChromaDB is an external, derived vector index and has no relational foreign key here. The user_question → ai_response → citation → knowledge_chunk chain is a retained compatibility schema, not proof that the current answer endpoint writes to it. audit_event records organization-scoped audit subjects, while system_administration_event records separate system-level events. Audit subject FKs are optional and the schema limits role/review/question subject columns to at most one non-NULL value.

2.2 Database Table Descriptions

Notation: PK means primary key; FK means foreign key. Rows below highlight central identifiers and relationships rather than reproducing every physical column. The live DDL and database migration order remain authoritative for full indexes, nullability, checks, policies and transitional compatibility fields.

No	Table	Description, key relationships and design role
01	user_account	PK user_id. Local Arden identity projection mapped to Supabase Auth user UUID; status and profile.
02	organization	PK organization_id. Tenant boundary; slug and lifecycle status.
03	department	PK department_id; FK organization_id -> organization. Organization-scoped department code and status.
04	organization_membership	PK membership_id; FKs organization_id and user_id. Unique member within an organization; active/suspended/left states.
05	department_membership	PK department_membership_id; FKs membership_id and department_id. Department participation and lifecycle; primary department maintained by newer membership contract.
06	organization_invitation	PK invitation_id; FKs organization_id, inviter membership, optional department_id and accepted user. Hashed token lifecycle; department optional after migration 0023.

No	Table	Description, key relationships and design role
07	audience	PK audience_id; FK organization_id and optionally department_id. Organization, department and explicit group readership, with configuration version.
08	audience_group_membership	PK audience_group_membership_id; FKs audience_id and membership_id. Explicit group membership with activation/revocation history.
09	private_workspace	PK workspace_id; FK membership_id. One owner-private workspace for an organization membership.
10	draft_document	PK draft_id; FK workspace_id. Owner's private note and its current version number.
11	draft_version	PK draft_version_id; FKs draft_id and created_by_membership_id. Immutable private note revision metadata with content hash/reference.
12	shared_draft	PK shared_draft_id; FKs organization_id, author_membership_id, optional source_private_version_id and source_published_version_id. Shared working copy and provenance.
13	shared_draft_version	PK shared_draft_version_id; FK shared_draft_id. Immutable versioned shared content, MIME, content URI and hash.
14	shared_draft_audience	Composite mapping keyed by shared_draft_id and legacy role_id. The current API uses canonical audiences, with role-backed compatibility retained by migration 0017.
15	shared_submission	PK submission_id; FKs shared_draft_id, shared_draft_version_id and submitting membership. Frozen target scope, audience, hash and status; publication_state progresses separately.
16	reviewer_grant	PK reviewer_grant_id; FKs organization, reviewer membership, granter and optional department. Current DEPARTMENT/ORGANIZATION authority with active dates and tags.
17	review_decision	PK review_decision_id; FKs submission_id and reviewer_membership_id. Immutable Approve/Reject decision and comment/audit linkage.
18	knowledge_base	PK knowledge_base_id; FK organization_id, optional department_id. Domain container for approved knowledge versions.
19	published_knowledge_version	PK published_version_id; FKs submission_id and knowledge_base_id. Publication/version record; older publisher identity field is legacy-compatible.
20	knowledge_current_publication	PK shared_draft_id; FK published_version_id (unique), organization_id. Active pointer and frozen readership; the library and AI use this current publication.
21	knowledge_index_generation	PK generation_id; FKs submission_id (unique), shared_draft_id and snapshot_version_id. Status PENDING/PROCESSING/READY/FAILED plus manifest.
22	knowledge_index_job	PK job_id; FK generation_id (unique). Durable queue, attempts, lease owner/expiry and terminal status.
23	knowledge_chunk	PK chunk_id; FK published_version_id. Historical relational chunk metadata; current live retrieval uses AI-owned Chroma generations.
24	user_question	PK question_id; FK membership_id. Schema-level question record; not proof of durable chat-history UI.
25	ai_response	PK response_id; FK question_id. Schema-level response record with status/model metadata.
26	citation	PK citation_id; FKs response_id and chunk_id. Historical citation lineage; live answer evidence is rechecked by Core.
27	audit_event	PK audit_event_id; FK organization_id, optional actor membership/review decision/question. Governance and security event stream.

No	Table	Description, key relationships and design role
28	system_administrator	PK user_id; FK user_account. Installation capability and revocation metadata, separate from organization membership.
29	system_administration_event	PK system_administration_event_id; FKs actor_user_id and optional target_user_id. Installation-level audit trail.
30	role, permission, role_permission	Legacy RBAC compatibility schema. New product permission design uses Member/Org Admin, ReviewerGrant and audiences; do not interpret these rows as current custom-role UI.
31	access_scope, role_assignment, authorization_policy	Compatibility-era scope/policy tables retained for migration and historical RLS bridges; current runtime authorization must follow newer migration policies.

2.3 Constraints Security and Transaction Rules

1. Every shared knowledge read and mutation checks authenticated organization membership, owner/ReviewerGrant/Org Admin authority where relevant, and present audience membership for published readership. RLS is a second enforcement layer, not a replacement for request validation.
2. An immutable shared_draft_version is the content source for a frozen shared_submission. Editing the author's working draft after submission creates another version without modifying the pending snapshot.
3. At most one open pending submission per draft; unique idempotency and version constraints prevent duplicate creation and lost updates. If-Match version checks return a precondition/conflict response.
4. Approve records the decision and PREPARING state. The durable worker produces a complete verified index manifest before a transaction activates the current-publication pointer. FAILED leaves the prior active pointer in place; SUPERSEDED prevents stale generations replacing newer content.
5. Reader access is not implied by review, administrator role or content authorship. Revoked membership/audience access must change library and Ask Arden visibility immediately when Core recomputes authorization.

3. Detailed Design

Each feature area identifies the user intent and screen, the implementing modules, the internal interaction, and failure/authorization cases. Class figures below are UML class/module relationship views of real service contracts and data types, not fabricated React component inheritance. Sequence diagrams are in chronological order and show alternate/error paths.

3.1 Identity and Session Management

A person creates an account, signs in and maintains an authenticated Arden session before opening an organization workspace. Supabase provides account identity; Fastify Core owns session transport and maps identity to the Arden user account.

User-facing entry and screens: Sign In / Create Account; Google sign-in redirect; Password recovery and new-password screens; authenticated workspace redirect.

Code evidence: packages/ui/src/sign-in; apps/api/src/auth; apps/api/src/app.ts.

3.1.1 Class Diagram

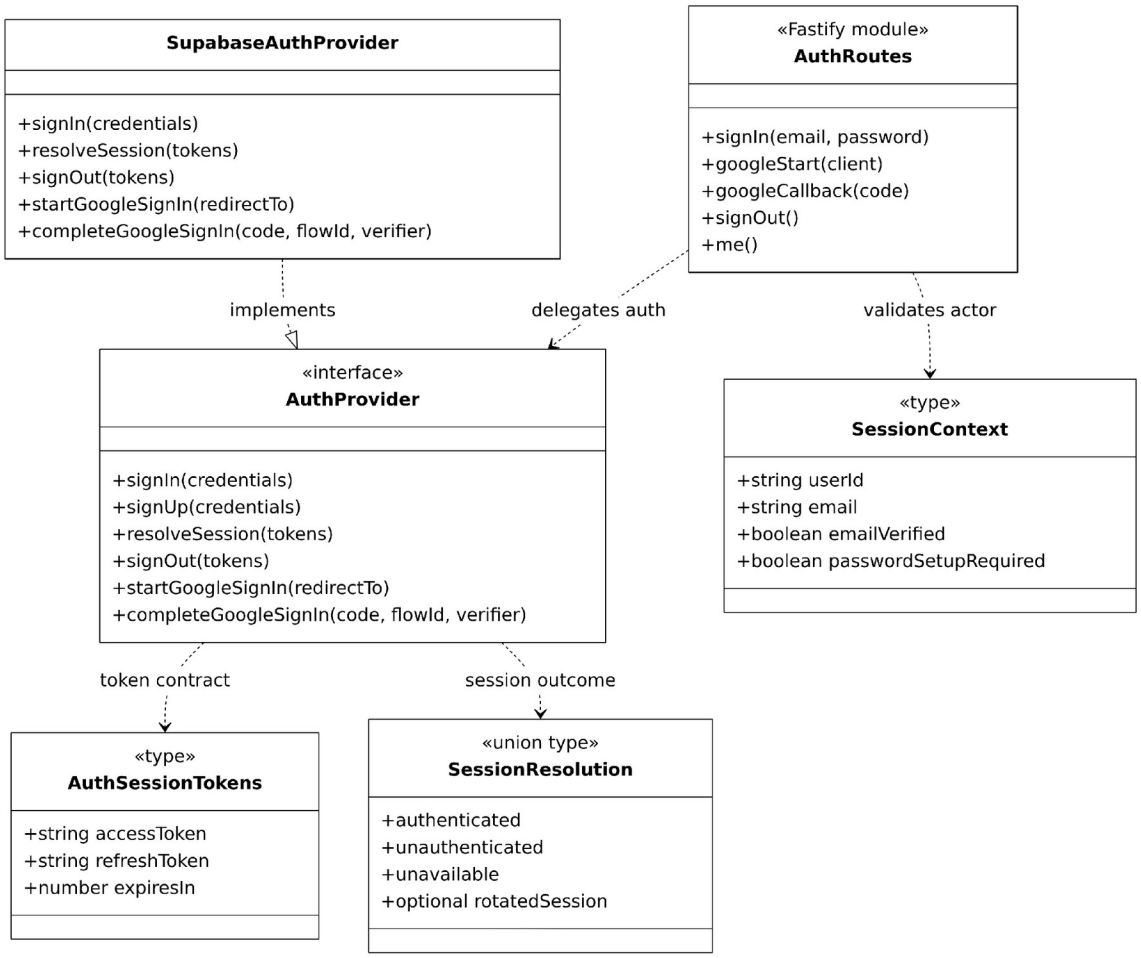


Figure C-01(a). Authentication Organization and Session Contracts — Fastify auth handlers, provider interface, and session types

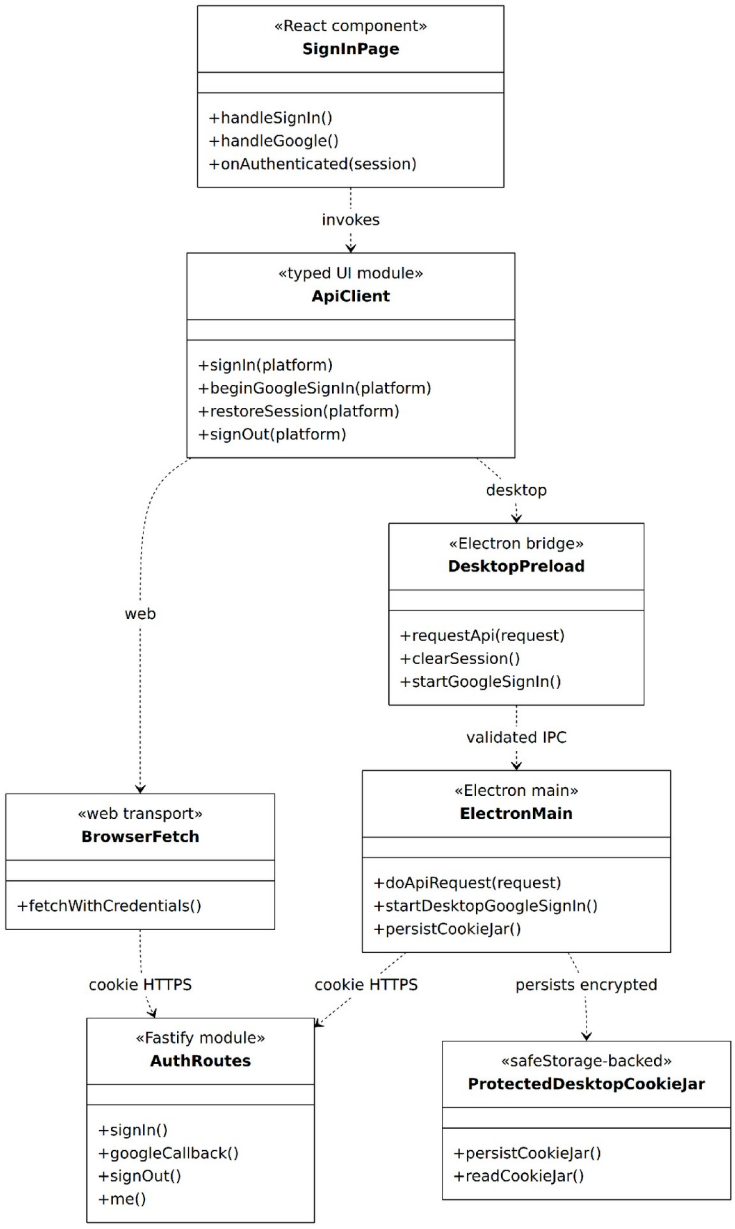


Figure C-01(b). Authentication Organization and Session Contracts — Browser and Electron desktop session transports

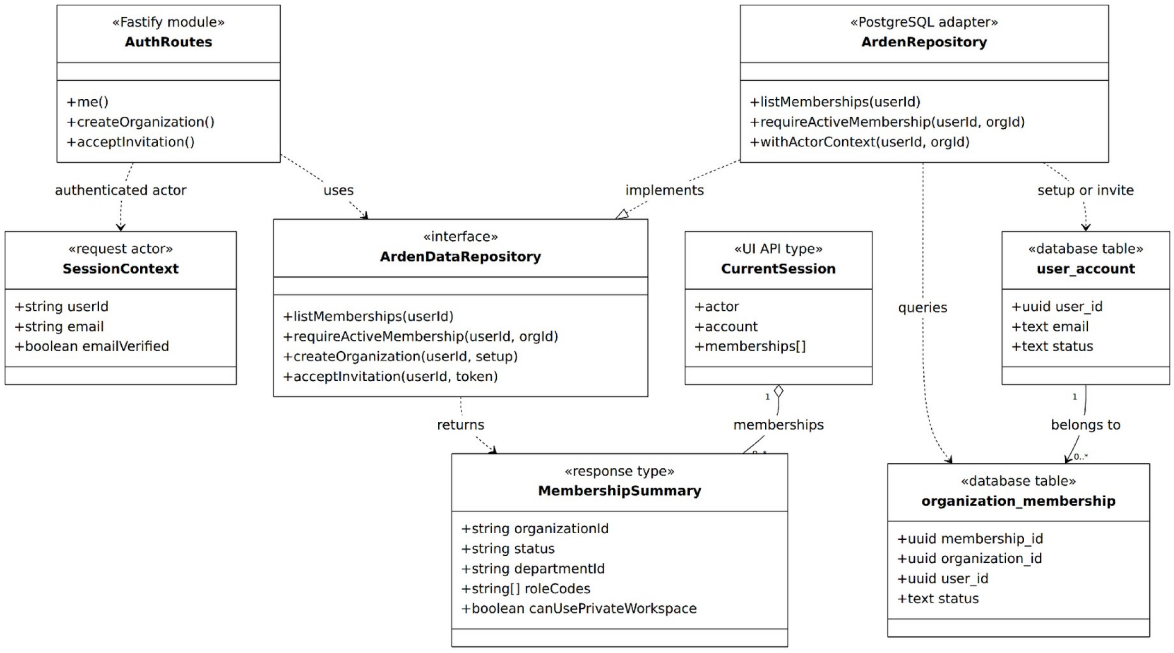


Figure C-01(c). Authentication Organization and Session Contracts — Identity persistence, membership projection, and access contracts

Implemented contracts and notation: These three UML-like class views distinguish real TypeScript interfaces, modules and data types; a module box does not imply a concrete OOP class. AuthProvider is implemented by SupabaseAuthProvider, and ArdenDataRepository by ArdenRepository. The Fastify route group is registered inside buildApp, and its authenticated actor is a SessionContext resolved from HttpOnly access/refresh cookies. The web client uses credentialed fetch; the implemented Electron preload and main-process IPC path persists cookies encrypted through OS safeStorage. /api/v1/me projects organization memberships from ArdenRepository; account records are upserted during organization creation or invitation acceptance rather than on every sign-in. Repository membership checks and PostgreSQL actor/RLS context enforce organization access. Source: apps/api/src/app.ts, auth/provider.ts, db/repository.ts, packages/ui/src/sign-in and shared/api-client.ts, and apps/desktop/src/{preload,main}.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
Auth route handlers	Sign-in/up, Google PKCE redirect/callback, password recovery and sign-out.	apps/api/src/app.ts; auth/*
Identity adapter	Supabase exchange/validation and current user UUID mapping.	Supabase Auth boundary
Session transport	Web HttpOnly cookie vs desktop encrypted OS session behind preload IPC.	apps/web; apps/desktop
Membership policy	Resolve active organization membership; redirect non-members to setup/invite.	organization_membership + RLS

Email Registration and Sign In Sequence

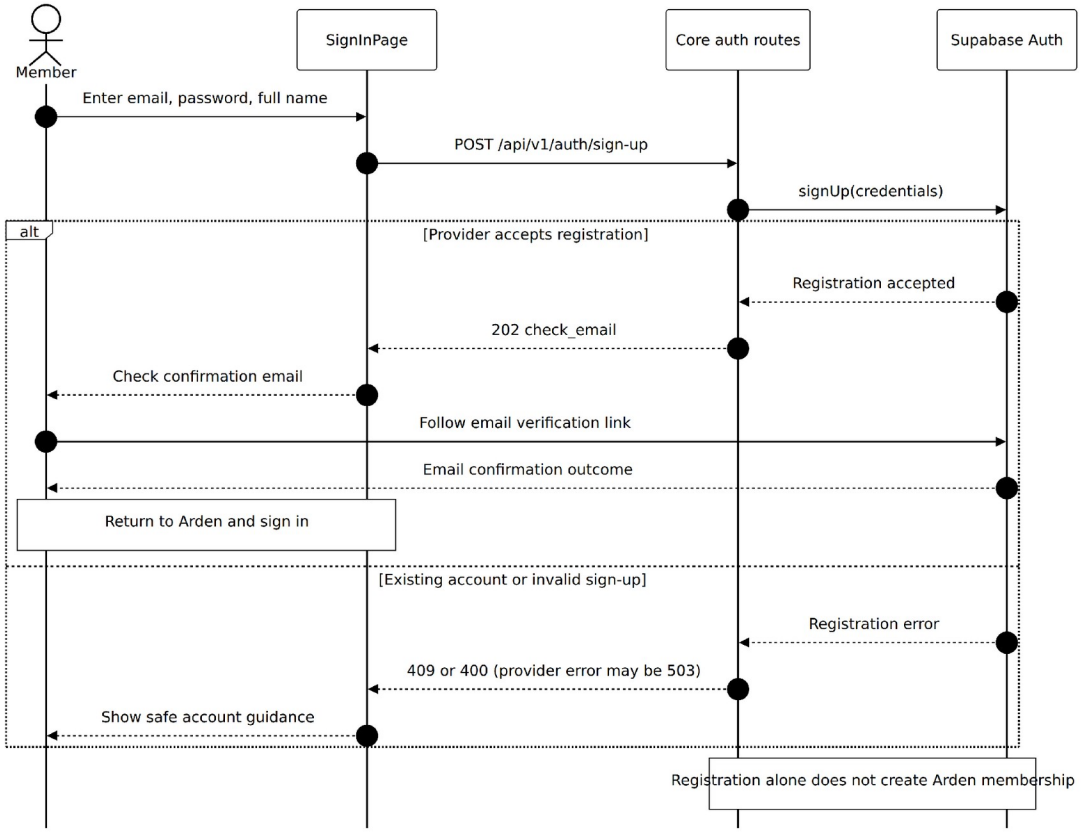


Figure S-01(a). Email Registration and Sign In Sequence — Email account registration and provider confirmation

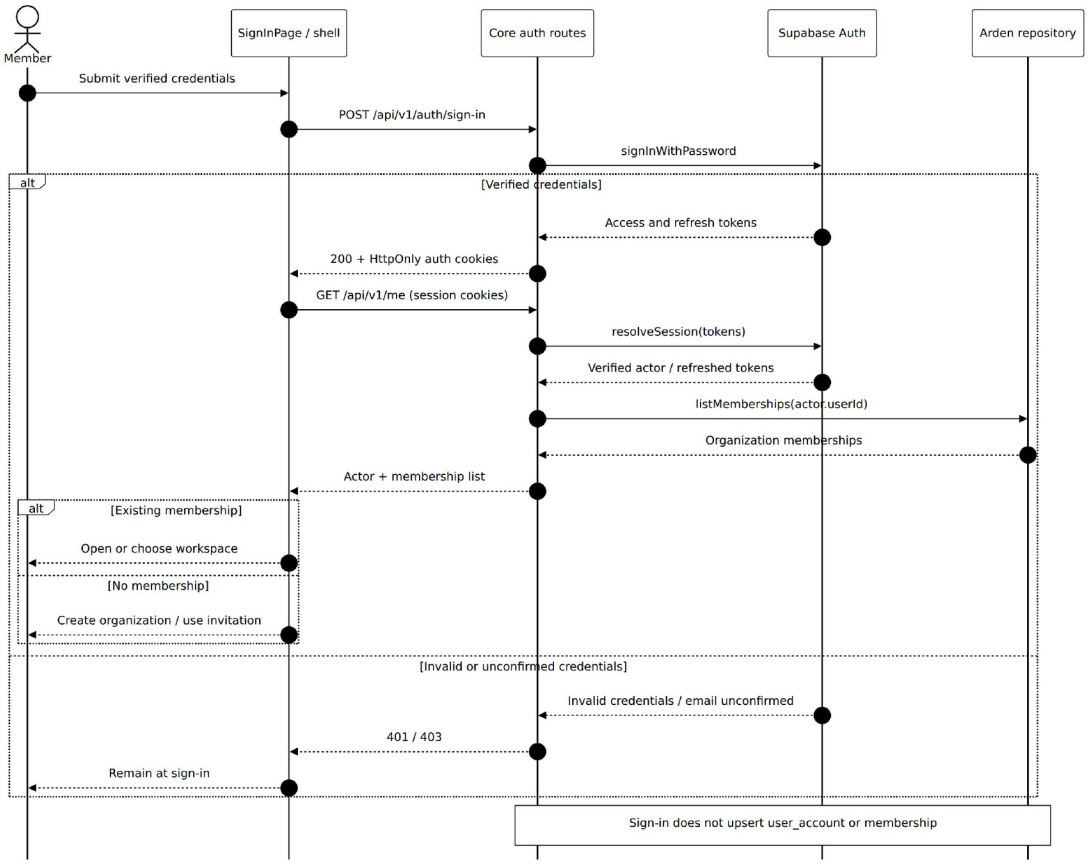


Figure S-01(b). Email Registration and Sign In Sequence — Password sign-in, cookies, and organization membership routing

Implemented behavior and notation: Registration returns 202 check_email and requires provider email confirmation; it does not create an Arden membership. Sign-in delegates to SupabaseAuthProvider and sets HttpOnly cookies. GET /api/v1/me resolves and, if needed, refreshes the session before obtaining organization memberships from ArdenRepository. Neither sign-in nor /me implicitly upserts user_account or membership. The Electron transport uses its implemented secure IPC/cookie bridge. Evidence: apps/api/src/app.ts and auth/provider.ts; db/repository.ts; packages/ui/src/sign-in and shared/api-client.ts; apps/desktop/src/main/index.ts.

1. User submits registration or credentials on the authentication screen.
2. Client calls the Core auth BFF; Core delegates password authentication to Supabase and validates the identity response.
3. Core establishes the HttpOnly browser session (or implemented Electron secure cookie bridge). Arden user_account is upserted during organization creation or invitation acceptance, not by sign-in.
4. Client requests /api/v1/me and resolves organization routing; authentication failures remain on the sign-in screen.

Exceptions and guarantees: No credentials or service role key are exposed to the shared React client. Missing organization membership is not implicitly created.

Google PKCE and Session Callback Sequence

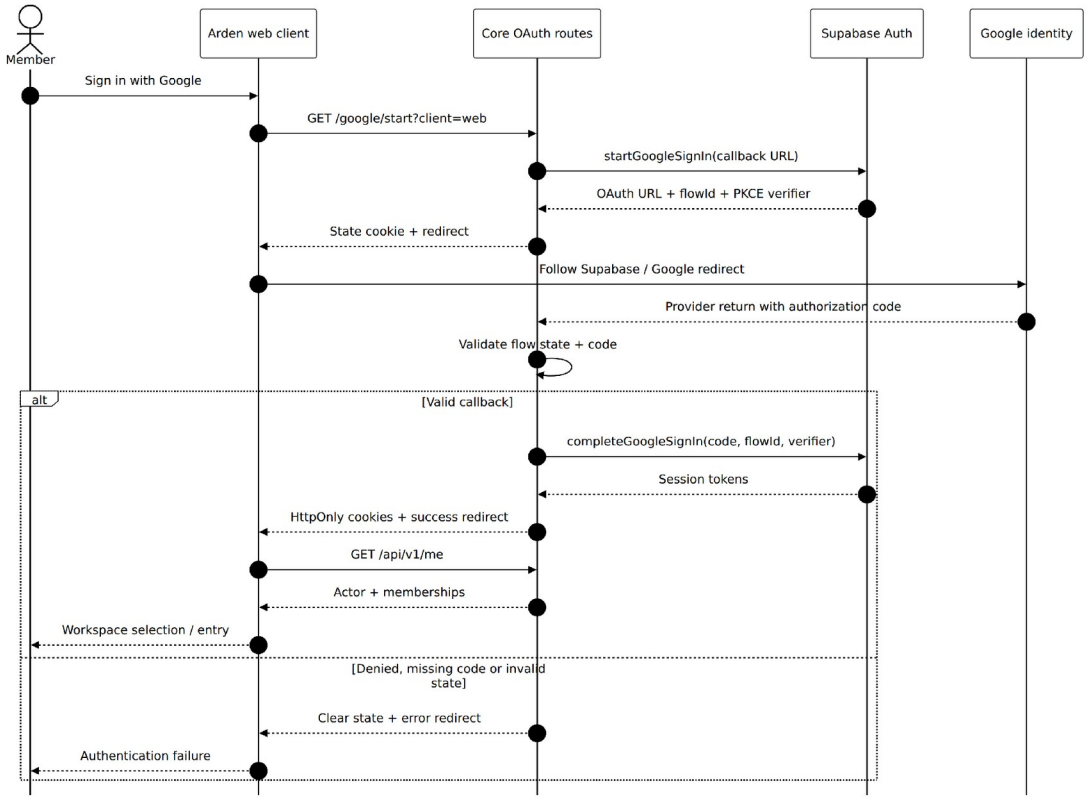


Figure S-02(a). Google PKCE and Session Callback Sequence — Browser Google PKCE, callback, and actor retrieval

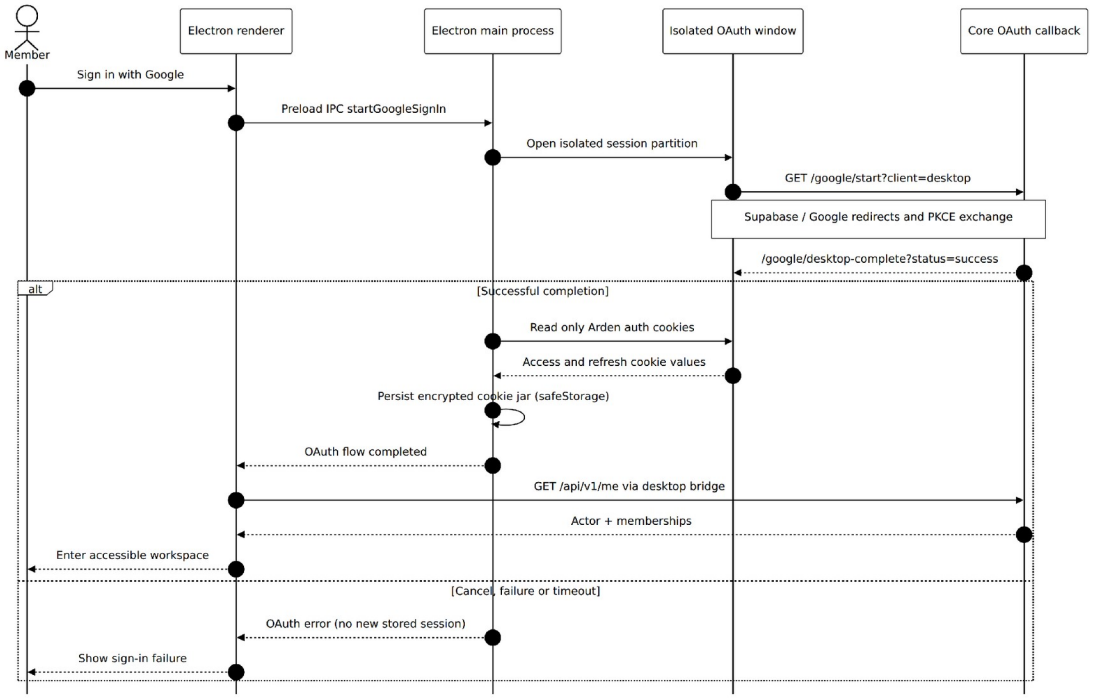


Figure S-02(b). Google PKCE and Session Callback Sequence — Electron isolated OAuth window and OS-encrypted session bridge

Implemented behavior and notation: Core constructs the provider redirect and stores OAuth state containing the flow ID, PKCE verifier, and web/desktop client type in a protected cookie. The callback validates the state and code, exchanges them via SupabaseAuthProvider, clears transient state, and sets HttpOnly Arden auth cookies. The implemented Electron OAuth window uses an isolated session partition; the main process imports only allowlisted Arden cookies into its safeStorage-protected cookie jar. Failed, denied, canceled, or timed-out flows do not establish a new authenticated session. Evidence: apps/api/src/app.ts; auth/provider.ts; apps/desktop/src/main/index.ts; packages/ui/src/shared/api-client.ts.

1. User selects Sign in with Google; Core generates a PKCE challenge and protected OAuth flow state.
2. Browser follows the authorized provider redirect and returns an authorization code to Core's callback.
3. Core validates the flow state and exchanges the code with the identity provider.
4. Core establishes the appropriate browser/desktop session and returns the client to Arden, which loads /me.

Exceptions and guarantees: Invalid state, provider denial or interrupted callback ends without an authenticated Arden session.

3.2 Organization Onboarding and Administration

An authenticated account becomes useful after joining or creating an organization. Organization Admins maintain departments, membership, audiences and scoped reviewer grants; installation System Admin remains a distinct non-content capability.

User-facing entry and screens: Workspace Select for live organization creation; InvitationLinkPage for live acceptance; Organization Administration with Members, Departments, Readers, Invitations and Review permissions. OrganizationSetupPage/InvitationAcceptPage are previews.

Code evidence: packages/ui/src/organization; apps/api/src/app.ts; apps/api/src/audiences; migrations 0002, 0017, 0022–0023.

3.2.1 Class Diagram

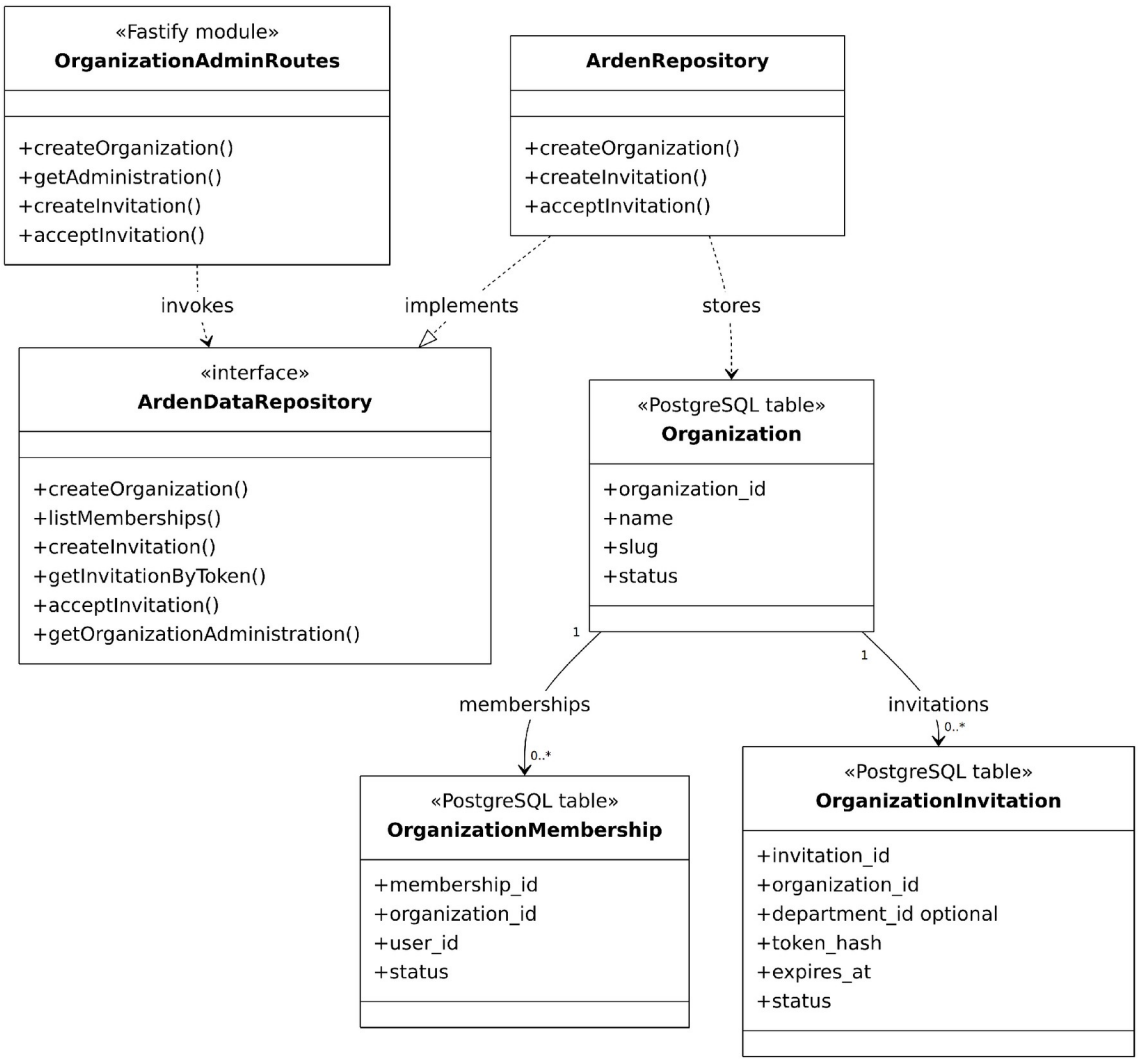


Figure C-02(a). Organization Membership and Access Contracts — Organization, membership, and invitation persistence contracts

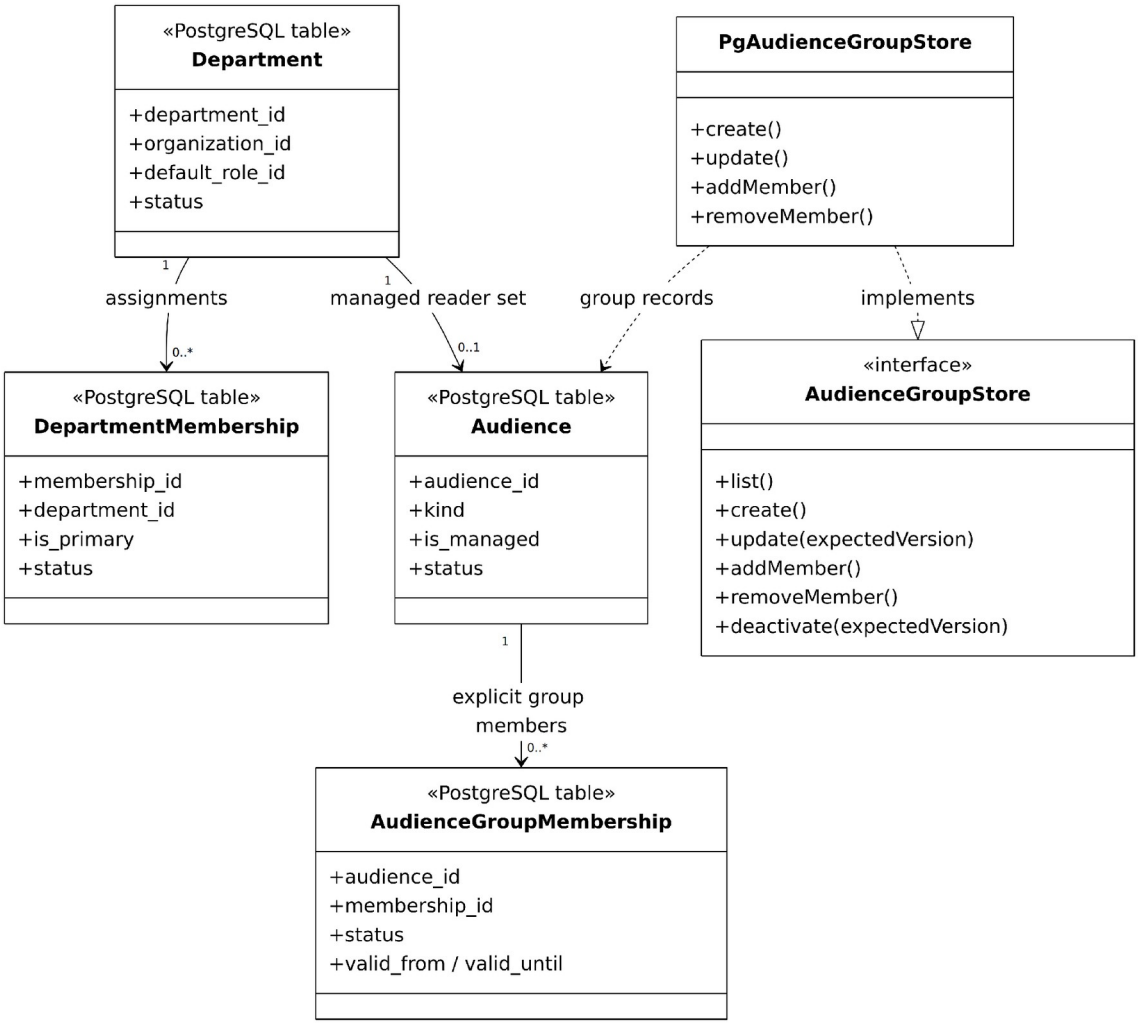


Figure C-02(b). Organization Membership and Access Contracts — Departments, audience groups, and reader access contracts

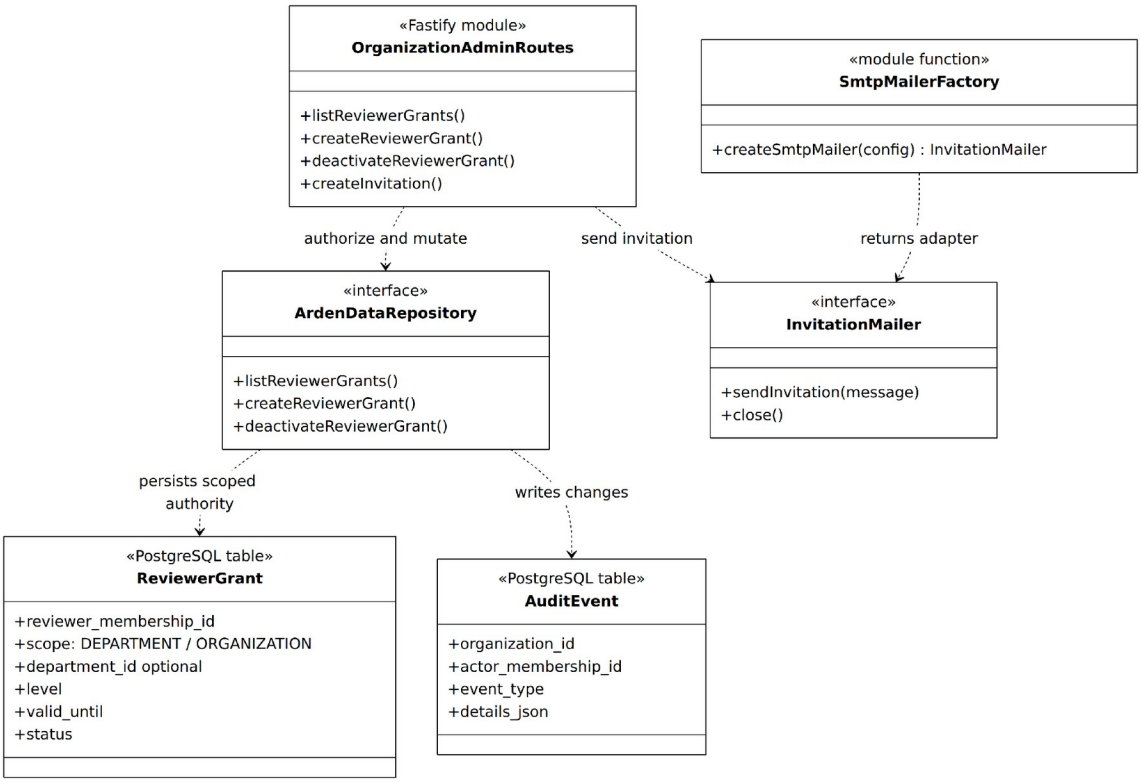


Figure C-02(c). Organization Membership and Access Contracts — Reviewer-grant authority, audit events, and SMTP invitation port

Implemented behavior and notation: UML-style logical contracts are grounded in actual TypeScript interfaces (ArdenDataRepository, AudienceGroupStore, InvitationMailer), their implemented repository/store adapters, Fastify route modules, and PostgreSQL records. Boxes labeled as modules, functions, or tables are not misrepresented as TypeScript classes. Organizations contain memberships and invitations; departments have primary assignments and managed reader audiences. AudienceGroupStore administers explicit reader groups. Reviewer grants are separate department/organization review authority managed by Org Admins; they do not equal reader audience membership. InvitationMailer is implemented by a createSmtpMailer factory, not by a named SmtpInvitationMailer class. Evidence: apps/api/src/app.ts; db/repository.ts; audiences/{routes,store}.ts; email/smtp-mailer.ts.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
Organization API	Bootstrap organization, list/modify departments and member context.	organization, department, membership
Invitation lifecycle	Create, resend, revoke and accept hashed-token invitations.	organization_invitation; SMTP
Audience administration	Maintain organization/department/group readers and group members.	audience, audience_group_membership
Reviewer grant policy	Grant/revoke DEPARTMENT or ORGANIZATION review authority.	reviewer_grant; no implied readership

Organization Creation and Invitation Acceptance Sequence

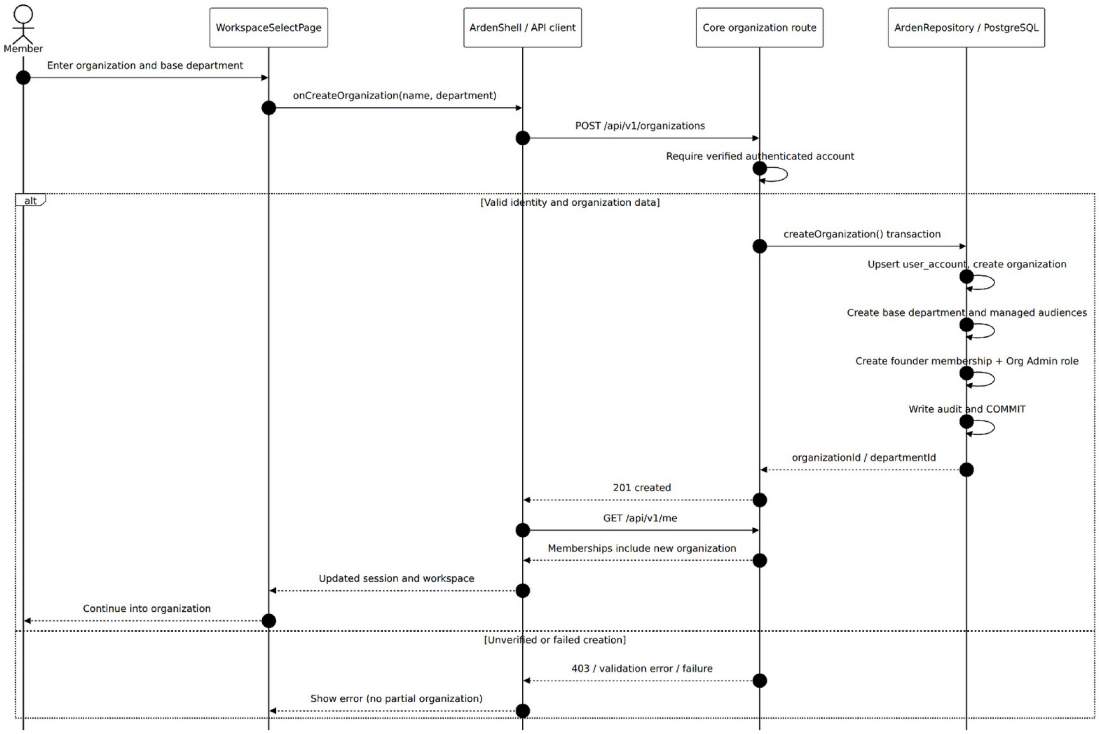


Figure S-03(a). Organization Creation and Invitation Acceptance Sequence — Live organization bootstrap from workspace selection

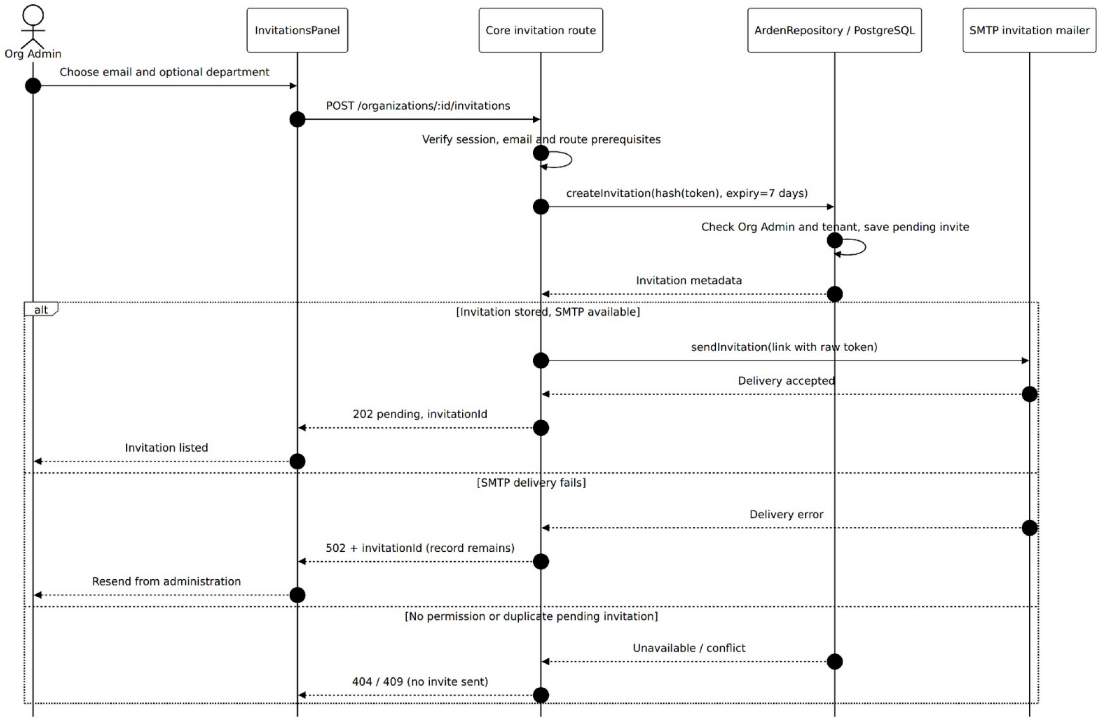


Figure S-03(b). Organization Creation and Invitation Acceptance Sequence — Admin invitation creation and SMTP delivery

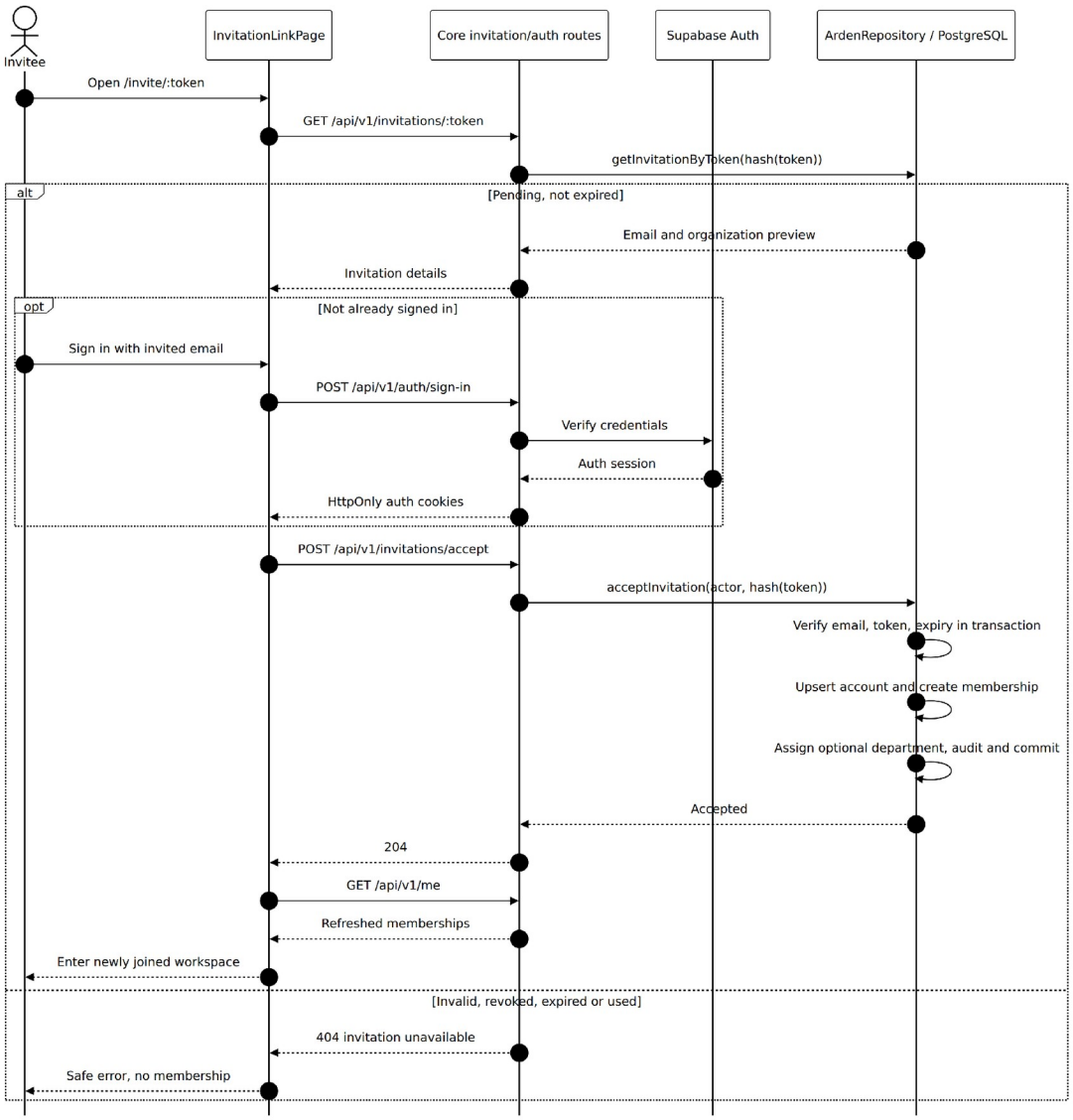


Figure S-03(c). Organization Creation and Invitation Acceptance Sequence — Invited member authentication and atomic membership acceptance

Implemented behavior and notation: The implemented live onboarding UI is WorkspaceSelectPage through ArdenShell, not the synthetic OrganizationSetupPage preview. Organization creation runs one repository transaction to create the account projection, organization, base department, managed audiences, founder membership, Org Admin assignment, and audit. Admin invitation creation stores a hash of a random token with a seven-day expiry; department assignment is optional. SMTP is required for this invitation route. If email delivery fails, the saved invitation remains available for resend (502 includes invitationId). The live InvitationLinkPage shows safe metadata, signs in if needed, then invokes an atomic email-verified invitation acceptance and refreshes /me. Evidence: ArdenShell, WorkspaceSelectPage, InvitationLinkPage, apps/api/src/app.ts, db/repository.ts.

1. A signed-in, email-verified person can create an organization from the live WorkspaceSelectPage (ArdenShell); OrganizationSetupPage is a synthetic preview.
2. Core validates tenant creation and writes the founding organization/membership atomically.

3. Org Admin creates an invitation with an optional department. Core stores a hashed seven-day token and sends its link through the configured required SMTP adapter; delivery failure leaves a saved invitation to resend.
4. Invitee uses the live InvitationLinkPage, signs in or creates and verifies an account, then Core checks token, expiry, invited email and existing state before creating membership.
5. The interface selects the accessible organization; replay/expired/revoked links show safe rejection states.

Exceptions and guarantees: Invitation acceptance and organization membership are server-authorized; the invite's department field is optional in the latest migration.

Member Department and Reviewer Grant Administration Sequence

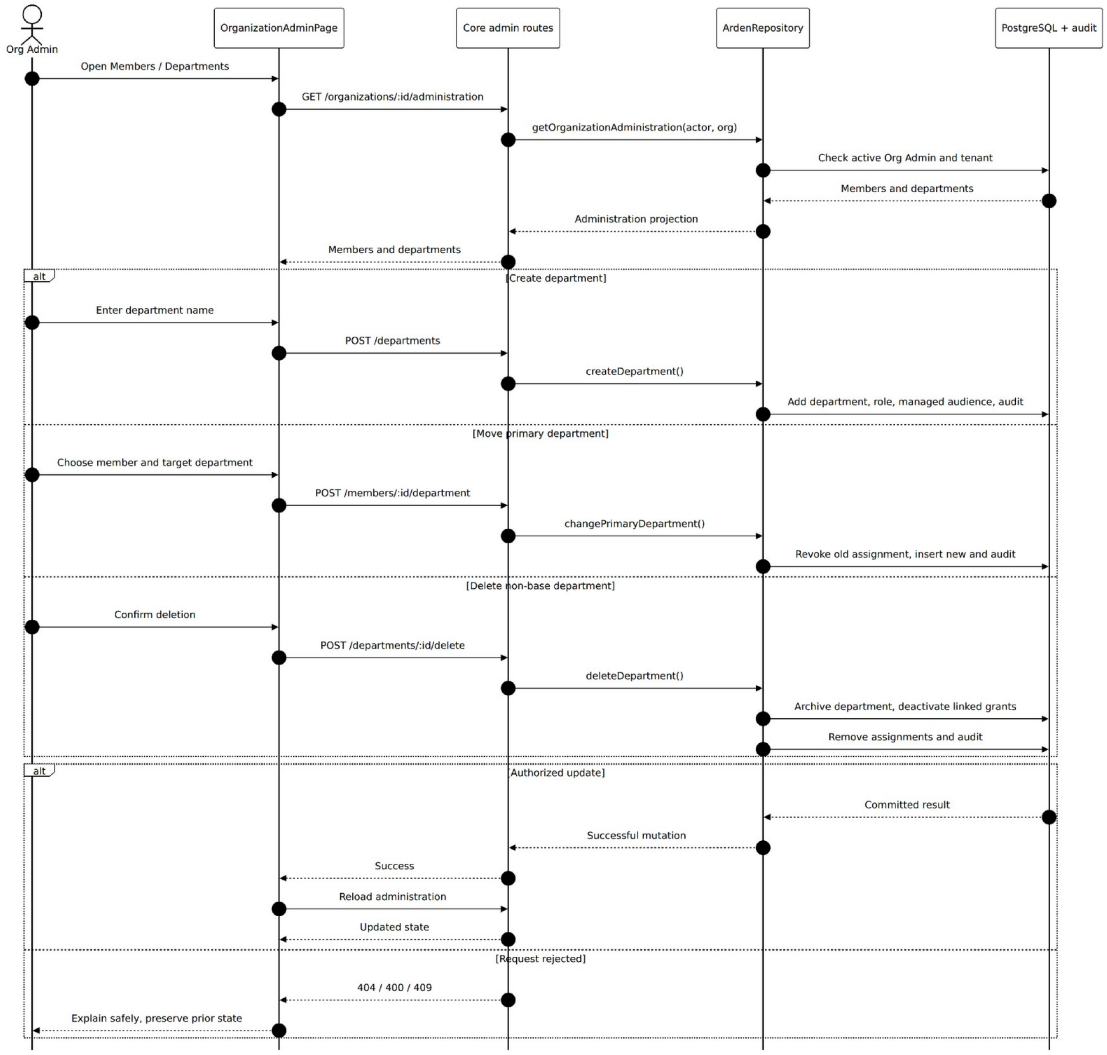


Figure S-04(a). Member Department and Reviewer Grant Administration Sequence — Organization administration of departments and member assignments

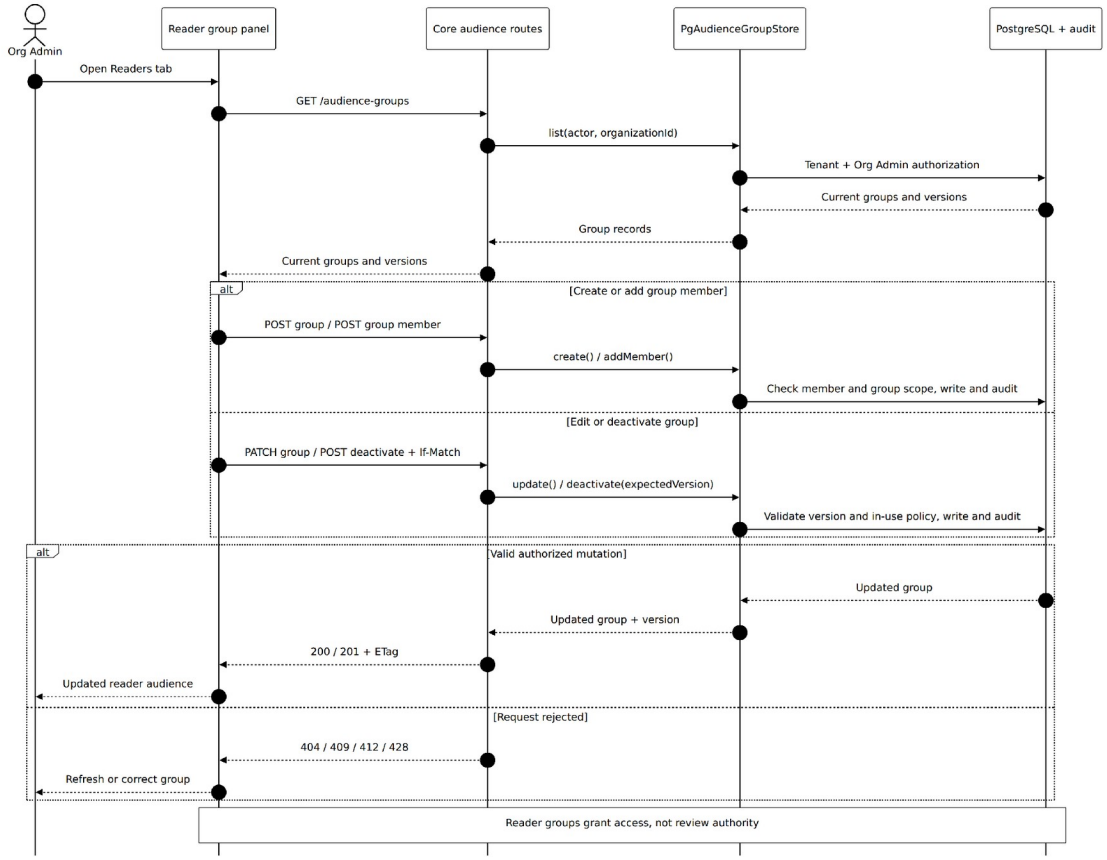


Figure S-04(b). Member Department and Reviewer Grant Administration Sequence — Reader audience-group and membership administration

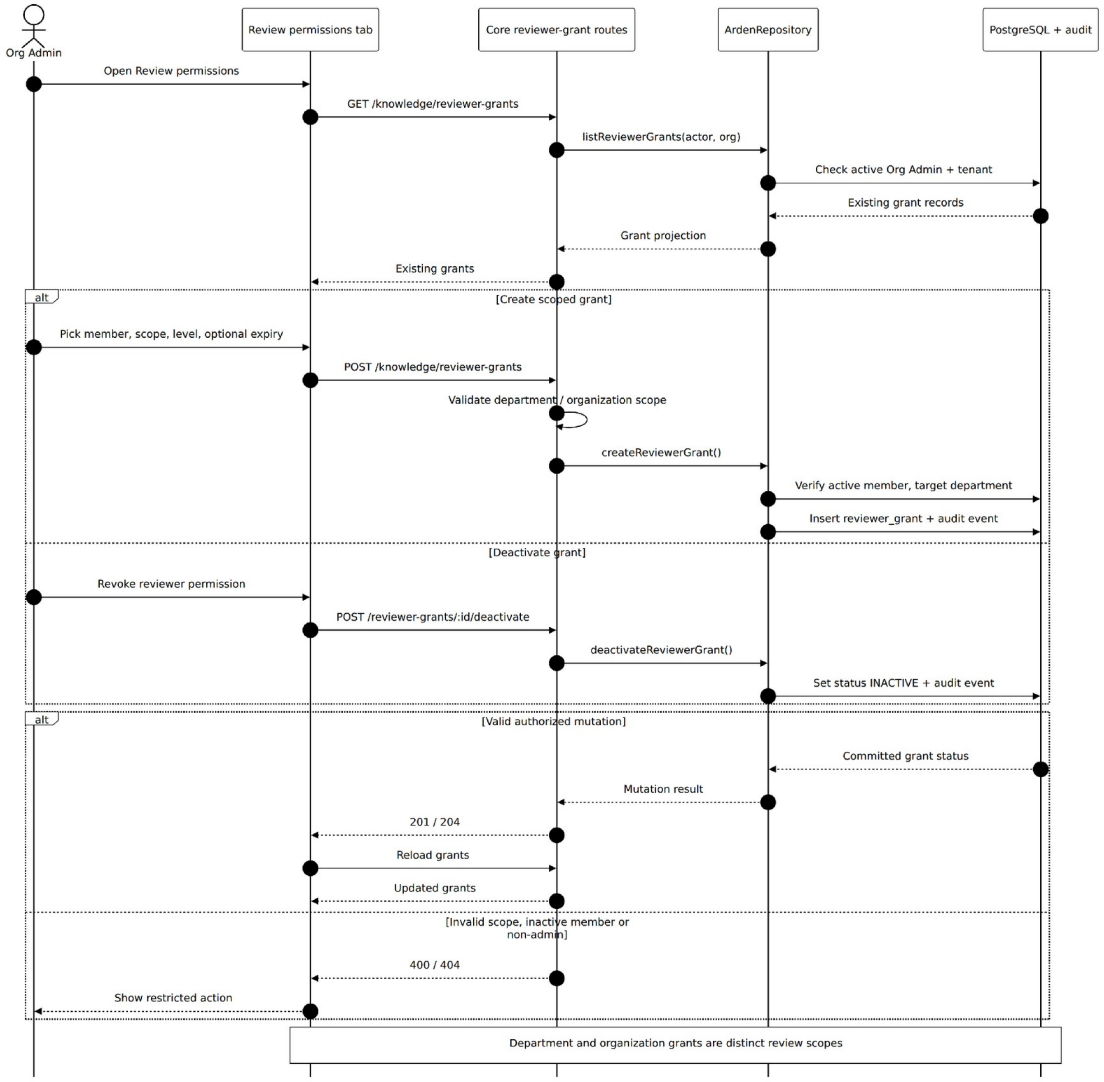


Figure S-04(c). Member Department and Reviewer Grant Administration Sequence — Scoped reviewer permission granting and deactivation

Implemented behavior and notation: Administration operations require an active Org Admin in the target tenant. The department APIs create role/audience/scope records, switch primary department assignments, or archive non-base departments and deactivate related grants. The separate audience group API validates group version preconditions (ETag/If-Match) and group-in-use rules. Reviewer grants have DEPARTMENT or ORGANIZATION scope and optional expiry, and are checked against active members and scoped departments; creates/deactivations are audited. Reader audience memberships never grant reviewer authority by themselves. Evidence: apps/api/src/app.ts, audiences/{routes,store}.ts, db/repository.ts; packages/ui/src/organization/OrganizationAdminPage.tsx.

1. Organization Admin opens the administration page and retrieves active departments, membership and grants.
2. An administrative change (department, primary department, group audience or reviewer grant) is submitted to Core.
3. Core confirms Org Admin capability, tenant scope, target membership and policy constraints.
4. Core commits the record changes and corresponding audit data and returns updated administration state.

Exceptions and guarantees: Reader audiences control who can view content, while scoped reviewer grants independently govern review authority. A department grant does not grant organization-wide review authority.

3.2.4 Onboarding Activity Diagram

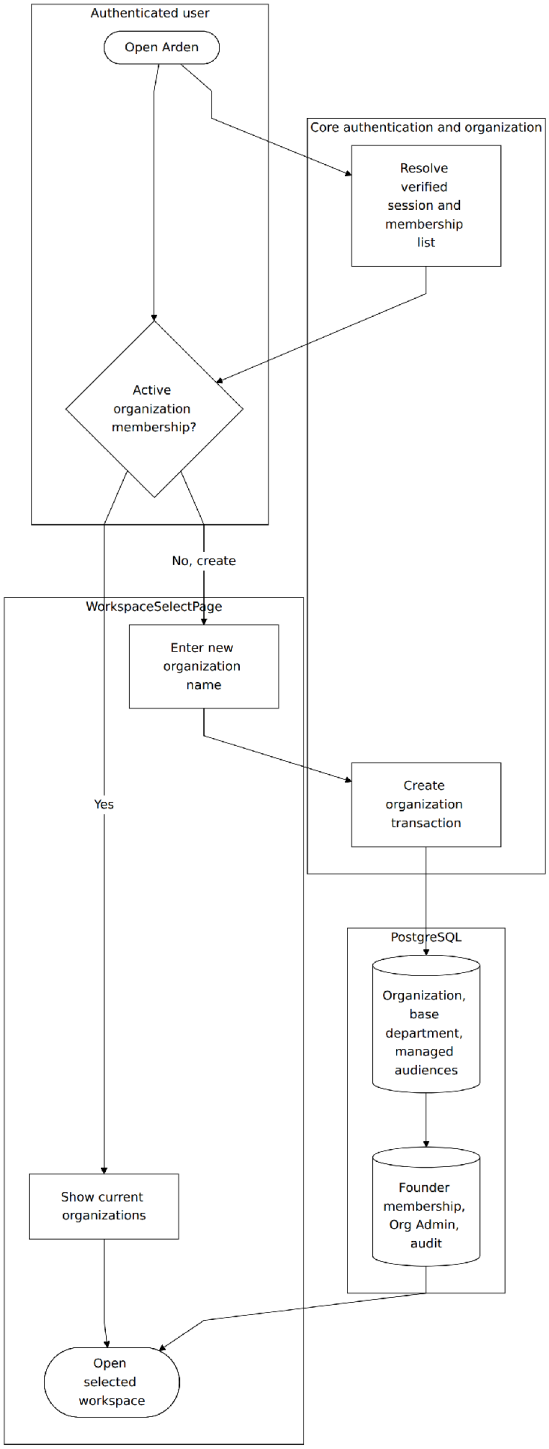


Figure U-01(a). Organization Onboarding Swimlane Activity — Existing membership and new organization bootstrap

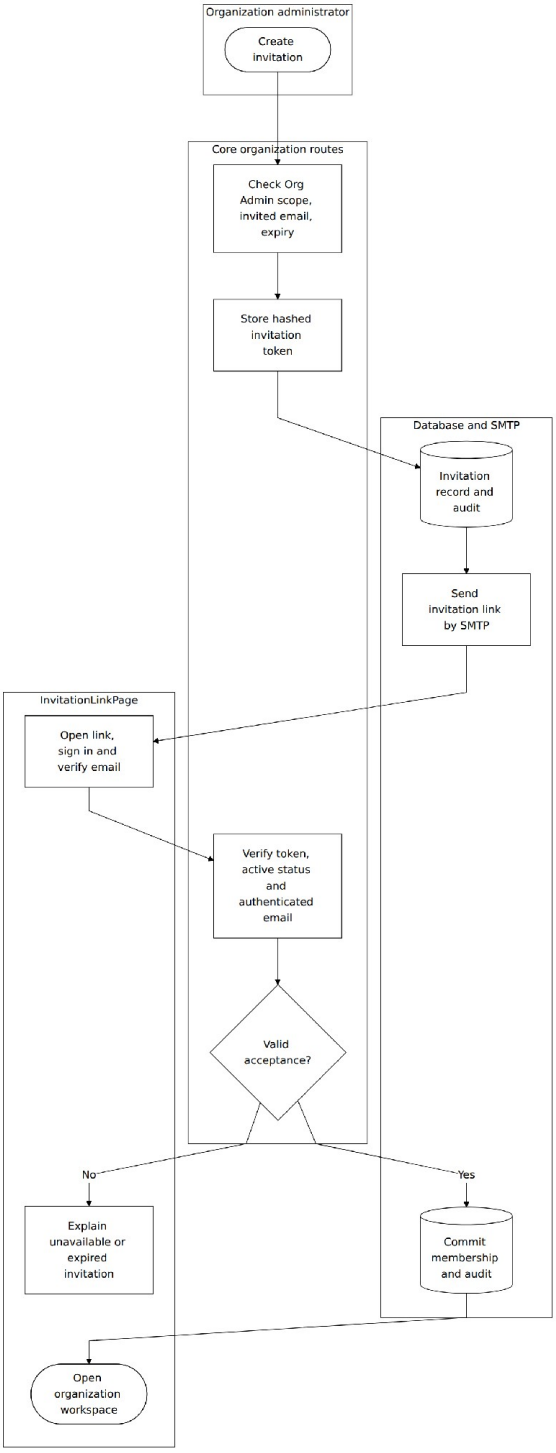


Figure U-01(b). Organization Onboarding Swimlane Activity — Admin invitation delivery and invited-member acceptance

Implementation evidence and scope: The implemented onboarding runs through authenticated WorkspaceSelectPage and InvitationLinkPage, not a synthetic setup screen. Core establishes founder or invited membership in PostgreSQL; invitations use token validation and configured SMTP delivery. The existing-member path goes directly to workspace selection. Evidence: apps/api/src/app.ts, db/repository.ts, packages/ui/src/workspace and invitation flows.

3.2.5 Invitation and Membership State Machines

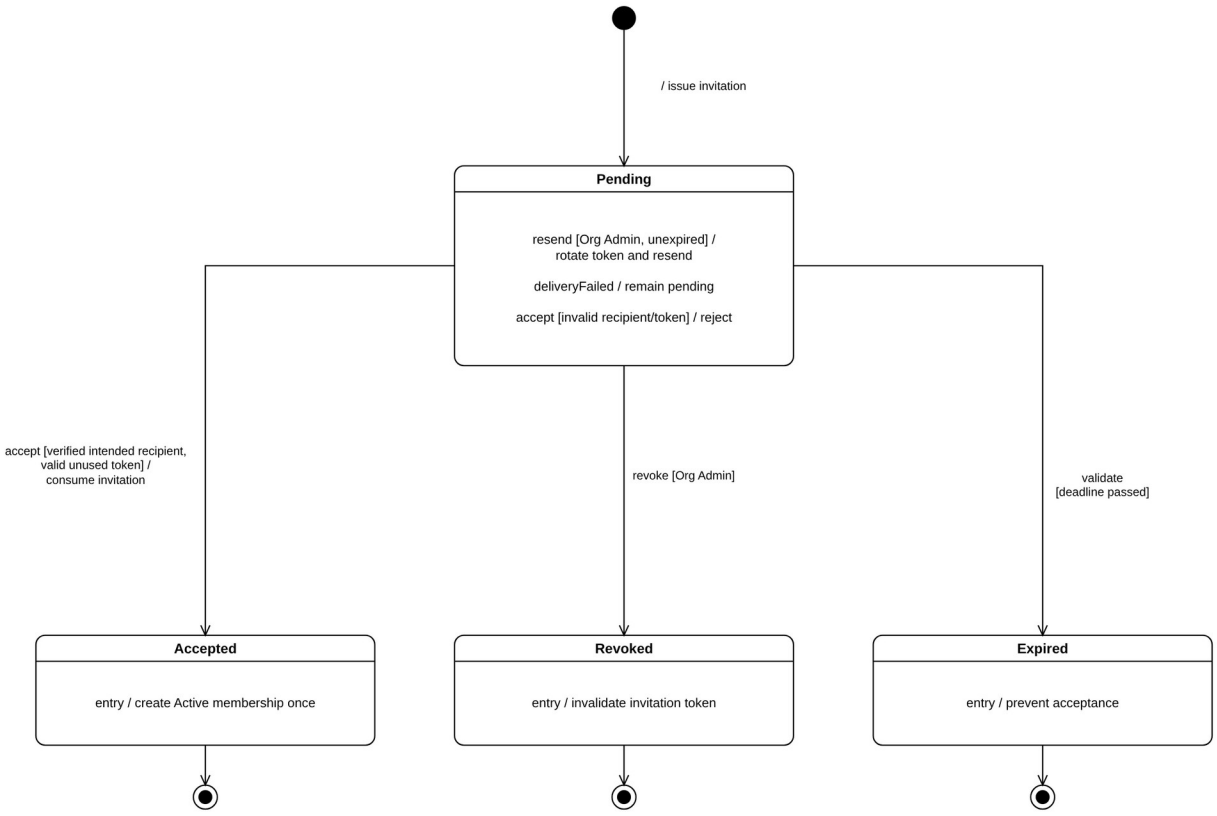


Figure M-01. Organization Invitation State Machine

Implementation evidence and scope: organization_invitation.status is PENDING, ACCEPTED, REVOKED or EXPIRED (migration 0002). Create and resend store a hashed, single-use token with a seven-day deadline; resend rotates the token. When SMTP delivery fails the invitation is kept and the API returns INVITATION_DELIVERY_FAILED, so it stays Pending and can be resent. Acceptance requires the signed-in, verified intended recipient and an unused, unexpired token, and creates the Active membership once, with the invited department or with none when departmentId was omitted. Revoke is an Org Admin action; an invitation read or accepted after expires_at is marked EXPIRED. Evidence: apps/api/src/app.ts invitation routes; apps/api/src/db/repository.ts.

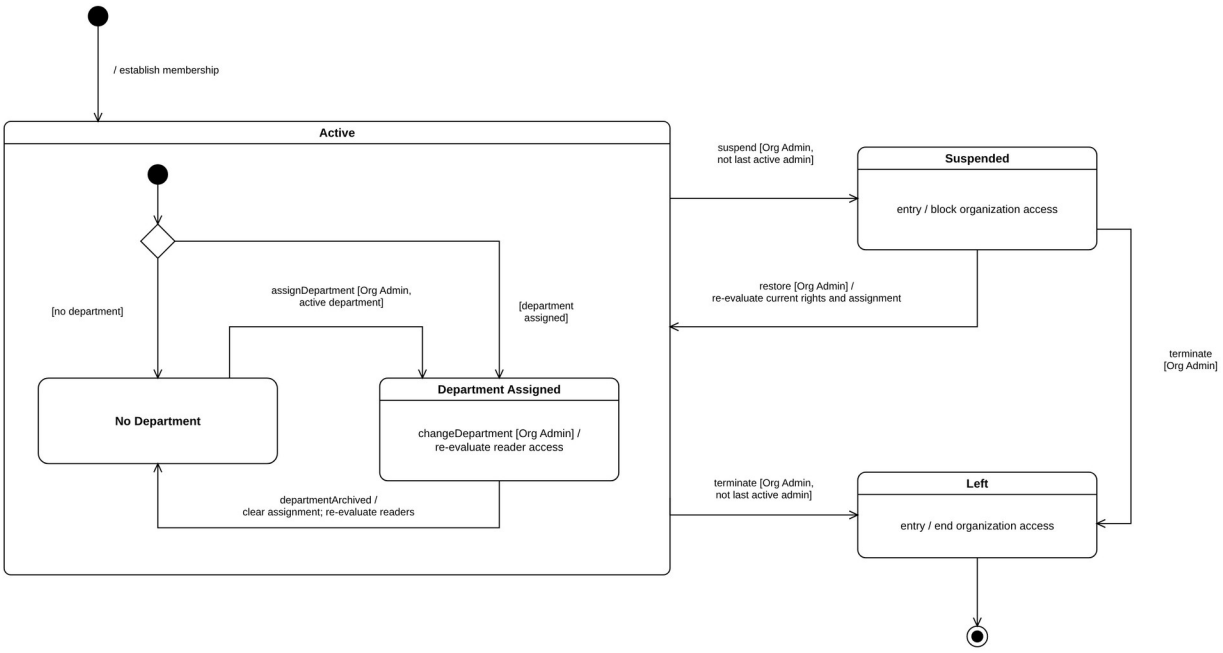


Figure M-02. Organization Membership State Machine

Implementation evidence and scope: organization_membership.status is INVITED, ACTIVE, SUSPENDED or LEFT (migration 0001). The sub-states of Active describe the primary department: a member invited without a department, or whose department is deleted (archived), has No Department because the department_membership becomes REMOVED (migration 0023, deleteDepartment). An Org Admin assigns or changes the primary department through POST /members/{membershipId}/department, and reader access is re-evaluated on the next request. Scope note: the staging branch has no API route or screen for suspend, restore or terminate; SUSPENDED and LEFT exist only in the schema, so those transitions are designed but not yet implemented.

3.3 Private Knowledge and Personal Workspace

A Member captures notes for themselves and later may choose to contribute a separate copy. The owner's original document, its versions and protected content bytes stay private throughout contribution. This is the actual staging implementation corresponding to the user's WF-04.

User-facing entry and screens: My Work → Private Notes; private-note list/search and editor; Create review draft status-island panel.

Code evidence: packages/ui/src/personal-workspace/PersonalWorkspacePage.tsx, ContributePrivateNoteDialog.tsx; apps/api/src/private-notes; apps/api/src/app.ts.

3.3.1 Class Diagram

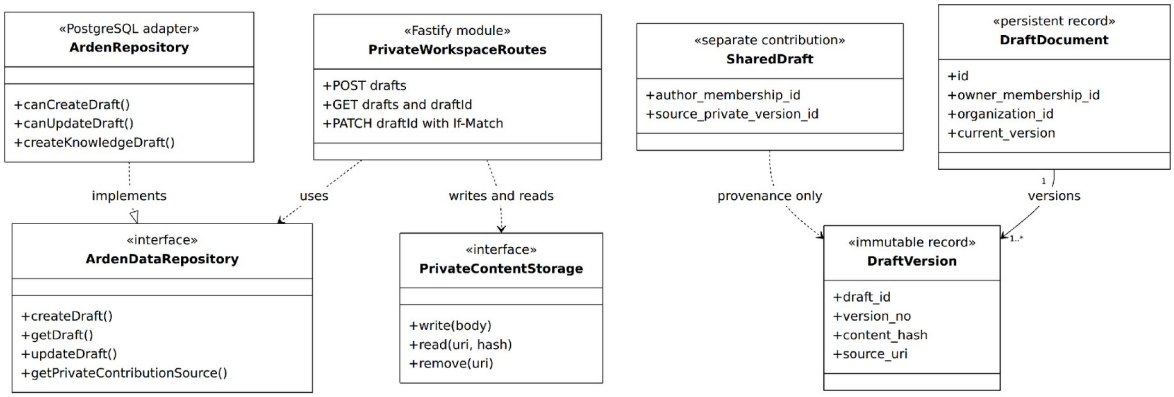


Figure C-03. Private Note Version and Contribution Contracts

Implementation evidence and scope: Private note CRUD uses tenant- and owner-checked Core endpoints and protected content objects. Each save creates an immutable private version. Contribution constructs a separate shared draft with provenance to a saved private version; it never mutates the private original. Evidence: apps/api/src/app.ts, db/repository.ts, private-notes/content-storage.ts, personal-workspace components.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
PrivateNotesController	Client selection, draft edits, dirty-state guard, save and list/search.	personal-workspace/usePrivateNotes.ts
Private-note handlers	Owner-scoped create/get/update and version resolution.	private_workspace/draft_document/ draft_version
LocalPrivateContentStorage	Persist protected bytes with opaque UUID URI and hash validation.	private-notes/content-storage.ts
Contribution action	Copy a saved private version into a new author-owned shared draft.	POST knowledge/drafts; Idempotency-Key

Create Edit and Save Private Note Sequence

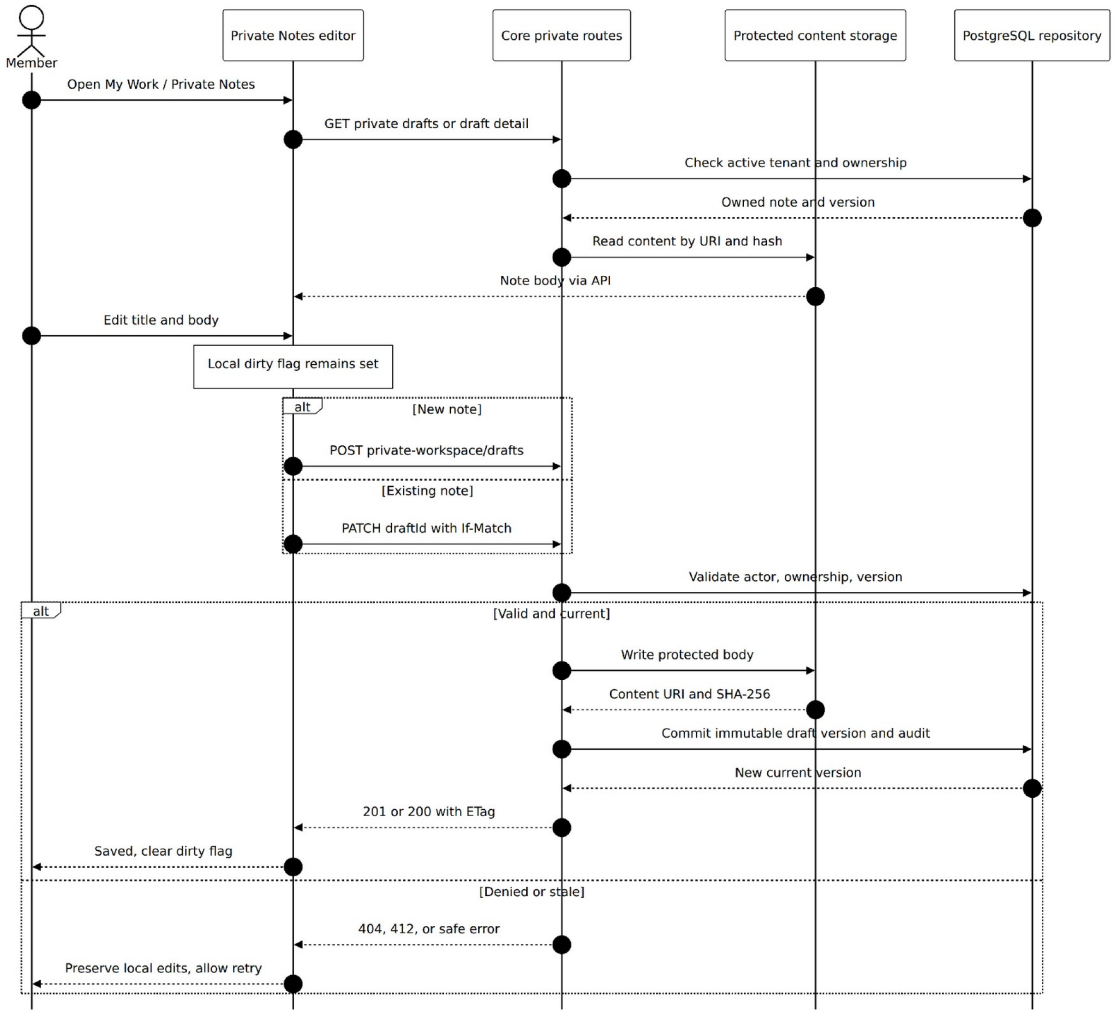


Figure S-05. Create Edit and Save Private Note Sequence

Implementation evidence and scope: The implemented POST/PATCH private-workspace routes verify active owner access and persist protected text plus a draft-version record. PATCH requires If-Match; stale edits fail without replacing the user's local editor buffer. Evidence: apps/api/src/app.ts private-workspace routes and packages/ui/src/personal-workspace/usePrivateNotes.ts.

1. Member opens Private Notes and selects New note or an existing note.
2. Editor maintains local edits and flags unsaved changes; Save sends the title/body through the authenticated Core API.
3. Core validates ownership, protects the content object and records a new immutable draft_version linked to draft_document.
4. The UI refreshes its version ID and clears the dirty flag; failure preserves the local draft and displays retry/error feedback.

Exceptions and guarantees: A member cannot read another member's note. Private notes do not automatically enter shared search or publication.

WF-04 Create Shared Copy from Private Note Sequence

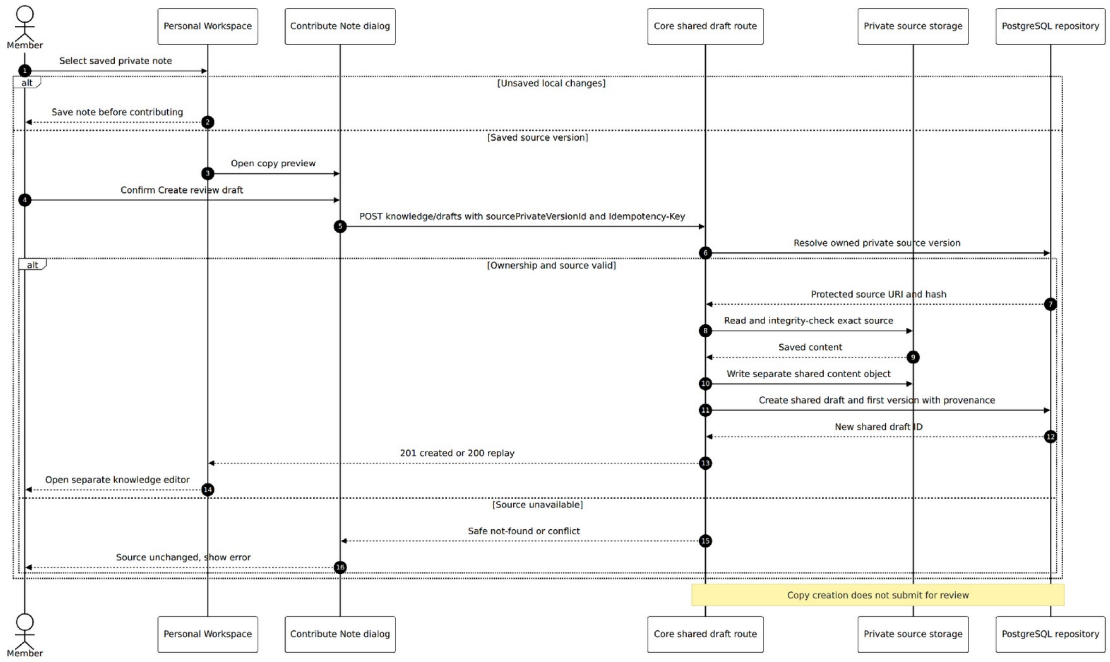


Figure S-06. WF-04 Create Shared Copy from Private Note Sequence

Implementation evidence and scope: The Create review draft action requires a saved private source version. Core reads that owner-authorized immutable content, writes a distinct shared content object and creates a shared draft/version with source_private_version provenance. The explicit submit operation is later. Evidence: ContributePrivateNoteDialog.tsx, apps/api/src/app.ts POST knowledge/drafts and repository createKnowledgeDraft.

1. Member selects a saved private note and activates Create review draft; UI requires no unsaved edits and shows a content preview.
2. The user confirms Create draft; the client generates an idempotency key bound to sourceVersionId.
3. Core verifies current actor owns the source private version, reads its protected bytes and creates a separate shared_draft with provenance.
4. Core creates the initial shared_draft_version and returns the new draft; UI opens the shared knowledge editor.
5. The member later chooses readers and explicitly submits the draft. Merely creating the copy does not create a PENDING_REVIEW submission.

Exceptions and guarantees: The original private draft/version remains unchanged. A retry reuses the same request key; another source user's note is never copyable.

3.3.4 Personal Contribution Activity Diagram

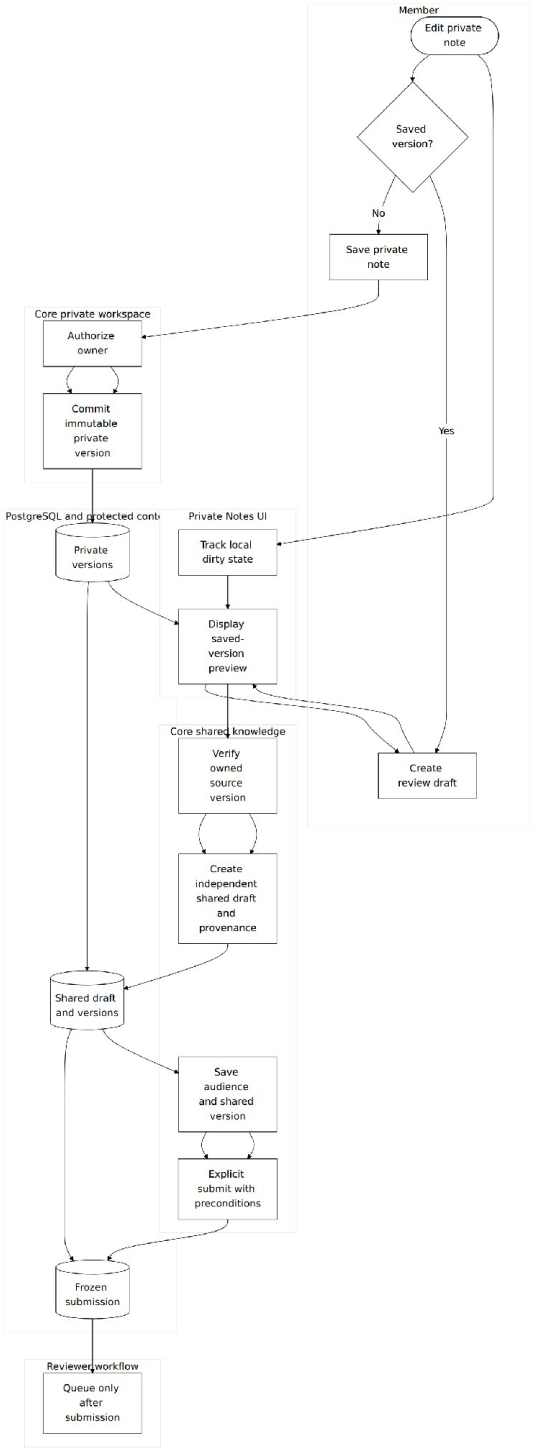


Figure U-02. WF-04 Private Note Contribution Swimlane Activity

Implementation evidence and scope: Private-note saving, source authorization, shared-copy creation, reader selection and review submission are separate steps. Only the final explicit submit creates a pending immutable snapshot. Evidence: apps/api/src/app.ts and packages/ui/src/personal-workspace and knowledge modules.

3.4 Shared Draft Authoring and Reader Selection

Members can create an author-owned shared draft directly in the editor, upload a PDF, or enter from a contributed private-note copy. The author edits a working version, chooses allowed audiences, and sends one immutable candidate for review.

User-facing entry and screens: Knowledge → My Contributions / Create Draft; rich-text editor or PDF upload; reader selector; Submit for review status.

Code evidence: packages/ui/src/knowledge/ServerKnowledgeFeaturePage.tsx and ReaderPicker.tsx; docs/engineering/mainflow-2-workflow.md; apps/api/src/app.ts.

3.4.1 Class Diagram

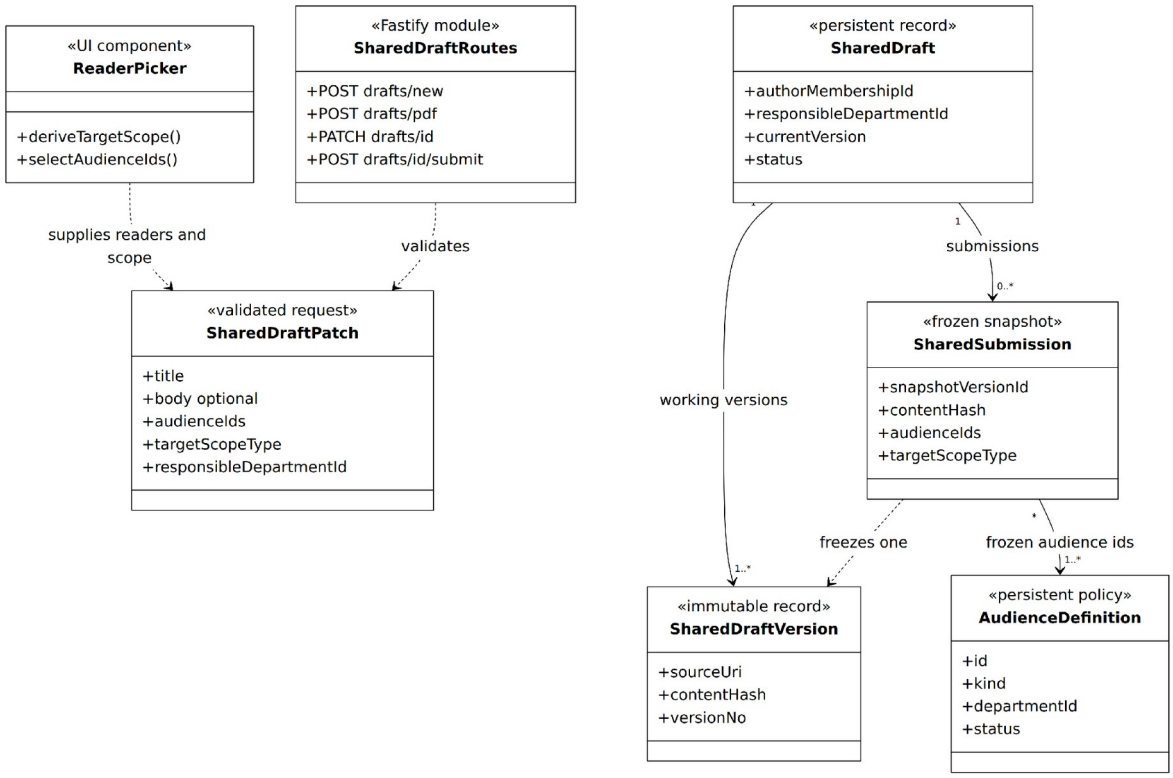


Figure C-04. Shared Draft and Audience Validation Contracts

Implementation evidence and scope: Shared drafts own working immutable versions and selected reader audiences. The author and primary department are resolved server-side; ReaderPicker derives organization/department scope. Submission freezes one exact version, content hash and audience. Evidence: apps/api/src/knowledge/policy.ts, audiences/policy.ts, db/repository.ts and ReaderPicker.tsx.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
Shared draft routes	Create from new document, PDF or private version; list and save versions.	POST /knowledge/drafts/new, /pdf and /drafts
Semantic Document	Versioned valid editor content and parsing/serialization.	packages/domain/semantic-document.ts
ReaderPicker	Select permitted org/department/group audiences and derive target scope.	knowledge/readerSelection.ts

Module / contract	Responsibility	Relevant boundary
Submit transaction	Freeze title, URI, hash, MIME, responsible department, audiences and chosen scope.	shared_submission; version/precondition checks

Create Text or PDF Shared Draft Sequence

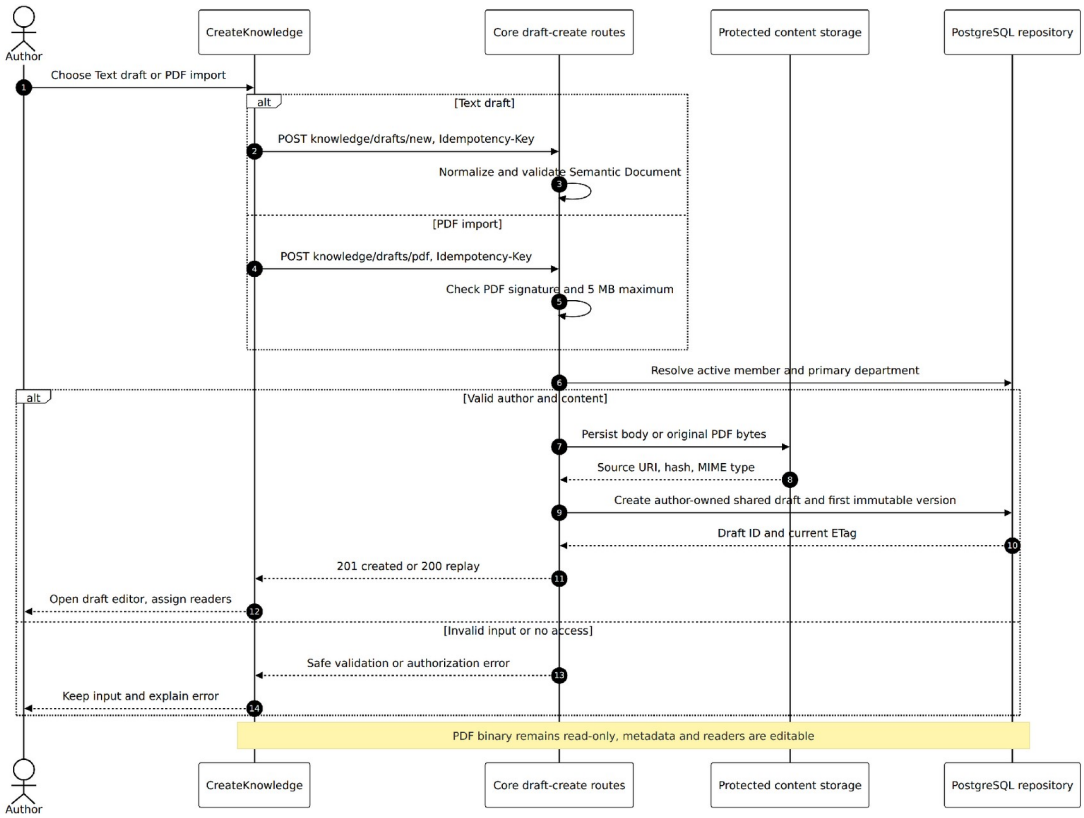


Figure S-07. Create Text or PDF Shared Draft Sequence

Implementation evidence and scope: Core provides separate text and PDF creation routes. Semantic Document text is normalized; PDF signature and 5 MB size are checked, and the original PDF remains binary. Both paths create an author-owned shared draft with an initial immutable version. Evidence: apps/api/src/app.ts POST knowledge/drafts/new and /pdf.

1. Author opens Create Draft and chooses rich-text authoring or PDF import.
2. Text route validates serialized Semantic Document; PDF route checks PDF signature and maximum 5 MB, keeping the file rather than making its body editable.
3. Core determines the author's active primary department and creates shared_draft plus initial immutable shared_draft_version.
4. UI opens the new draft detail, showing author ownership and review by the responsible department.

Exceptions and guarantees: A PDF draft's binary body cannot be edited in the rich-text editor; metadata/readers remain manageable.

Save Draft Version and Audience Sequence

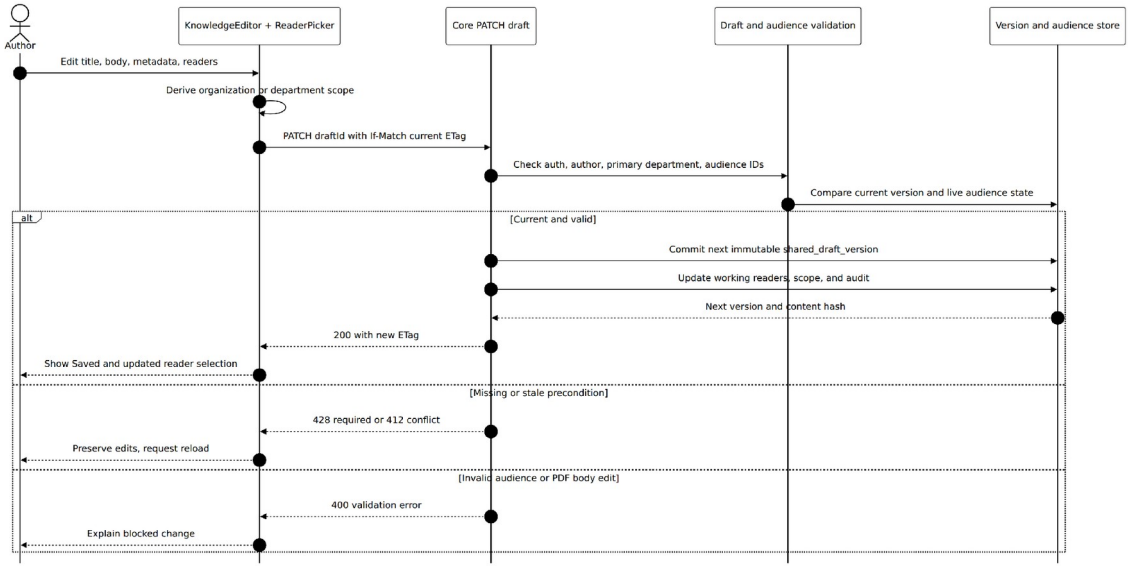


Figure S-08. Save Draft Version and Audience Sequence

Implementation evidence and scope: PATCH requires If-Match and validates live ownership, audience scope, author primary department and content representation. A successful save makes a new working version; stale versions return 412 and missing preconditions 428. PDF body editing is rejected. Evidence: apps/api/src/app.ts PATCH knowledge/drafts/:draftId, knowledge/policy.ts.

1. Author edits title/body or allowed metadata and selects organization/department/group readers.
2. UI derives the target scope and sends PATCH with If-Match set to the displayed current version.
3. Core rechecks author ownership, primary department, audience membership, reader/scope pairing and content validity.
4. Core writes the next immutable working version and updates reader selections; the client refreshes the version.

Exceptions and guarantees: No readers, stale version, invalid scope or unavailable grant conditions surface as actionable errors; stale If-Match returns 412, absent returns 428.

Submit Immutable Review Snapshot Sequence

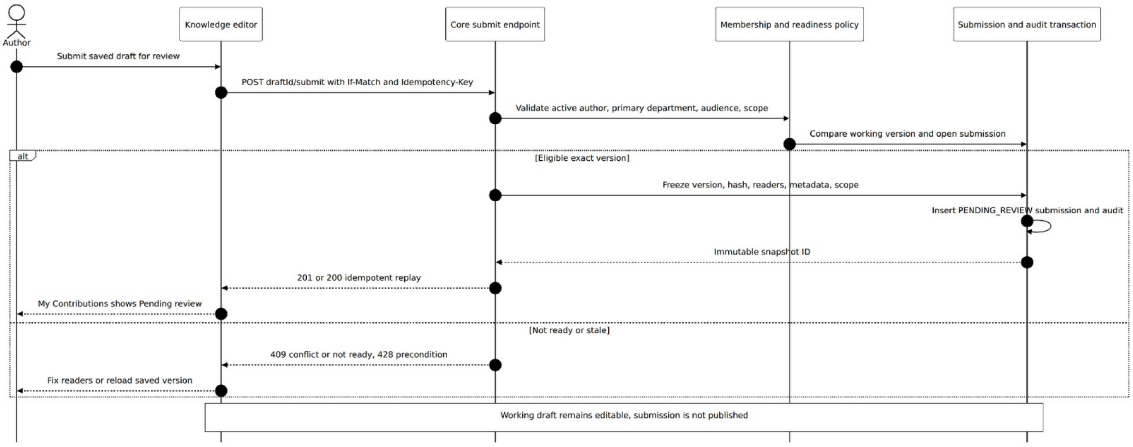


Figure S-09. Submit Immutable Review Snapshot Sequence

Implementation evidence and scope: Submit requires If-Match and Idempotency-Key. The repository freezes one saved version, current content hash, resolved audience and target scope into PENDING_REVIEW with audit evidence; the working draft stays independently editable. Evidence: apps/api/src/app.ts POST knowledge/drafts/:draftId/submit, db/repository.ts.

1. Author selects Submit for review from a saved draft with a valid audience selection.
2. Client sends the current If-Match and Idempotency-Key; Core checks membership, authorship, primary department and readiness.
3. Core freezes exactly one shared_draft_version, content hash, title, target scope and audience into shared_submission.
4. Submission becomes PENDING_REVIEW; author sees it in My Contributions while the separate working draft remains editable.

Exceptions and guarantees: A pending submission is not published or searchable. A second open candidate or replay with conflicting payload is rejected.

3.5 Review Governance and Approval

Assigned reviewers inspect immutable submissions and decide Approve or Reject. Review grants are scope-limited and checked again at queue/list, detail and decision time. Organization Admin has broader decision authority but no automatic reader access.

User-facing entry and screens: Reviews queue; submission preview and decision screen; My Contributions review status.

Code evidence: apps/api/src/knowledge/review-queue.ts, review-decision.ts and policy.ts; docs/engineering/mainflow-2-workflow.md.

3.5.1 Class Diagram

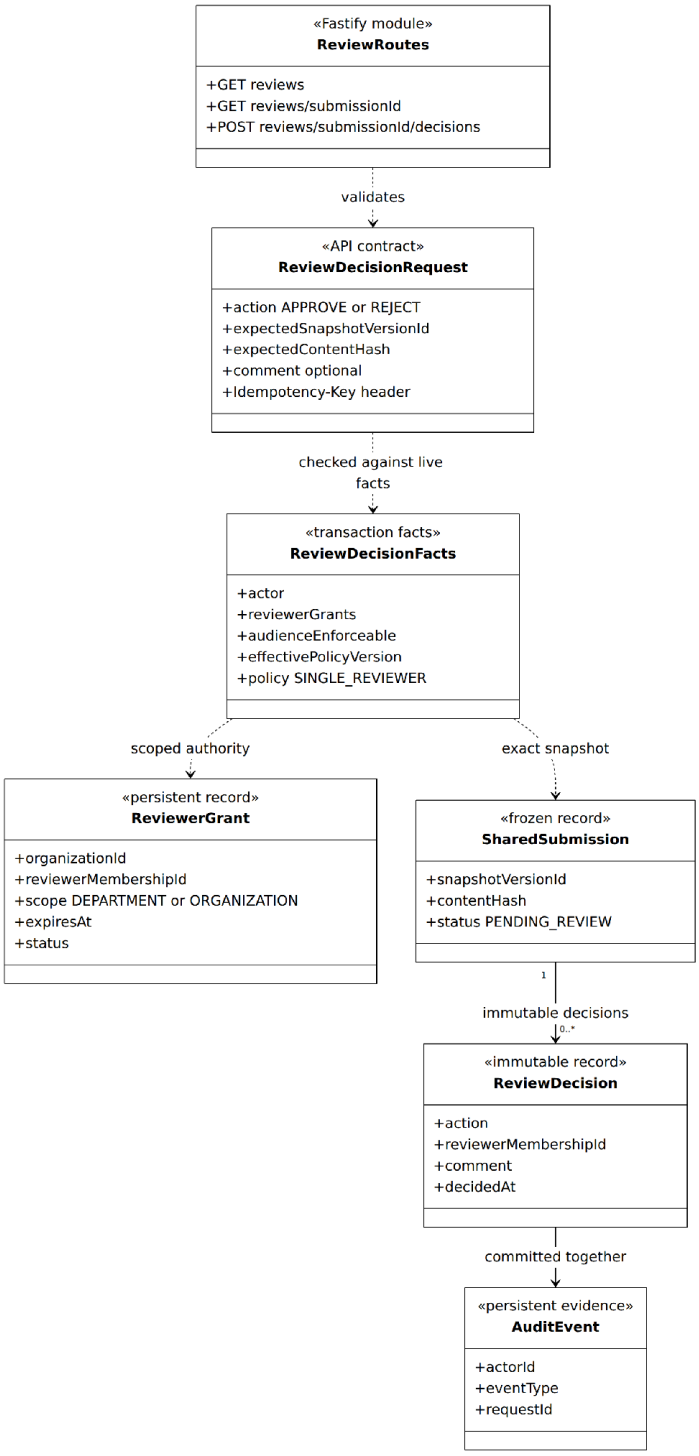


Figure C-05. Reviewer Grant Policy and Decision Contracts

Implementation evidence and scope: Review authorization is computed from live active membership, scoped reviewer grants and immutable snapshot identity, with Org Admin privilege evaluated separately. REJECT requires a comment; APPROVE creates preparation work, not an immediately active publication. Evidence: apps/api/src/knowledge/review-decision.ts, review-queue.ts and db/repository.ts.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
ReviewerGrant	Delegated DEPARTMENT or ORGANIZATION authority, optionally tag/date-scoped.	reviewer_grant; migration 0022
ReviewQueue	Return only eligible pending snapshots; prevent ordinary self-review.	knowledge/review-queue.ts
ReviewDecision	Enforce Approve or Reject and any required reason.	knowledge/review-decision.ts
Audit	Record decision actor, scope and Org Admin self-approval where applicable.	review_decision; audit_event

Reviewer Queue and Snapshot Detail Sequence

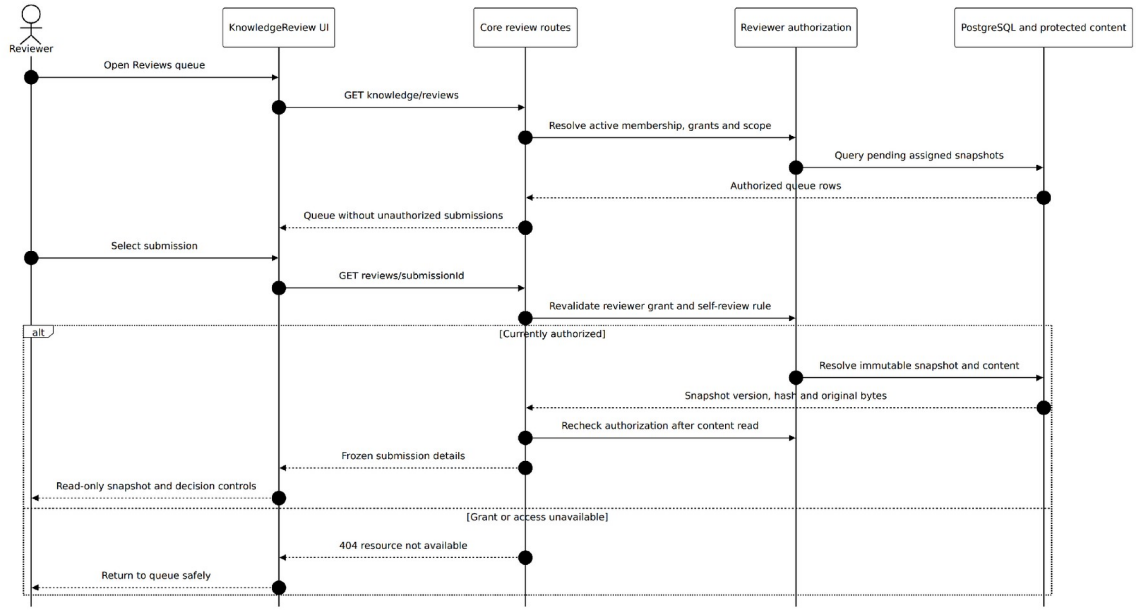


Figure S-10. Reviewer Queue and Snapshot Detail Sequence

Implementation evidence and scope: Review list and detail use repository-level assigned-review queries. Detail first checks reviewer authority, reads the exact frozen source and then repeats the authorization and identity check. Revoked/expired grants return a safe unavailable result. Evidence: apps/api/src/app.ts GET knowledge/reviews and /:submissionId.

1. Reviewer opens Reviews; the client requests only pending assignments for the current organization.
2. Core computes the actor's active grants and audience/scope eligibility, excluding self-submission for ordinary reviewers.
3. Core returns eligible queue cards; selecting one loads the frozen snapshot, source content and readership detail.
4. Any revoked or expired grant is rechecked when loading detail and yields a safe unavailable response.

Exceptions and guarantees: The reviewer views the submitted version, not subsequent edits to the author's working draft.

Approve or Reject Submission Sequence

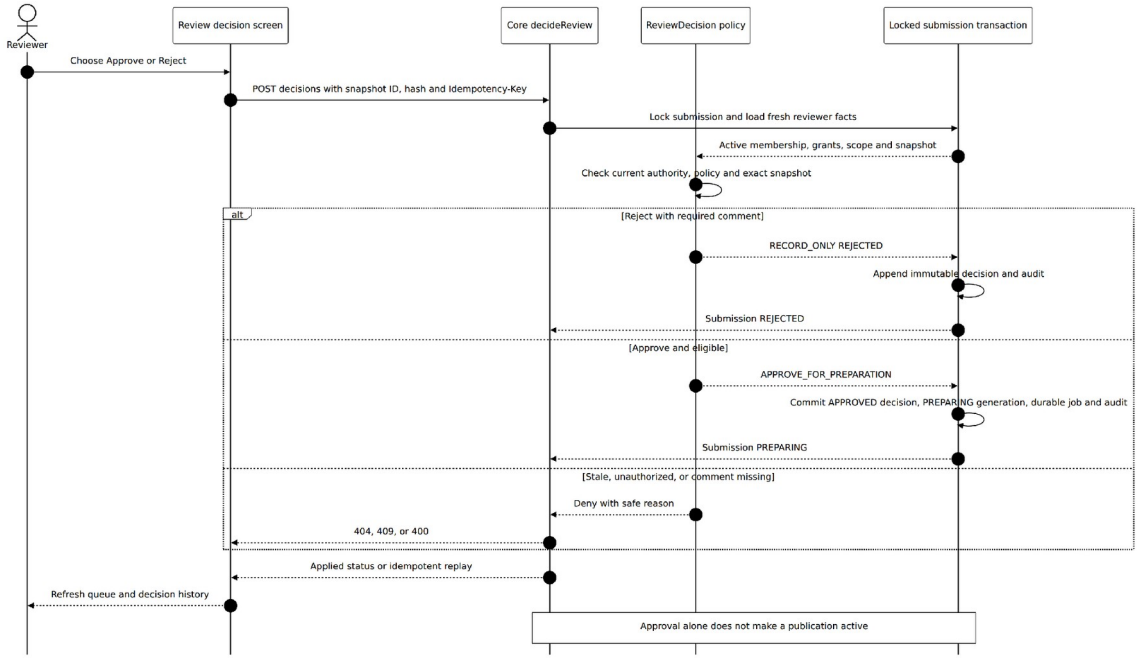


Figure S-11. Approve or Reject Submission Sequence

Implementation evidence and scope: The decision request supplies action, exact snapshotVersionId, contentHash and idempotency key. Within the repository transaction Core locks and reauthorizes the snapshot. Reject records a comment and immutable decision; approve atomically records APPROVED/PREPARING plus a durable index job and audit. Evidence: review-decision.ts and app.ts decision route.

1. Reviewer chooses Approve or Reject for the pending snapshot; Reject requires the required comment.
2. Core checks current role/grant, self-review prohibition for ordinary reviewer and unchanged pending state.
3. Core stores an immutable review_decision and audit event in a transaction.
4. Reject closes the candidate as REJECTED. Approve marks APPROVED / PREPARING and schedules an indexing generation; no active publication changes yet.

Exceptions and guarantees: Org Admin self-approval is permitted with explicit audited selfApproved flag. There is no separate Publisher approval step.

3.5.4 End to End Governance Activity Diagram

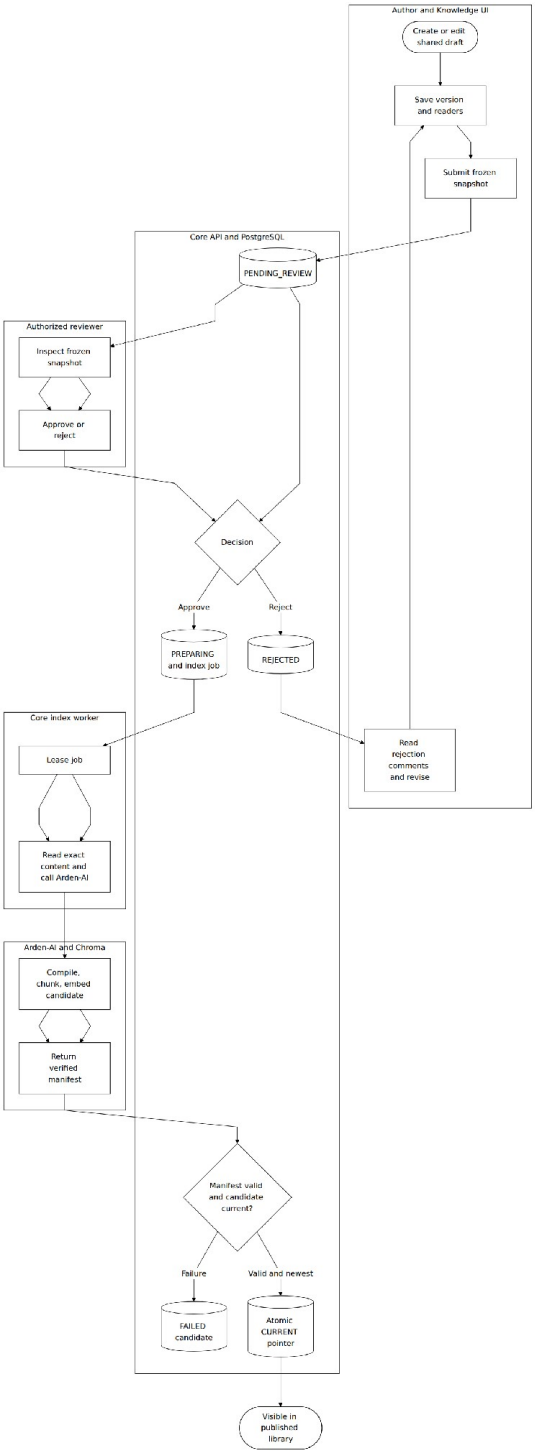


Figure U-03. Author Review Publication Swimlane Activity

Implementation evidence and scope: Human review is the boundary between a frozen submitted snapshot and index preparation. Reject returns the author to a separate working draft; approve creates a durable job. Publication becomes visible only after verified candidate indexing and atomic current-pointer activation. Evidence: review-decision.ts, indexing/worker.ts, indexing/store.ts and repository.ts.

3.5.5 Knowledge Submission State Machine

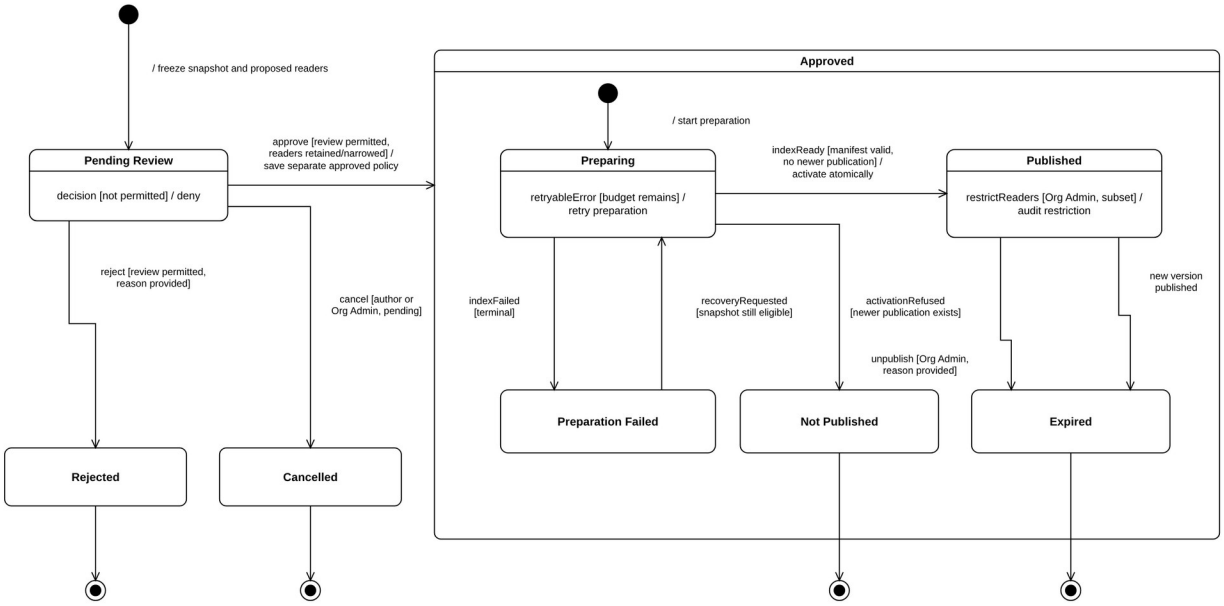


Figure M-03. Knowledge Submission State Machine

Implementation evidence and scope: shared_submission.status is PENDING_REVIEW, APPROVED, REJECTED or WITHDRAWN, and an approved submission carries publication_state PREPARING, ACTIVE, FAILED or SUPERSEDED (migrations 0018 and 0021). The diagram maps to these values as follows: Pending Review = PENDING_REVIEW; Rejected = REJECTED; Cancelled = WITHDRAWN; Preparing, Published and Preparation Failed = APPROVED with PREPARING, ACTIVE and FAILED; Not Published and Expired = SUPERSEDED, i.e. a newer publication was activated before or after this one. Implemented on staging: submit, approve, reject, retry inside Preparing, terminal failure and supersession. Designed but not yet exposed: cancel (WITHDRAWN exists in the schema but has no route), recoveryRequested, restrictReaders and unpublish (planned, UC-29).

3.6 Indexing and Atomic Publication

After a human approves a frozen snapshot, Core prepares a durable indexing generation. Arden-AI produces a manifest for that exact source. Only Core may activate the candidate as the current publication once the generation is complete and the policy checks still pass.

User-facing entry and screens: Author submission status: Preparing, Active or Failed; reviewer decision history; published knowledge appears only after activation.

Code evidence: apps/api/src/indexing/worker.ts, store.ts and ai-client.ts; Arden-AI /v1/index-generations; migrations 0018–0021.

3.6.1 Class Diagram

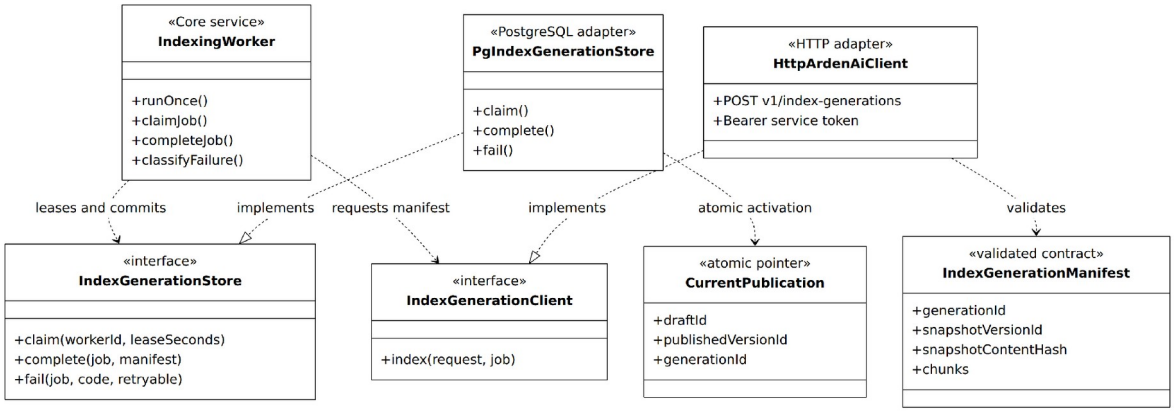


Figure C-06. Durable Index Job and Publication Contracts

Implementation evidence and scope: IndexingWorker uses a PostgreSQL-backed IndexGenerationStore for leases and transactional completion/failure, plus an HTTP Arden-AI client with service-token authentication. A manifest must match the frozen source before the current publication pointer can change. Evidence: apps/api/src/indexing/{worker,store,ai-client,contracts}.ts.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
IndexGeneration/IndexJob	Durable status and job leasing for one immutable approved snapshot.	knowledge_index_generation; knowledge_index_job
Core worker	Lease pending jobs, call AI service, verify manifest and complete/fail generation.	indexing/worker.ts
Arden-AI indexing	Compile source, split sections, embed, persist Chroma candidate, return manifest.	POST /v1/index-generations
Activation transaction	Swap current pointer only when manifest is complete and candidate is current.	knowledge_current_publication; migration 0021

Approved Snapshot Index and Activate Sequence

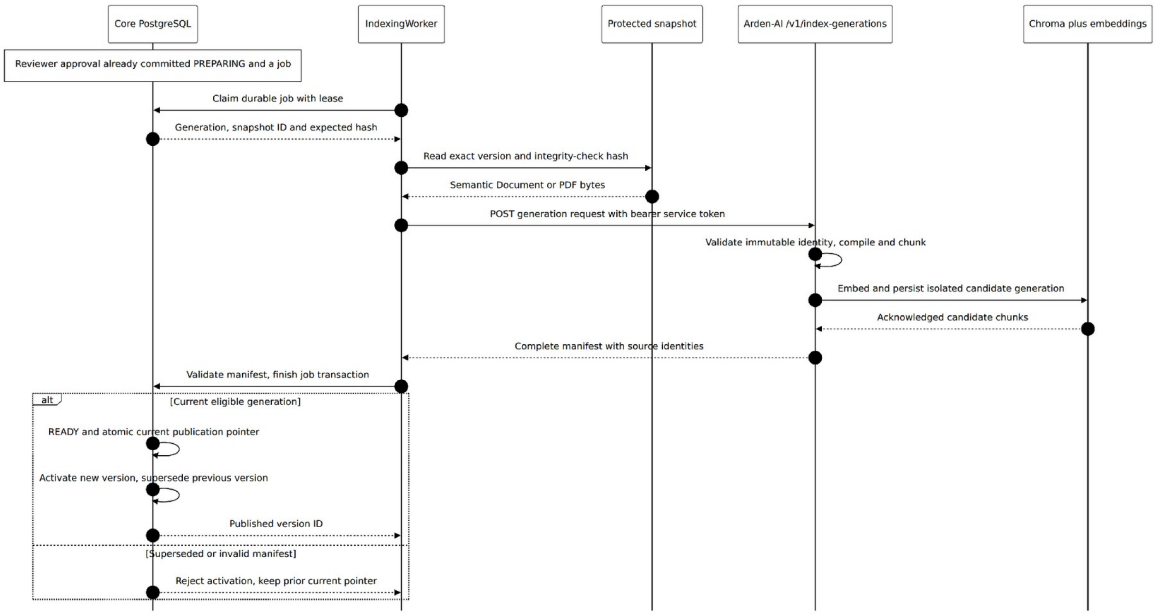


Figure S-12. Approved Snapshot Index and Activate Sequence

Implementation evidence and scope: The worker claims an approved PREPARING job, verifies exact snapshot bytes, calls Arden-AI /v1/index-generations and receives a manifest after successful candidate indexing. Core validates identities and atomically activates the new current publication while superseding the old version. Evidence: apps/api/src/indexing/worker.ts, store.ts; Arden-AI application/indexing.py and presentation/api.py.

1. Approval commits the PREPARING snapshot and a durable index generation/job.
2. Worker leases the job and sends the exact version/hash/organization to Arden-AI using the internal service token.
3. AI deterministically compiles Semantic Document or PDF text, chunks it, obtains embeddings and persists the candidate generation in Chroma.
4. AI returns a manifest only for a completely acknowledged generation; Core validates snapshot identity and manifest consistency.
5. Core commits READY and atomically updates knowledge_current_publication; submission becomes ACTIVE and the previous active snapshot becomes SUPERSEDED where relevant.

Exceptions and guarantees: Partial embeddings are not activated. An older delayed generation must never replace a newer active publication.

Index Failure Retry and Supersession Sequence

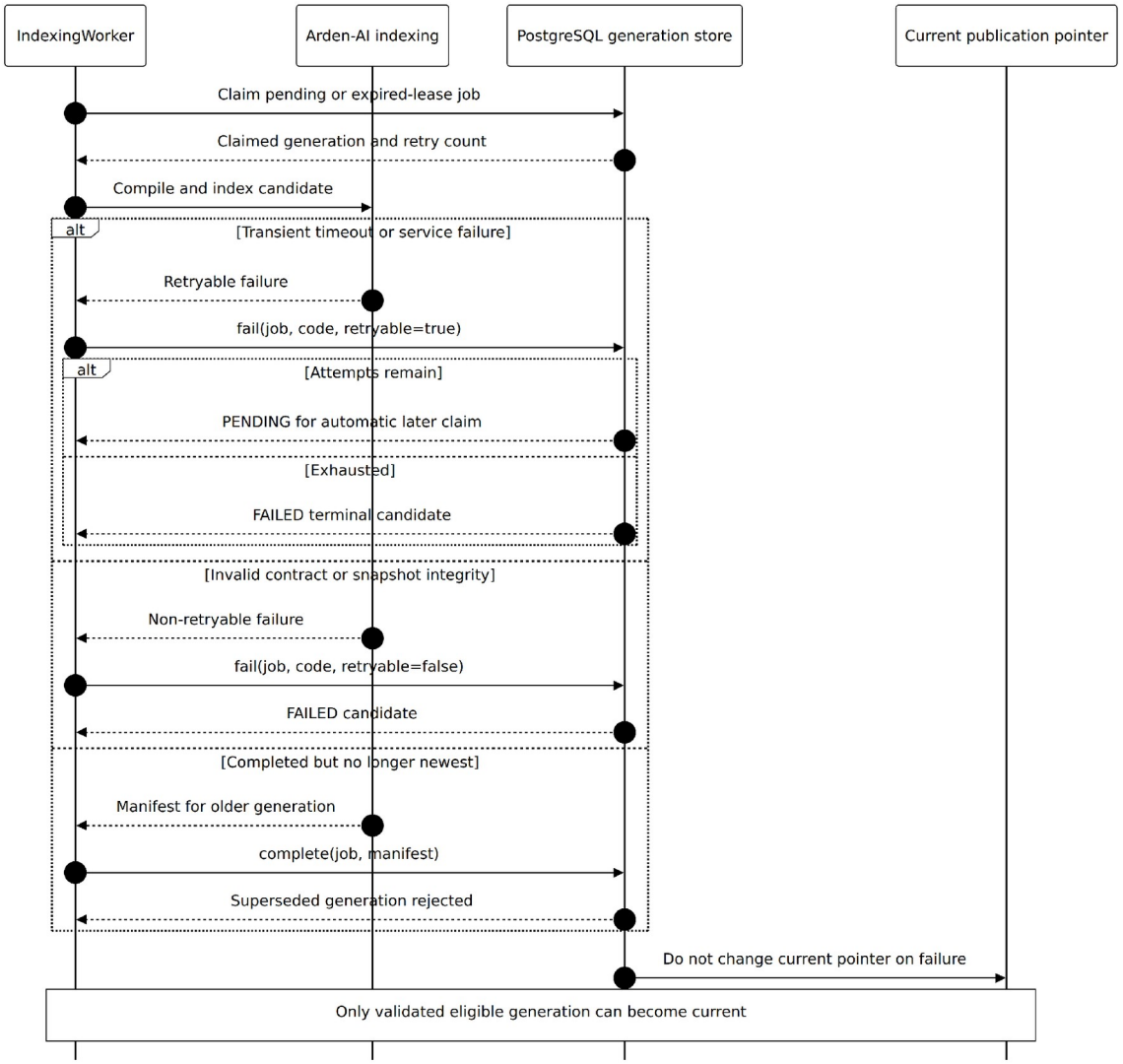


Figure S-13. Index Failure Retry and Supersession Sequence

Implementation evidence and scope: Retryable worker failures may return a job to PENDING until attempts are exhausted; nonretryable or exhausted candidates become FAILED. Neither failure nor late superseded results move the existing current publication pointer. This diagram does not claim a deployed manual retry button. Evidence: apps/api/src/indexing/worker.ts and store.ts.

1. Worker observes model/vector/manifest failure or lease exhaustion and records the failed attempt.
2. Core marks the generation FAILED and the associated approved submission publication_state FAILED.
3. Existing knowledge_current_publication stays unchanged, so the new failed candidate is absent from library and AI answers.
4. On an authorized retry or recovery, the worker rechecks current eligibility; a candidate superseded by newer publication cannot overwrite the current pointer.

Exceptions and guarantees: Distinguish transient retry from terminal failure and do not imply a working manual retry button unless the current UI provides one.

3.6.4 Index Generation State Machine

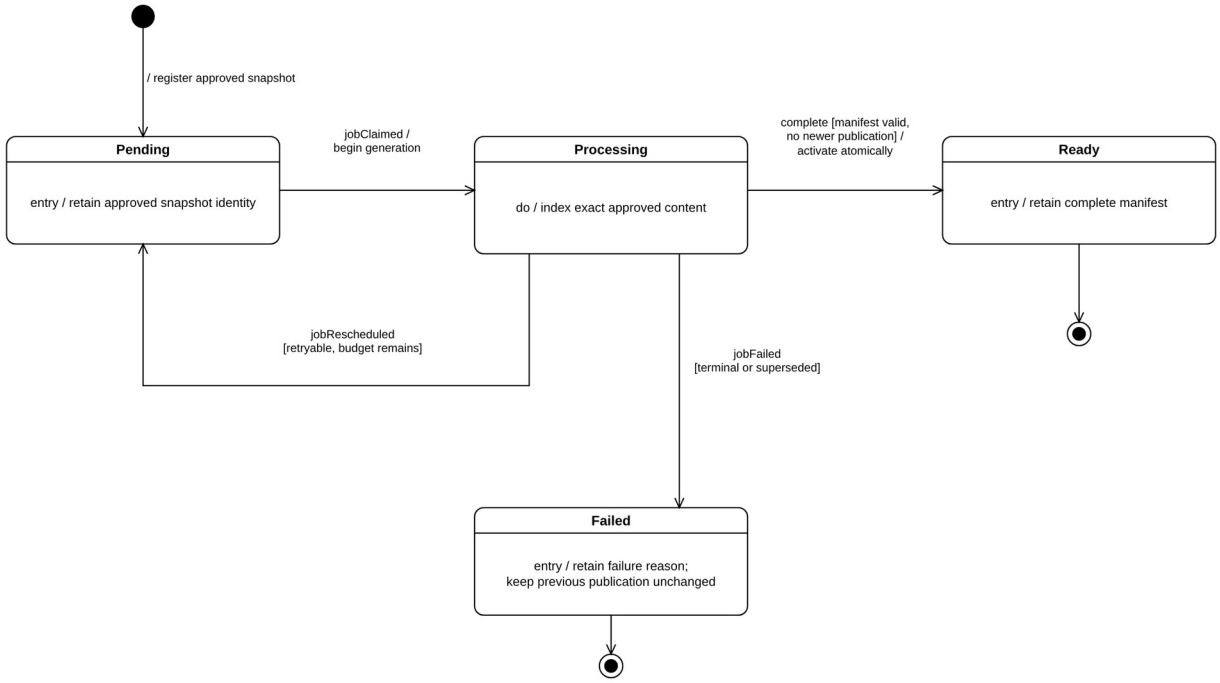


Figure M-04. Knowledge Index Generation State Machine

Implementation evidence and scope: knowledge_index_generation.status is PENDING, PROCESSING, READY or FAILED, and its job is PENDING, RUNNING, SUCCEEDED or FAILED (migration 0019). The worker leases a pending job (Pending → Processing). A retryable error reschedules the job and returns the generation to PENDING while attempts remain (max_attempts = 3). A complete, valid manifest with no newer publication is activated atomically (Ready; the submission becomes ACTIVE). A terminal error, exhausted attempts or a newer publication ends in Failed (failure_code INDEX_GENERATION_SUPERSEDED in the last case) and the previous current publication stays readable. Evidence: apps/api/src/indexing/worker.ts and store.ts; migrations 0019 and 0021.

3.7 Published Knowledge Library

Members browse only the currently active knowledge versions for which their present audience membership gives readership. Published material is official only after automatic index-ready activation.

User-facing entry and screens: Knowledge → Library / published document detail; citation deep-link target.

Code evidence: apps/api/src/app.ts published list/detail routes; packages/ui/src/knowledge/KnowledgeDocumentDetail.tsx and ServerKnowledgeFeaturePage.tsx.

List and Open Authorized Publication Sequence

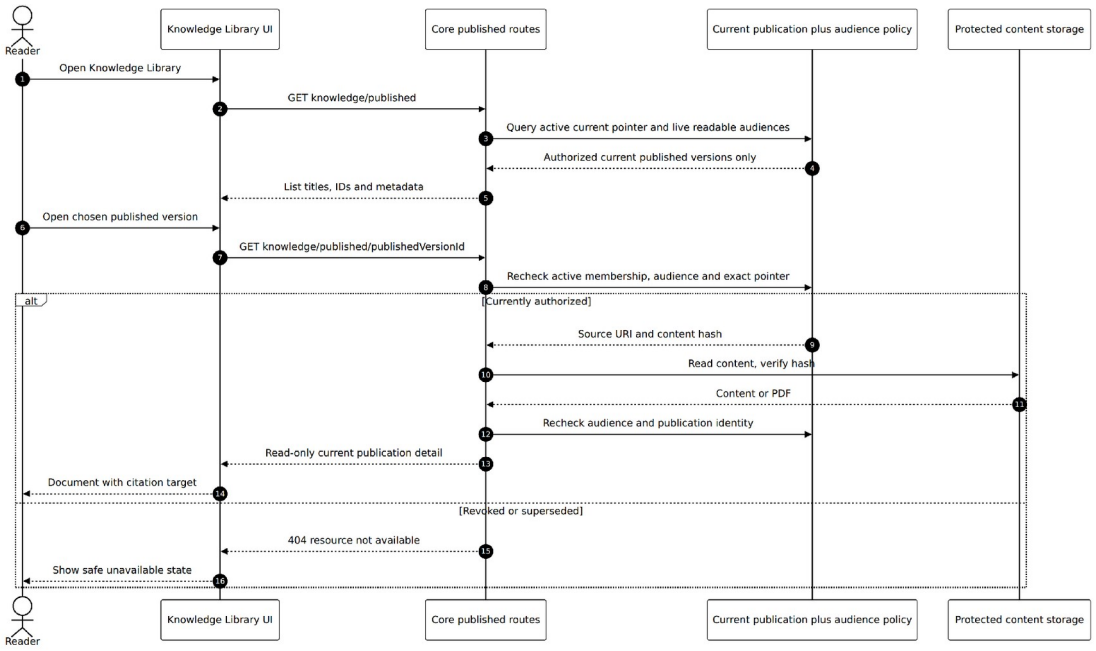


Figure S-14. List and Open Authorized Publication Sequence

Implementation evidence and scope: Published list and detail endpoints use the active publication pointer and current reader audience policy, not the mere existence of a stored document. Detail reauthorizes after reading protected bytes; revoked and superseded versions are not exposed. Evidence: apps/api/src/app.ts /knowledge/published routes and knowledge/policy.ts.

1. Member opens the published library for the selected organization.
2. Core checks live membership/audience and joins each published record through knowledge_current_publication.
3. Only active and readable current records appear; selecting a record rechecks access to exact version/content.
4. UI shows the read-only document and provenance/citation target; unavailable or revoked items return a safe error.

Exceptions and guarantees: Historical PREPARING, FAILED, REJECTED, SUPERSEDED or merely authored drafts must not leak through list, detail or citation routes.

3.8 Ask Arden with Permission Aware Citations

A Member asks a question in Arden. Core calculates a current authorized publication-generation allow-list, Arden-AI retrieves only those Chroma candidates, and Core revalidates returned evidence before exposing the grounded answer and source links.

User-facing entry and screens: Ask Arden page; question input, response and clickable citations to published detail.

Code evidence: packages/ui/src/ask-arden/AskArdenPage.tsx; apps/api/src/answers/*; Arden-AI /v1/answers.

3.8.1 Class Diagram

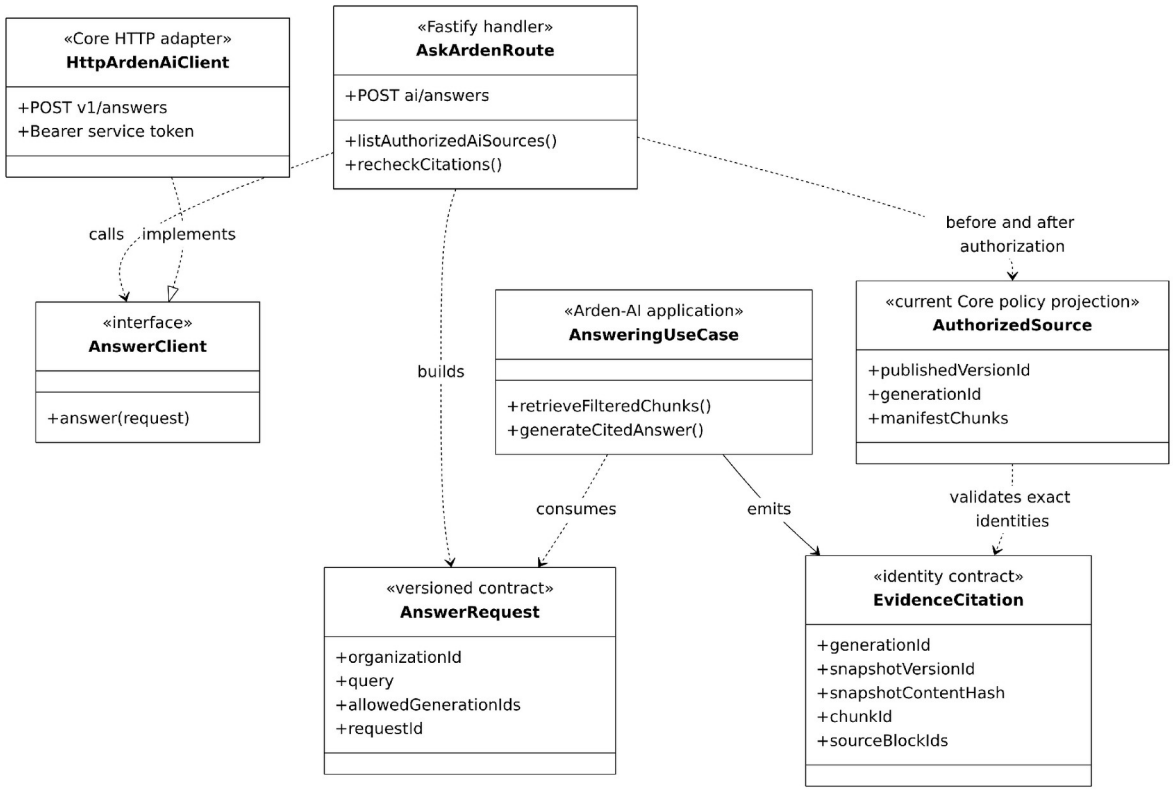


Figure C-07. Permission Aware Answer and Citation Contracts

Implementation evidence and scope: Core sends an allowedGenerationIds list derived from current AccessPolicy to Arden-AI. Answer evidence retains generation, immutable snapshot and manifest chunk identities. Core rechecks those identities and live publication access after the AI response. Evidence: apps/api/src/app.ts /ai/answers, answers/contracts.ts, Arden-AI application/answering.py.

Relevant implementation modules and responsibilities

Module / contract	Responsibility	Relevant boundary
Core answer endpoint	Validate question, resolve actor and organization, derive active permitted generation IDs.	POST /api/v1/ai/answers
Arden-AI answers	Filter Chroma by organization and generation allow-list, assemble context and generate grounded response.	POST /v1/answers
Evidence contract	Return exact generation/chunk/source identifiers and citation offsets.	Arden-AI/contracts; answers/contracts.ts
Citation resolution	Recheck current membership, active pointer, audience access and source page before returning links.	answers/citation-page.ts

Ask Arden Answer Retrieval and Evidence Recheck Sequence

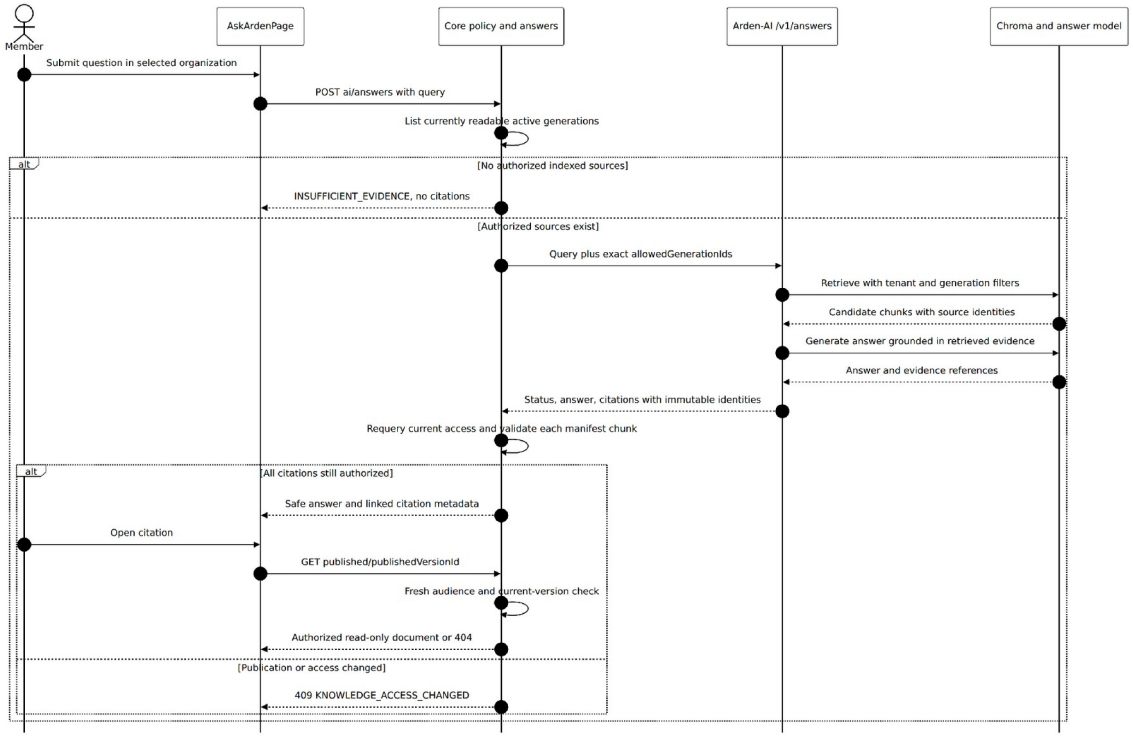


Figure S-15. Ask Arden Answer Retrieval and Evidence Recheck Sequence

Implementation evidence and scope: Ask Arden first queries currently authorized index generations. Arden-AI filters retrieval by organization and generation; after answer generation Core rechecks the cited manifest chunks and access. If access changed it returns 409; opening a citation performs another published-detail authorization check. Evidence: apps/api/src/app.ts /ai/answers and /knowledge/published/:publishedVersionId; Arden-AI application/answering.py.

1. User sends a question in Ask Arden; Core authenticates and derives current organization and active readable generation IDs.
2. Core calls Arden-AI with that allow-list; AI applies organization and generation filters in Chroma and assembles cited candidate chunks.
3. AI generates a response with evidence references; Core checks that each exact evidence identity remains authorized and current.
4. Core returns the answer and safe citation metadata; the UI renders source links to the read-only published document.
5. Opening a citation runs a fresh access check against the current publication and audience state.

Exceptions and guarantees: No eligible evidence or revoked access must not be invented as cited facts. Current Chroma index is not the source of access policy.

3.8.4 Ask Arden Activity Diagram

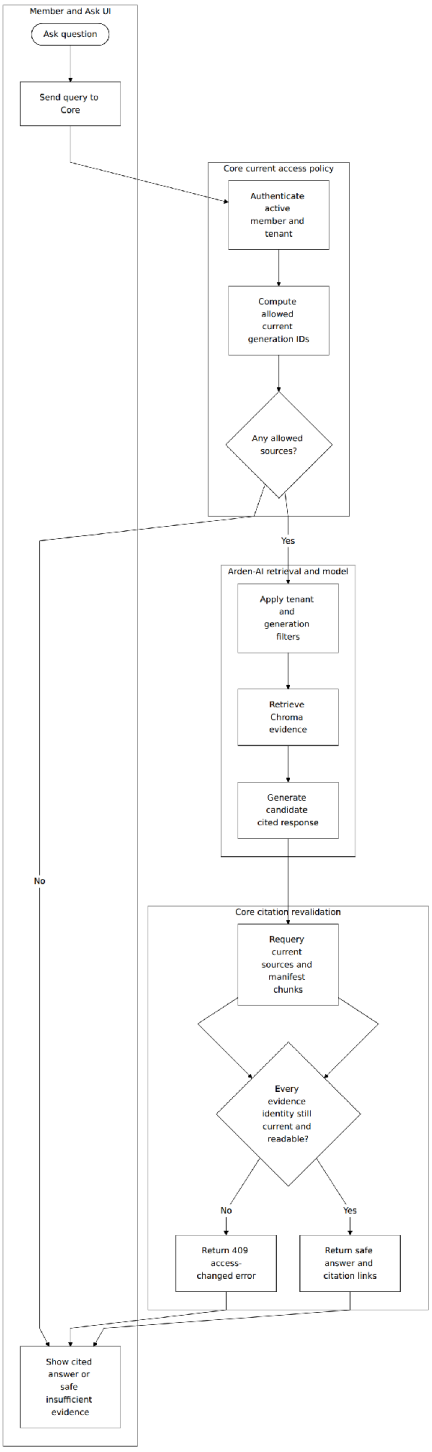


Figure U-04. Ask Arden Permission Filtering Swimlane Activity

Implementation evidence and scope: A question with no authorized indexed sources immediately returns INSUFFICIENT_EVIDENCE. Otherwise AI retrieval is constrained to allowed generations and every returned citation is rechecked against live current publication metadata. Evidence: apps/api/src/app.ts /ai/answers and Arden-AI application/answering.py.

3.9 Installation System Administration and Security Boundaries

A separately privileged Arden operations user can administer installation-level status and administrator lifecycle without becoming a member of arbitrary customer organizations or gaining the right to read, review or publish their documents.

User-facing entry and screens: System Administration interfaces where exposed; organization administration is a separate screen and capability.

Code evidence: apps/api/src/system-administration; migration 0017; docs/decisions/0010-audiences-and-system-administration.md.

Installation Administrator Authority and Audit Sequence

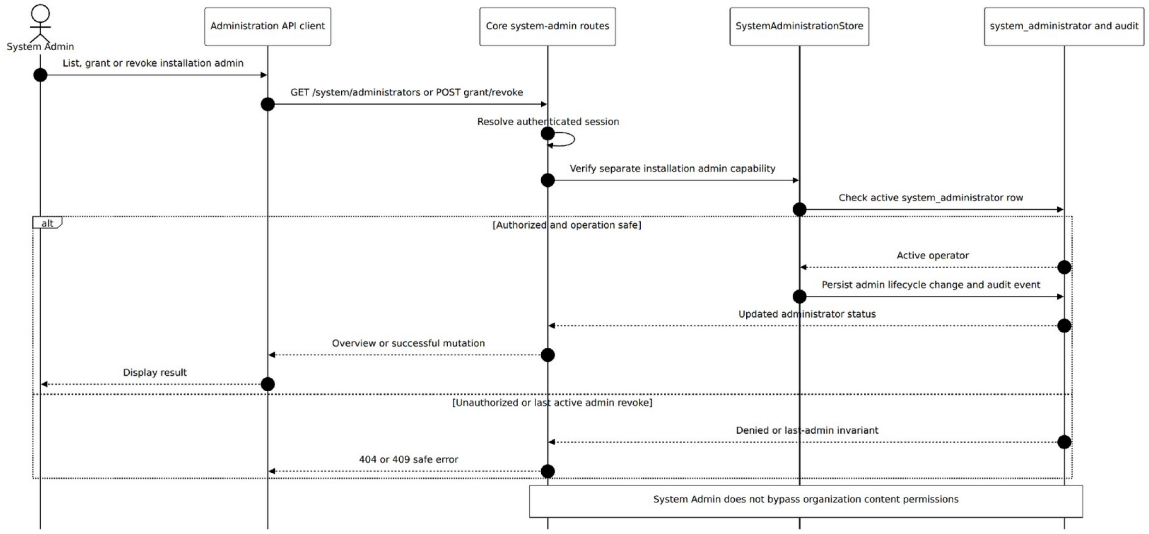


Figure S-16. Installation Administrator Authority and Audit Sequence

Implementation evidence and scope: Installation administrator CRUD is supported by separate Core /api/v1/system/administrators API routes backed by system_administrator records and system_administration_event audit. The last active administrator cannot be revoked. This role does not grant organization knowledge access and is not presented as a fully rolled-out UI. Evidence: apps/api/src/system-administration/routes.ts and store.ts.

1. Authenticated installation operator calls a designated system-administration route.
2. Core verifies an active system_administrator row rather than an organization membership grant.
3. An allowed installation-level operation updates its own lifecycle state and records system_administration_event.
4. A content/list/review request still requires independent organization membership and content policy; installation capability supplies no bypass.

Exceptions and guarantees: The API foundation is represented in code, but this report does not claim full public System Admin UI rollout.